

TECNOLOGIA DA INFORMAÇÃO

SQL (DML)



Presidente: Gabriel Granjeiro

Vice-Presidente: Rodrigo Calado

Diretor Pedagógico: Erico Teixeira

Diretora de Produção Educacional: Vivian Higashi

Gerente de Produção Digital: Bárbara Guerra

Todo o material desta apostila (incluindo textos e imagens) está protegido por direitos autorais do Gran. Será proibida toda forma de plágio, cópia, reprodução ou qualquer outra forma de uso, não autorizada expressamente, seja ela onerosa ou não, sujeitando-se o transgressor às penalidades previstas civil e criminalmente.

CÓDIGO:

251023529851



PATRÍCIA QUINTÃO

Mestre em Engenharia de Sistemas e computação pela COPPE/UFRJ, Especialista em Gerência de Informática e Bacharel em Informática pela UFV. Atualmente é professora no Gran Cursos Online; Analista Legislativo (Área de Governança de TI), na Assembleia Legislativa de MG; Escritora e Personal & Professional Coach. Atua como professora de Cursinhos e Faculdades, na área de Tecnologia da Informação, desde 2008. É membro: da Sociedade Brasileira de Coaching, do PMI, da ISACA, da Comissão de Estudo de Técnicas de Segurança (CE-21:027.00) da ABNT, responsável pela elaboração das normas brasileiras sobre gestão da Segurança da Informação. Autora dos livros: Informática FCC - Questões comentadas e organizadas por assunto, 3ª. edição e 1001 questões comentadas de informática (Cespe/UnB), 2ª. edição, pela Editora Gen/Método. Foi aprovada nos seguintes concursos: Analista Legislativo, na especialidade de Administração de Rede, na Assembleia Legislativa do Estado de MG; Professora titular do Departamento de Ciência da Computação do Instituto Federal de Educação, Ciência e Tecnologia; Professora substituta do DCC da UFJF; Analista de TI/Suporte, PRODABEL; Analista do Ministério Público MG; Analista de Sistemas, DATAPREV, Segurança da Informação; Analista de Sistemas, INFRAERO; Analista - TIC, PRODEMGE; Analista de Sistemas, Prefeitura de Juiz de Fora; Analista de Sistemas, SERPRO; Analista Judiciário (Informática), TRF 2ª Região RJ/ES, etc.

GRAN
CONCURSOS

SUMÁRIO

Apresentação	4
SQL (DML)	5
Termos, Definições e Abreviaturas	5
Introdução	7
SQL (DML): Fundamentos da Consulta de Dados	12
Análise de Dados: Funções Agregadas e Agrupamentos	22
Relacionamentos entre Tabelas: JOINS e Operações de Conjunto	29
Comandos de Manipulação: INSERT, UPDATE e DELETE	51
Lógica de Três Estados e Tratamento de NULL	58
Resumo	67
Exercícios	74
Gabarito	89
Gabarito Comentado	90
Referências	128

APRESENTAÇÃO

Olá, querido(a) amigo(a), tudo bem?

“O Poder do Entusiasmo”: descubra o entusiasmo na vida!!

Nós somos capazes de transformar as coisas e fazê-las acontecer. Faça o entusiasmo ocorrer pela certeza de que você é capaz de realizações eficazes e de... **VENCER OBSTÁCULOS E SER FELIZ!!**

Traço aqui a aula sobre **SQL (DML)** e tópicos relacionados. Espero que aproveite!

Continue firme nessa caminhada e ótimos estudos!

Um abraço,

Profª Patrícia Lima Quintão

SQL (DML)

TERMOS, DEFINIÇÕES E ABREVIATURAS

A seguir destacamos alguns termos/definições e abreviaturas:

- Alias – Nome alternativo temporário atribuído a colunas ou tabelas durante a execução de uma consulta para melhorar legibilidade;
- AND – Operador lógico que retorna TRUE apenas quando ambas as condições são verdadeiras;
- ANSI – American National Standards Institute, organização que padronizou a linguagem SQL;
- BETWEEN – Operador que verifica se um valor está dentro de um intervalo específico (inclusive);
- COUNT – Função agregada que conta o número de linhas ou valores não NULL em um conjunto de resultados;
- CROSS JOIN – Tipo de junção que retorna o produto cartesiano entre duas tabelas;
- DCL – *Data Control Language*, sublinguagem SQL responsável pelo controle de acesso e segurança;
- DDL – *Data Definition Language*, sublinguagem SQL responsável pela definição e modificação de estruturas;
- DELETE – Comando DML usado para remover registros específicos de uma tabela;
- DISTINCT – Palavra-chave que elimina linhas duplicadas do resultado de uma consulta;
- DML – *Data Manipulation Language*, sublinguagem SQL responsável pela manipulação de dados;
- DQL – *Data Query Language*, classificação específica para o comando SELECT em algumas abordagens;
- DTL – *Data Transaction Language*, sublinguagem SQL responsável pelo controle de transações;
- EXISTS – Operador que testa se uma subconsulta retorna pelo menos um registro;
- EXCEPT – Operação de conjunto que retorna linhas da primeira consulta que não aparecem na segunda;
- FALSE – Um dos três valores possíveis na lógica de três estados do SQL (TRUE, FALSE, UNKNOWN);
- FETCH FIRST – Cláusula padrão SQL para limitar o número de registros retornados (Oracle 12c+);
- FROM – Cláusula que especifica de qual(is) tabela(s) os dados serão recuperados;

- FULL OUTER JOIN – Tipo de junção que retorna todas as linhas de ambas as tabelas, preenchendo com NULL onde não há correspondência;
- GROUP BY – Cláusula que agrupa linhas com valores idênticos em colunas especificadas;
- HAVING – Cláusula que filtra grupos criados por GROUP BY, similar ao WHERE mas para grupos;
- IN – Operador que verifica se um valor está presente em uma lista ou subconsulta;
- INNER JOIN – Tipo de junção que retorna apenas registros com correspondência em ambas as tabelas;
- INSERT – Comando DML usado para adicionar novos registros a uma tabela;
- INTERSECT – Operação de conjunto que retorna apenas linhas que aparecem em ambas as consultas;
- IS NULL – Operador usado para testar se um valor é NULL;
- IS NOT NULL – Operador usado para testar se um valor não é NULL;
- JOIN – Operação que combina linhas de duas ou mais tabelas baseada em uma condição relacionada;
- LEFT JOIN – Tipo de junção que retorna todas as linhas da tabela esquerda e correspondências da direita;
- LIKE – Operador usado para busca de padrões em strings usando wildcards (% e _);
- MAX – Função agregada que retorna o maior valor de um conjunto, ignorando valores NULL;
- MIN – Função agregada que retorna o menor valor de um conjunto, ignorando valores NULL;
- NOT – Operador lógico que inverte o valor lógico de uma condição;
- NOT EXISTS – Operador que testa se uma subconsulta não retorna nenhum registro;
- NULL – Valor especial que representa ausência de dados ou valor desconhecido;
- ON – Palavra-chave usada para especificar a condição de junção em JOINS;
- OR – Operador lógico que retorna TRUE se pelo menos uma das condições for verdadeira;
- ORDER BY – Cláusula que especifica como os resultados devem ser ordenados;
- RIGHT JOIN – Tipo de junção que retorna todas as linhas da tabela direita e correspondências da esquerda;
- SELECT – Comando usado para recuperar dados de uma ou mais tabelas (classificação controversa entre DML/DQL);
- SELF JOIN – Técnica de junção de uma tabela consigo mesma, útil para estruturas hierárquicas;
- SGBD – Sistema de Gerenciamento de Banco de Dados;
- SQL – *Structured Query Language*, linguagem padrão para acesso e manipulação de bancos relacionais;

- SUM – Função agregada que calcula a soma de valores numéricos, ignorando valores NULL;
- TRUE – Um dos três valores possíveis na lógica de três estados do SQL;
- TRUNCATE – Comando DDL que remove todos os registros de uma tabela mantendo sua estrutura;
- UNION – Operação de conjunto que combina resultados de consultas removendo duplicatas automaticamente;
- UNION ALL – Operação de conjunto que combina resultados de consultas preservando todas as linhas;
- UNKNOWN – Terceiro valor lógico no SQL, representado por NULL em comparações;
- UPDATE – Comando DML usado para modificar dados existentes em uma tabela;
- WHERE – Cláusula que especifica condições para filtrar registros (equivale à seleção da álgebra relacional).

INTRODUÇÃO

1. O que é SQL

A **SQL (Structured Query Language)**, ou **Linguagem de Consulta Estruturada** foi desenvolvida originalmente pela IBM na década de 1970 e tornou-se a **linguagem padrão para acesso e manipulação de bancos de dados relacionais**, sendo padronizada pela ANSI (American National Standards Institute) e ISO (International Organization for Standardization).

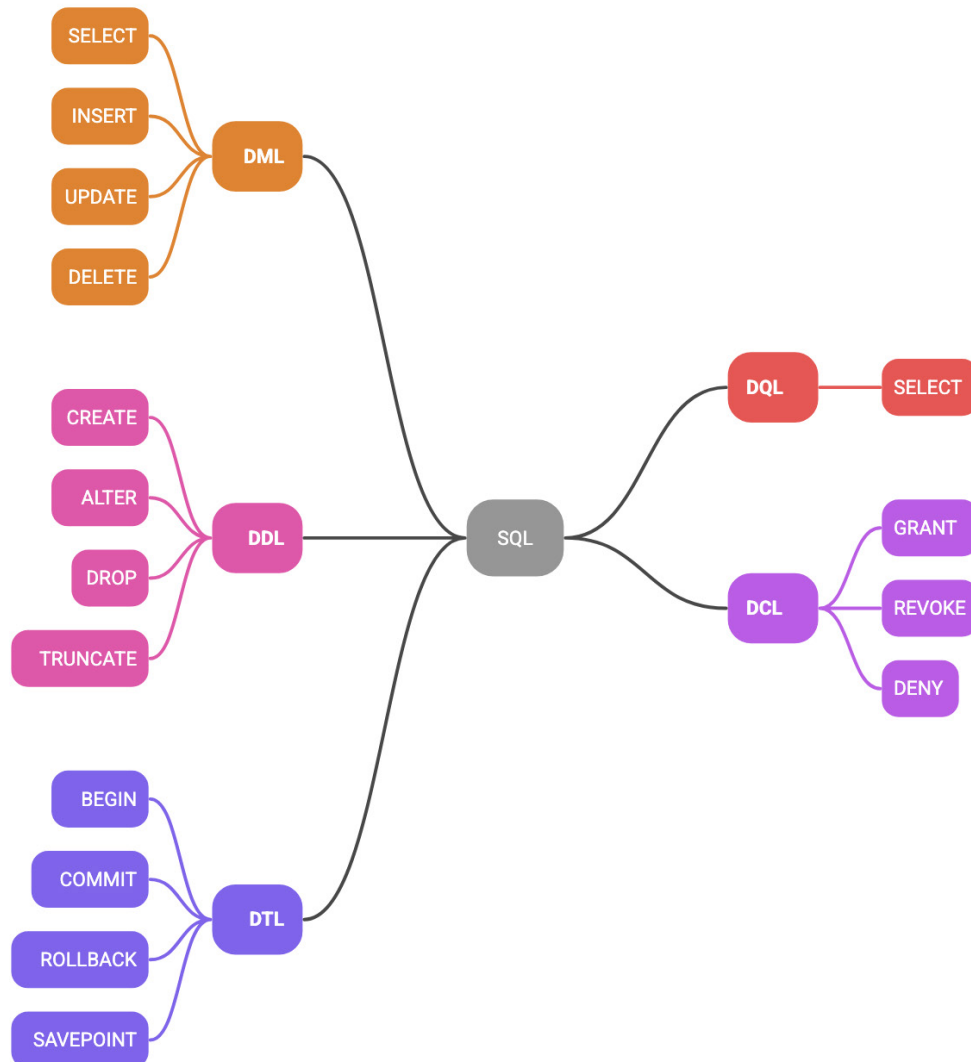
A SQL possui características que a tornam única e amplamente adotada no mercado de trabalho e, conseqüentemente, nas provas de concursos públicos. Primeiramente, trata-se de uma **linguagem declarativa ou não procedural**, o que significa que o usuário expressa o que deseja obter dos dados, sem necessidade de especificar como o sistema deve executar a operação. Esta característica contrasta com linguagens procedurais, onde é necessário descrever passo a passo o algoritmo para alcançar o resultado desejado.

Além disso, a SQL é considerada uma **linguagem de alto nível**, permitindo que operações complexas sejam realizadas com comandos relativamente simples e intuitivos. Sua **sintaxe baseada em palavras do idioma inglês** facilita a compreensão e memorização, sendo bastante intuitiva.

2. As Sublinguagens do SQL

A linguagem SQL **não constitui um bloco monolítico, mas um conjunto organizado de sublinguagens**, cada uma com propósitos específicos e bem definidos. Esta divisão conceitual é fundamental para a compreensão adequada da SQL e representa um dos tópicos mais cobrados em questões conceituais de concursos.

Estrutura das Linguagens SQL



2.1. DML – Data Manipulation Language (Linguagem de Manipulação de Dados)

A **DML** constitui o coração operacional da SQL, sendo responsável pela **manipulação direta dos dados armazenados nas tabelas**. Seus comandos permitem a **inserção, atualização, exclusão e**, segundo algumas classificações, a **consulta de registros**.

Comandos principais da DML:

- **INSERT:** Adiciona novos registros às tabelas;
- **UPDATE:** Modifica registros existentes;
- **DELETE:** Remove registros das tabelas;
- **SELECT:** Recupera dados das tabelas (Classificação controversa).

ATENÇÃO

A DML opera exclusivamente sobre os **dados** das tabelas, sem afetar suas estruturas ou definições. Esta característica a distingue claramente da DDL, que trabalha com a estrutura dos objetos do banco de dados.

2.2. DDL – Data Definition Language (Linguagem de Definição de Dados)

A **DDL** é responsável pela **definição e modificação da estrutura dos objetos do banco de dados**. Enquanto a DML trabalha com o conteúdo das tabelas, a DDL **define como essas tabelas são criadas, alteradas ou removidas**.

Comandos principais da DDL:

- **CREATE**: cria novos objetos (tabelas, índices, views etc.);
- **ALTER**: modifica a estrutura de objetos existentes;
- **DROP**: remove objetos do banco de dados;
- **TRUNCATE**: remove todos os registros de uma tabela (estrutura permanece).

Sublinguagens relacionadas à DDL:

- **VDL** (View Definition Language): especializada na definição de visões;
- **SDL** (Storage Definition Language): focada na especificação do armazenamento físico.

2.3. DCL – Data Control Language (Linguagem de Controle de Dados)

A **DCL** gerencia os aspectos de **segurança e autorização no banco de dados**, controlando **quem pode acessar quais dados e que operações podem ser realizadas**.

Comandos principais da DCL:

- **GRANT**: concede permissões a usuários ou grupos;
- **REVOKE**: remove permissões previamente concedidas;
- **DENY**: nega explicitamente permissões (específico de alguns SGBDs).

2.4. DTL – Data Transaction Language (Linguagem de Transação de Dados)

A **DTL** **controla o comportamento das transações**, garantindo a integridade e consistência dos dados durante operações que envolvem múltiplas modificações.

Comandos principais da DTL:

- **COMMIT**: confirma permanentemente as alterações da transação;
- **ROLLBACK**: desfaz todas as alterações da transação atual;
- **SAVEPOINT**: cria pontos de salvamento dentro de uma transação;
- **BEGIN/START TRANSACTION**: inicia uma nova transação.

2.5. DQL – Data Query Language (Linguagem de Consulta de Dados)

A **DQL** é uma classificação mais recente e específica, criada para resolver a ambiguidade conceitual em torno do **comando SELECT**. Segundo esta abordagem, **a DQL seria responsável exclusivamente pela recuperação de dados, sem modificá-los**.

Comando principal da DQL:

- **SELECT**: recupera dados das tabelas sem modificá-los.

3. Visão Geral dos Comandos DML

Independentemente da controvérsia sobre o SELECT, os comandos tradicionalmente classificados como DML constituem o núcleo operacional de qualquer sistema de banco de dados. Compreender suas funções, sintaxes básicas e inter-relações é fundamental para o domínio da manipulação de dados.

3.1. Características Comuns dos Comandos DML

Todos os comandos DML compartilham características importantes que os distinguem de outras sublinguagens:

- **Trabalham com dados, não com estruturas**: os comandos DML **operam sobre o conteúdo das tabelas, sem afetar suas definições ou esquemas**;
- **São transacionais**: todas as operações DML **podem ser agrupadas em transações**, permitindo confirmação (COMMIT) ou cancelamento (ROLLBACK) das modificações;
- **Respeitam restrições de integridade**: os comandos DML **devem obedecer às regras de integridade definidas no banco de dados**, como chaves primárias, chaves estrangeiras e restrições de domínio;
- **Podem ser condicionais**: **com exceção do INSERT básico, todos os comandos DML podem incluir cláusulas WHERE** para especificar quais registros devem ser afetados.

3.2. SELECT

O **SELECT** é o comando mais utilizado e cobrado em concursos. Sua função principal é **recuperar informações de uma ou mais tabelas**, apresentando-as de acordo com critérios especificados pelo usuário.

Sintaxe básica:

```
SELECT coluna1, coluna2, ... FROM nome_da_tabela WHERE condição;
```

3.2.1. Características Importantes

- Não modifica dados existentes;
- Pode trabalhar com uma ou múltiplas tabelas;
- Suporta funções de agregação e agrupamento;
- Permite ordenação e filtragem complexa dos resultados.

3.3. INSERT

O **INSERT** **adiciona novos registros às tabelas**, sendo essencial para a alimentação inicial e manutenção contínua dos dados.

Sintaxe básica:

```
INSERT INTO nome_da_tabela (coluna1, coluna2, ...) VALUES (valor1, valor2, ...);
```

3.3.1. Características Importantes

- Cria novos registros na tabela;
- Deve respeitar todas as restrições de integridade;
- Pode inserir um ou múltiplos registros simultaneamente;
- Suporta inserção baseada em subconsultas.

3.4. UPDATE

O **UPDATE** **modifica dados existentes nas tabelas, permitindo a correção e atualização de informações armazenadas.**

Sintaxe básica:

```
UPDATE nome_da_tabela SET coluna1 = valor1, coluna2 = valor2, ... WHERE condição;
```

3.4.1. Características Importantes

- Modifica registros existentes;
- A cláusula WHERE é crucial para evitar atualizações indesejadas;
- Pode atualizar uma ou múltiplas colunas simultaneamente;
- Suporta atualizações baseadas em subconsultas.

3.5. DELETE

O **DELETE** **remove registros das tabelas**, sendo utilizado para limpeza e manutenção dos dados.

Sintaxe básica:

```
DELETE FROM nome_da_tabela WHERE condição;
```

3.5.1. Características Importantes

- Remove registros completos (não colunas específicas);
- A cláusula WHERE é essencial para evitar exclusões em massa;
- Pode ser revertido dentro de uma transação;
- Difere do TRUNCATE (DDL) em aspectos de performance e recuperação.

ATENÇÃO

Com exceção de SELECT, todos os outros comandos DML são aplicáveis apenas a dados dentro de tabelas em um banco de dados. A principal diferença entre SELECT e todas as outras instruções DML é seu impacto nos dados em nível de linha:

* Para alterar os dados reais que residem nas tabelas, use as instruções **INSERT**, **DELETE** e **UPDATE**;

* Para acessar os dados no objeto de banco de dados, use **SELECT**.

SQL (DML): FUNDAMENTOS DA CONSULTA DE DADOS

1. Instrução SELECT

O comando **SELECT** constitui o alicerce de qualquer operação de consulta em SQL, sendo **responsável por recuperar informações de um banco de dados**. Embora seja frequentemente o primeiro comando que estudantes aprendem, sua compreensão adequada vai muito além da simples memorização de sintaxe, envolvendo conceitos fundamentais da teoria de bancos de dados relacionais.

Um ponto crucial que frequentemente gera confusão refere-se à relação entre o comando **SELECT** e as operações da álgebra relacional. É fundamental compreender que o **SELECT em SQL não realiza a mesma função que a operação de SELEÇÃO da álgebra relacional**.

Na álgebra relacional, a operação de SELEÇÃO (representada pelo símbolo σ – sigma) é responsável por filtrar linhas de uma relação com base em uma condição específica. Por outro lado, a operação de PROJEÇÃO (representada pelo símbolo π – pi) é responsável por especificar quais colunas devem ser retornadas.

O comando **SELECT em SQL**, na verdade, **realiza a função da operação de PROJEÇÃO**, pois após o **SELECT** são informadas as colunas que se deseja retornar. **A função de filtrar linhas (equivalente à SELEÇÃO da álgebra relacional) é realizada pela cláusula WHERE**.

A sintaxe básica de uma instrução **SELECT** segue a seguinte estrutura:

```
SELECT coluna1, coluna2, ... FROM nome_da_tabela WHERE condição;
```

Esta estrutura pode ser decomposta em três componentes essenciais:

- **SELECT**: especifica **quais colunas serão retornadas** (operação de PROJEÇÃO);
- **FROM**: indica de **qual tabela** os dados serão recuperados;
- **WHERE**: define **quais linhas serão filtradas** (operação de SELEÇÃO).

1.1. Variações da Sintaxe Básica

O comando **SELECT** oferece flexibilidade em sua utilização, permitindo diferentes abordagens conforme a necessidade.

1.1.1. Seleção de Colunas Específicas

```
SELECT Nome_Cliente, Cidade FROM Clientes;
```

Esta forma permite especificar exatamente quais colunas devem ser retornadas, oferecendo controle preciso sobre o resultado da consulta.

1.1.2. Seleção de Todas as Colunas

```
SELECT * FROM nome_da_tabela;
```

O asterisco (*) é um caractere especial que instrui o SGBD a retornar todas as colunas da tabela. Embora seja conveniente para exploração de dados, seu uso em aplicações de produção deve ser cuidadoso, pois pode impactar a performance quando a tabela possui muitas colunas.

1.1.3. Consulta com Condição

`SELECT coluna1, coluna2 FROM nome_da_tabela WHERE condição;`

Esta é a forma mais completa e frequentemente utilizada, combinando a especificação de colunas com critérios de filtro.

1.1.4. Consulta sem Condição

`SELECT coluna1, coluna2 FROM nome_da_tabela;`

É possível realizar uma instrução `SELECT` sem indicar nenhuma condição. Neste caso, a cláusula `WHERE` é opcional, e todos os registros da tabela serão retornados.

EXEMPLO

Para ilustrar os conceitos apresentados, consideremos uma tabela `CLIENTES` com a seguinte estrutura e dados:

IDCliente	Nome_Cliente	Nome_Contato	Endereco	Cidade	CEP	País
1	Alfreds Futterkiste	Maria Anders	Obere Str. 57	Berlin	12209	Germany
2	Ana Trujillo Emparedados y helados	Ana Trujillo	Avda. de la Constitución 2222	México D.F.	05021	Mexico
3	Antonio Moreno Taquería	Antonio Moreno	Mataderos 2312	México D.F.	05023	Mexico
4	Blondel père et fils	Frédérique Citeaux	24, place Kléber	Strasbourg	67000	France

Exemplo 1 – Seleção de Colunas Específicas

Consulta:

`SELECT Nome_Cliente, Cidade FROM Clientes;`

Resultado:

Nome_Cliente	Cidade
Alfreds Futterkiste	Berlin
Ana Trujillo Emparedados y helados	México D.F.
Antonio Moreno Taquería	México D.F.
Blondel père et fils	Strasbourg

Obs.: Note que após o `SELECT` são indicados os atributos que aparecerão no resultado. Caso seja utilizado `*`, então todos os atributos serão exibidos.

EXEMPLO

Exemplo 2 – Consulta com Filtro

Consulta:

`SELECT * FROM Clientes WHERE Pais = 'Mexico';`

Resultado:

IDCliente	Nome_Cliente	Nome_Contato	Endereco	Cidade	CEP	País
2	Ana Trujillo Emparedados y helados	Ana Trujillo	Avda. de la Constitución 2222	México D.F.		
3	Antonio Moreno Taquería	Antonio Moreno	Mataderos 2312	México D.F.		

2. Eliminação de Duplicatas com DISTINCT

Em muitas situações práticas, uma tabela pode conter **valores duplicados em determinadas colunas**, e frequentemente estamos interessados apenas nos valores únicos ou distintos. Para atender a esta necessidade, a SQL oferece a **cláusula DISTINCT**, que **elimina registros duplicados do resultado da consulta**.

A sintaxe para utilização do DISTINCT é:

```
SELECT DISTINCT coluna1, coluna2, ... FROM nome_da_tabela WHERE condição;
```

Ao usar SELECT DISTINCT, somente serão retornados os valores diferentes ou distintos, isto é, não são exibidos valores duplicados no resultado.

EXEMPLO

Utilizando a mesma tabela CLIENTES do exemplo anterior, suponha que desejamos saber em que países possuímos clientes:

Consulta sem DISTINCT:

```
SELECT Pais FROM Clientes;
```

Resultado:

País
Germany
Mexico
Mexico
France

Note que o país "Mexico" aparece duas vezes, pois temos dois clientes mexicanos.

Consulta com DISTINCT:

```
SELECT DISTINCT Pais FROM Clientes;
```

Resultado:

País
Germany
Mexico
France

Agora as duplicatas aparecem somente uma vez no resultado.

ATENÇÃO

Cuidado, pois a avaliação de redundância considera **todos os valores retornados**. Assim, se mais de um atributo for retornado, somente serão consideradas duplicadas as linhas que possuírem os **mesmos valores para todos os atributos**.

EXEMPLO

```
SELECT DISTINCT Nome, Pais FROM Clientes;
```

Neste caso, os pares (João, Mexico) e (João, Espanha) não são considerados duplicados, pois embora o nome seja igual, o país é diferente.

Esta nuance é frequentemente explorada em questões de concurso que testam a compreensão profunda do funcionamento do DISTINCT.

3. Definição de Alias (Apelidos)

Os **Alias (apelidos)** constituem uma funcionalidade fundamental da SQL que **permite atribuir nomes alternativos temporários a colunas e tabelas durante a execução de uma consulta**. Esta funcionalidade é especialmente útil para melhorar a legibilidade das consultas e simplificar referências a objetos com nomes complexos.

3.1. Alias para Colunas

Os **Alias de colunas** são utilizados para fornecer **nomes mais descritivos ou concisos aos campos retornados** pela consulta, especialmente úteis quando se trabalha com funções agregadas ou expressões calculadas.

Sintaxe para Alias de Colunas:

```
SELECT coluna1 AS alias1, coluna2 AS alias2 FROM tabela;
```

ou

```
SELECT coluna1 alias1, coluna2 alias2 FROM tabela;
```

Obs.: A palavra-chave **AS** é opcional, mas recomendada para maior clareza.

EXEMPLO

```
SELECT Nome_Cliente AS Cliente, Cidade AS Localidade FROM Clientes;
```

Resultado:

Cliente	Localidade
Alfreds Futterkiste	Berlin
Ana Trujillo Emparedados y helados	México D.F.
Antonio Moreno Taquería	México D.F.
Blondel père et fils	Strasbourg

3.2. Alias para Tabelas

Os **Alias de tabelas** são particularmente úteis em **consultas que envolvem múltiplas tabelas (JOINS) ou quando o nome da tabela é muito longo.**

Sintaxe para Alias de Tabelas:

```
SELECT c.Nome_Cliente, c.Cidade FROM Clientes AS c;
```

ou

```
SELECT c.Nome_Cliente, c.Cidade FROM Clientes c;
```

DICA

Em questões de concurso, é comum encontrar Alias obrigatórios quando se trabalha com **funções agregadas** ou **expressões calculadas**, pois muitos SGBDs exigem que colunas derivadas tenham nomes explícitos.

4. Filtragem de Dados com WHERE

A cláusula **WHERE** é responsável por implementar a operação de **SELEÇÃO da álgebra relacional**, permitindo filtrar registros com base em **condições específicas**. Esta cláusula é fundamental para a construção de consultas eficientes e precisas.

4.1. Conceito de Condição

Uma **condição** é uma **expressão condicional (booleana) que identifica tuplas a serem recuperadas**. Em SQL, as condições na cláusula WHERE utilizam diversos tipos de operadores para comparar valores e determinar quais registros devem ser incluídos no resultado.

4.2. Operadores Básicos de Comparação

Os operadores de comparação constituem a base para a construção de condições simples:

Operador	Significado	Exemplo	Descrição
=	igual	WHERE idade = 25	Valores exatamente iguais
<	menor	WHERE salario < 5000	Valor menor que
<=	menor ou igual	WHERE nota <= 7.0	Valor menor ou igual a
>	maior	WHERE quantidade > 100	Valor maior que
>=	maior ou igual	WHERE preco >= 50.00	Valor maior ou igual a
<>	diferente	WHERE status <> 'Ativo'	Valores diferentes

4.3. Operadores Lógicos

Os operadores lógicos permitem combinar múltiplas condições em uma única cláusula WHERE.

Operador	Função	Exemplo
AND	Ambas condições devem ser verdadeiras	WHERE idade > 18 AND salario > 3000
OR	Pelo menos uma condição deve ser verdadeira	WHERE cidade = 'São Paulo' OR cidade = 'Rio de Janeiro'
NOT	Nega a condição	WHERE NOT status = 'Inativo'

EXEMPLO

Utilizando nossa tabela CLIENTES, vamos supor que desejamos recuperar apenas os clientes que residem no México:

```
SELECT * FROM Clientes WHERE Pais = 'Mexico';
```

Resultado:

IDCliente	Nome_Cliente	Nome_Contato	Endereco	Cidade	CEP	País
2	Ana Trujillo Emparedados y helados	Ana Trujillo	Avda. de la Constitución 2222	México D.F.		
3	Antonio Moreno Taquería	Antonio Moreno	Mataderos 2312	México D.F.		

5. Operadores Especiais

Além dos operadores básicos de comparação, a SQL oferece operadores especiais que facilitam a construção de condições mais complexas e específicas.

5.1. Operador BETWEEN

O operador **BETWEEN** recupera os registros que estão em um determinado intervalo. Os valores podem ser números, texto ou data, e tanto os valores de início como o de fim são incluídos no resultado.

Sintaxe:

```
SELECT coluna1, coluna2, ... FROM nome_da_tabela WHERE coluna BETWEEN valor1 AND valor2;
```

5.1.1. Características Importantes

- **Inclusivo:** ambos os valores limite (valor1 e valor2) são incluídos no resultado;
- **Versatilidade:** funciona com números, datas e texto;
- **Performance:** geralmente mais eficiente que usar >= e <= separadamente.

Característica	Descrição	Exemplo
Inclusivo	Ambos os valores limite são incluídos	BETWEEN 1 AND 10 (inclui 1 e 10)
Versatilidade	Funciona com números, datas e texto	BETWEEN 'A' AND 'M'

Característica	Descrição	Exemplo
Performance	Mais eficiente que $\geq$ e $\leq$ separadamente	Otimização automática
Equivalência	BETWEEN x AND y ($\geq$ x AND $\leq$ y)	Mesmo resultado

EXEMPLO

Suponha que queremos recuperar clientes que possuam CEP entre 00000 e 06000:

```
SELECT * FROM Clientes WHERE CEP BETWEEN 00000 AND 06000;
```

Resultado:

IDCliente	Nome_Cliente	Nome_Contato	Endereco	Cidade	CEP	País
2	Ana Trujillo Emparedados y helados	Ana Trujillo	Avda. de la Constitución 2222	México D.F.		
3	Antonio Moreno Taquería	Antonio Moreno	Mataderos 2312	México D.F.		

Obs.: O operador BETWEEN pode ser bastante útil para um banco com uma infinidade de dados em que queremos separar dados com base em intervalos.

5.2. Operador IN

O operador IN permite especificar **múltiplos valores para uma condição**, isto é, a condição será testada com base na lista de valores indicada.

Sintaxe:

```
SELECT coluna1, coluna2, ... FROM nome_da_tabela WHERE coluna IN (valor1, valor2, ...);
```

5.2.1. Características Importantes

- **Múltiplos valores:** substitui múltiplas condições OR;
- **Legibilidade:** torna o código mais limpo e fácil de entender;
- **Flexibilidade:** pode ser usado com subconsultas.

Característica	Descrição	Exemplo
Múltiplos valores	Substitui múltiplas condições OR	IN ('A','B','C') = (= 'A' OR = 'B' OR = 'C')
Legibilidade	Torna o código mais limpo	Mais fácil de ler e manter
Flexibilidade	Pode ser usado com subconsultas	WHERE id IN (SELECT...)
Performance	Otimizado pelo SGBD	Melhor que múltiplos OR

EXEMPLO

Vamos supor que desejamos consultar os clientes de cidades específicas, digamos Berlin e Strasbourg:

```
SELECT * FROM Clientes WHERE Cidade IN ('Berlin', 'Strasbourg');
```

Resultado:

IDCliente	Nome_Cliente	Nome_Contato	Endereco	Cidade	CEP	País
1	Alfreds Futterkiste	Maria Anders	Obere Str. 57	Berlin		Germany
4	Blondel père et fils	Frédérique Citeaux	24, place Kléber			France

Obs.: Note que somente foram retornados os registros cuja cidade está informada na cláusula IN, que foram Berlin e Strasbourg.

5.3. Operador LIKE

O operador **LIKE** é utilizado para buscar um **padrão específico em uma coluna**, sendo especialmente útil para pesquisas de texto que não requerem correspondência exata.

Sintaxe:

```
SELECT coluna1, coluna2, ... FROM nome_da_tabela WHERE coluna LIKE padrão;
```

Caracteres especiais (Wildcards):

Caractere	Nome	Função	Exemplo	Resultado
%	Percentual	Representa zero, um ou múltiplos caracteres	A%	Qualquer string que comece com 'A'
_	Underscore	Representa exatamente um caractere	_r%	Qualquer string com 'r' como 2º caractere

5.3.1. Exemplos de Padrões:

Padrão	Significado	Exemplo de Match
A%	Qualquer string que comece com 'A'	Ana, 'Antonio'
%a	Qualquer string que termine com 'a'	Maria, 'Ana'
%or%	Qualquer string que contenha 'or'	Antonio, 'Moreno'
_r%	Qualquer string que tenha 'r' como segundo caractere	Frédérique
a_%_%	String que comece com 'a' e tenha pelo menos 3 chars	ana, 'antonio'

EXEMPLO

```
SELECT * FROM Clientes WHERE Nome_Cliente LIKE 'A%';
```

Resultado:

IDCliente	Nome_Cliente	Nome_Contato	Endereco	Cidade	CEP	País
1	Alfreds Futterkiste	Maria Anders	Obere Str. 57	Berlin	12209	Germany
2	Ana Trujillo Emparedados y helados	Ana Trujillo	Avda. de la Constitución 2222	México D.F.	5021	Mexico

Esta consulta retornará todos os clientes cujo nome comece com a letra 'A'.

5.4. Operador IS NULL

O operador **IS NULL** é utilizado para **identificar registros que possuem valores nulos em determinada coluna**. É importante compreender que **NULL representa a ausência de valor, não um valor zero ou string vazia**.

Sintaxe:

```
SELECT coluna1, coluna2, ... FROM nome_da_tabela WHERE coluna IS NULL;
```

5.4.1. Características Importantes

- **NULL ≠ 0**: NULL não é zero;
- **NULL ≠ ""**: NULL não é string vazia;
- **Comparação especial**: não se pode usar = NULL, deve-se usar IS NULL;
- **Negação**: para valores não nulos, usa-se IS NOT NULL.

Conceito	Descrição	Exemplo
NULL ≠ 0	NULL não é zero	NULL ≠ 0
NULL ≠ ""	NULL não é string vazia	NULL ≠ ""
Comparação especial	Não se pode usar = NULL	Usar IS NULL
Negação	Para valores não nulos	Usar IS NOT NULL
Lógica de 3 estados	TRUE, FALSE, UNKNOWN	NULL gera UNKNOWN

EXEMPLO

```
SELECT * FROM Clientes WHERE Nome_Contato IS NULL;
```

Esta consulta retornará todos os clientes que não possuem nome de contato cadastrado.

6. Ordenação com ORDER BY

A cláusula **ORDER BY** permite **organizar os resultados de uma consulta em uma ordem específica, seja crescente ou decrescente**. Esta funcionalidade é essencial para apresentar dados de forma organizada e facilitar a análise dos resultados.

Sintaxe básica:

```
SELECT coluna1, coluna2, ... FROM nome_da_tabela WHERE condição ORDER BY
coluna1 [ASC|DESC];
```

6.1. Tipos de Ordenação

6.1.1. Ordenação Crescente (ASC)

- **Padrão**: **se não especificado, a ordenação é crescente**;
- **Números**: do menor para o maior;
- **Texto**: ordem alfabética (A-Z);
- **Datas**: da mais antiga para a mais recente.

6.1.2. Ordenação Decrescente (DESC)

- **Números:** do maior para o menor;
- **Texto:** ordem alfabética reversa (Z-A);
- **Datas:** da mais recente para a mais antiga.

6.1.3. Ordenação por Múltiplas Colunas

É possível ordenar por múltiplas colunas, especificando a ordem de prioridade:

```
SELECT * FROM Clientes ORDER BY Pais ASC, Cidade DESC;
```

Neste exemplo, os registros serão primeiro ordenados por país (crescente) e, dentro de cada país, por cidade (decrescente).

EXEMPLO

```
SELECT Nome_Cliente, Cidade, Pais FROM Clientes ORDER BY Pais, Nome_Cliente;
```

Resultado:

Nome_Cliente	Cidade	Pais
Blondel père et fils	Strasbourg	France
Alfreds Futterkiste	Berlin	Germany
Ana Trujillo Emparedados y helados	México D.F.	Mexico
Antonio Moreno Taquería	México D.F.	Mexico

ATENÇÃO



É comum a confusão entre ORDER BY com GROUP BY. Lembre-se:

- * **ORDER BY:** organiza a apresentação dos resultados;
- * **GROUP BY:** agrupa registros para aplicação de funções agregadas.

ANÁLISE DE DADOS: FUNÇÕES AGREGADAS E AGRUPAMENTOS

1. Introdução às Funções Agregadas

Uma **função agregada** é uma **função que opera sobre um conjunto de valores (múltiplas linhas) e retorna um único valor como resultado**. Estas funções são essenciais para **análises estatísticas, relatórios gerenciais e operações de sumarização de dados**.

As **funções agregadas permitem realizar cálculos estatísticos sobre conjuntos de dados, transformando múltiplas linhas em um único valor resumido**.

Característica	Descrição	Exemplo
Múltiplas → Uma	Processa várias linhas, retorna um valor	1000 vendas → total vendido
Ignora NULL	Valores NULL são automaticamente ignorados	COUNT(coluna) ignora NULLs
Requer GROUP BY	Para usar com outras colunas não agregadas	SELECT cidade, COUNT(*) FROM... GROUP BY cidade
Não pode usar WHERE	WHERE filtra antes da agregação	Use HAVING para filtrar grupos
Ordem de execução	Executadas após WHERE e GROUP BY	WHERE → GROUP BY → Agregação → HAVING

2. As Cinco Funções Agregadas Principais

Para demonstrar as funções agregadas, utilizaremos uma tabela VENDAS com os seguintes dados:

Estrutura da Tabela VENDAS:

```
CREATE TABLE VENDAS ( ID_Venda INT PRIMARY KEY, Vendedor VARCHAR(50), Produto VARCHAR(50), Quantidade INT, Valor_Unitario DECIMAL(10,2), Valor_Total DECIMAL(10,2), Data_Venda DATE, Regiao VARCHAR(30) );
```

Dados da Tabela VENDAS:

ID_Venda	Vendedor	Produto	Quantidade	Valor_Unitario	Valor_Total	Data_Venda	Regiao
1	João	Notebook	2	2500.00	5000.00	2024-01-15	Sul
2	Maria	Mouse	10	50.00	500.00	2024-01-16	Norte
3	João	Teclado	5	100.00	500.00	2024-01-17	Sul
4	Ana	Monitor	3	800.00	2400.00	2024-01-18	Centro
5	Maria	Notebook	1	2500.00	2500.00	2024-01-19	Norte
6	Pedro	Mouse	NULL	50.00	NULL	2024-01-20	Leste
7	Ana	Teclado	8	100.00	800.00	2024-01-21	Centro

2.1. Função COUNT (Contagem de Registros)

A função **COUNT** é utilizada para **contar o número de registros que atendem a determinados critérios**. Esta é provavelmente a função agregada mais utilizada.

2.1.1. Sintaxe e Variações

Sintaxe	Função	Comportamento com NULL
COUNT(*)	Conta TODAS as linhas da tabela/grupo	Inclui linhas com NULL
COUNT(coluna)	Conta linhas onde a coluna NÃO é NULL	Exclui linhas com NULL
COUNT(DISTINCT)	Conta valores únicos (não nulos)	Exclui NULL e duplicatas

EXEMPLO

Exemplo 1 – COUNT(*)

SELECT COUNT(*) AS Total_Vendas FROM VENDAS;

Resultado:

Total_Vendas
7

Exemplo 2 – COUNT(coluna)

SELECT COUNT(Quantidade) AS Vendas_Com_Quantidade FROM VENDAS;

Resultado:

Vendas_Com_Quantidade
6

Note que COUNT(Quantidade) retorna 6, não 7, porque a linha do Pedro tem Quantidade = NULL.

EXEMPLO

Exemplo 3 – COUNT (DISTINCT)

SELECT COUNT(DISTINCT Vendedor) AS Numero_Vendedores FROM VENDAS;

Resultado:

Numero_Vendedores
4

2.2. Função SUM (Soma de Valores)

A função **SUM** calcula a **soma total de valores numéricos em uma coluna específica**.

2.2.1. Características Importantes

- Ignora valores NULL automaticamente;
- Só funciona com colunas numéricas;
- Retorna NULL se todos os valores forem NULL.

EXEMPLO

SELECT SUM(Valor_Total) AS Faturamento_Total FROM VENDAS;

Resultado:

Faturamento_Total
11700.00

Obs.: O valor NULL da linha do Pedro não afeta o cálculo.

2.3. Função AVG (Média Aritmética)

A função **AVG** **calcula a média aritmética dos valores numéricos em uma coluna.**

2.3.1. Características Importantes

- Ignora valores NULL no cálculo;
- Divide pela quantidade de valores não nulos;
- Pode retornar valores decimais.

EXEMPLO

```
SELECT AVG(Quantidade) AS Media_Quantidade FROM VENDAS;
```

Resultado:

Media_Quantidade
4.83

Cálculo: $(2 + 10 + 5 + 3 + 1 + 8) \div 6 = 29 \div 6 = 4.83$

2.4. Função MAX (Valor Máximo)

A função **MAX** **retorna o maior valor encontrado na coluna especificada.**

2.4.1. Características Importantes

- Funciona com números, datas e texto;
- Ignora valores NULL;
- Para texto, usa ordem alfabética.

EXEMPLO

Maior valor de venda:

```
SELECT MAX(Valor_Total) AS Maior_Venda FROM VENDAS;
```

Resultado:

Maior_Venda
5000.00

2.5. Função MIN (Valor Mínimo)

A função **MIN** **retorna o menor valor encontrado na coluna especificada.**

2.5.1. Características Importantes

- Funciona com números, datas e texto;
- Ignora valores NULL;
- Para texto, usa ordem alfabética.

EXEMPLO

```
SELECT MIN(Valor_Total) AS Menor_Venda FROM VENDAS;
```

Resultado:

Menor_Venda
500.00

Resumo das funções

Função	Propósito	Tipo de Dados	Tratamento NULL	Exemplo de Uso
COUNT	Contar registros	Qualquer	Varia conforme sintaxe	COUNT(*), COUNT(coluna)
SUM	Somar valores	Numérico	Ignora NULL	SUM(vendas)
AVG	Calcular média	Numérico	Ignora NULL	AVG(salario)
MAX	Encontrar maior valor	Numérico/Data/Texto	Ignora NULL	MAX(data_nascimento)
MIN	Encontrar menor valor	Numérico/Data/Texto	Ignora NULL	MIN(preco)

3. GROUP BY (Agrupamento de Dados)

O **GROUP BY** **divide os registros de uma tabela em grupos baseados nos valores de uma ou mais colunas.** Para cada grupo formado, as funções agregadas são calculadas independentemente.

A cláusula **GROUP BY** é fundamental para análises estatísticas mais sofisticadas, permitindo **agrupar registros com base em valores comuns e aplicar funções agregadas a cada grupo separadamente.**

Sintaxe básica:

```
SELECT coluna_agrupamento, FUNCAO_AGREGADA(coluna) FROM tabela WHERE condição  
GROUP BY coluna_agrupamento;
```

3.1. Regras Importantes do GROUP BY

Regra	Descrição	Exemplo Correto	Exemplo Incorreto
Colunas no SELECT	Toda coluna não agregada deve estar no GROUP BY	SELECT regiao, COUNT(*) GROUP BY regiao	SELECT regiao, vendedor, COUNT(*) GROUP BY regiao
Ordem de execução	GROUP BY executa antes das funções agregadas	WHERE → GROUP BY → Agregação	Não se aplica
Múltiplas colunas	Pode agrupar por várias colunas	GROUP BY regiao, vendedor	Não se aplica
Alias não permitido	Não pode usar alias de coluna no GROUP BY	GROUP BY regiao	GROUP BY r (onde r é alias)

EXEMPLO

Exemplo 1 – Vendas por Região

```
SELECT Regiao, COUNT(*) AS Numero_Vendas, SUM(Valor_Total) AS Total_Faturamento FROM VENDAS WHERE Valor_Total IS NOT NULL GROUP BY Regiao;
```

Resultado:

Regiao	Numero_Vendas	Total_Faturamento
Centro	2	3200.00
Norte	2	3000.00
Sul	2	5500.00

Exemplo 2 – Análise por Vendedor

```
ELECT Vendedor, COUNT(*) AS Vendas_Realizadas, AVG(Valor_Total) AS Ticket_Medio FROM VENDAS WHERE Valor_Total IS NOT NULL GROUP BY Vendedor;
```

Resultado:

	Vendas_Realizadas	Ticket_Medio
Ana	2	1600.00
João	2	2750.00
Maria	2	1500.00

Exemplo 3 – Agrupamento por Múltiplas Colunas

```
SELECT Regiao, Vendedor, COUNT(*) AS Vendas, SUM(Valor_Total) AS Total FROM VENDAS WHERE Valor_Total IS NOT NULL GROUP BY Regiao, Vendedor ORDER BY Regiao, Vendedor;
```

Resultado:

	Vendedor	Vendas	Total
	Ana	2	3200.00
Norte	Maria	2	3000.00
Sul	João	2	5500.00

4. HAVING (Filtragem de Grupos)

A cláusula **HAVING** é utilizada para **filtrar grupos após a aplicação das funções agregadas**. É o **equivalente ao WHERE, mas para grupos ao invés de linhas individuais**.

Sintaxe:

```
SELECT coluna_agrupamento, FUNCAO_AGREGADA(coluna) FROM tabela WHERE condição_
linhas GROUP BY coluna_agrupamento HAVING condição_grupos;
```

4.1. Diferença Fundamental: WHERE vs HAVING

Aspecto	WHERE	HAVING
Momento de execução	Antes do GROUP BY	Depois do GROUP BY
Filtra	Linhas individuais	Grupos formados
Pode usar	Colunas da tabela	Funções agregadas
Exemplo	WHERE salario > 5000	HAVING COUNT(*) > 10
Ordem na consulta	SELECT...FROM...WHERE...GROUP BY	SELECT...FROM...WHERE...GROUP BY... HAVING

EXEMPLO

Exemplo 1 – Regiões com Mais de um Vendedor

```
SELECT Regiao, COUNT(*) AS Numero_Vendas FROM VENDAS WHERE Valor_Total
IS NOT NULL GROUP BY Regiao HAVING COUNT(*) > 1;
```

Resultado:

Regiao	Numero_Vendas
Centro	2
Norte	2
Sul	2

Exemplo 2 – Vendedores com Faturamento Acima de R\$3.000,00

```
SELECT Vendedor, SUM(Valor_Total) AS Faturamento_Total FROM VENDAS WHERE
Valor_Total IS NOT NULL GROUP BY Vendedor HAVING SUM(Valor_Total) >
3000;
```

Resultado:

Vendedor	Faturamento_Total
João	5500.00

Exemplo 3 – Combinando WHERE e HAVING

SELECT Regiao, AVG(Valor_Total) AS Ticket_Medio FROM VENDAS WHERE Data_Venda >= '2024-01-16' -- WHERE: filtra linhas GROUP BY Regiao HAVING AVG(Valor_Total) > 1000; -- HAVING: filtra grupos

Resultado:

Regiao	Ticket_Medio
Centro	1600.00
Norte	1500.00

5. Ordem de Execução das Cláusulas SQL

Compreender a ordem de execução das cláusulas SQL é fundamental para construir consultas corretas.

5.1. Ordem Lógica de Execução

Ordem	Cláusula	Função	Exemplo
1	FROM	Define as tabelas fonte	FROM VENDAS
2	WHERE	Filtra linhas individuais	WHERE Valor_Total > 1000
3	GROUP BY	Agrupar linhas por valores comuns	GROUP BY Regiao
4	HAVING	Filtra grupos após agregação	HAVING COUNT(*) > 2
5	SELECT	Especifica colunas do resultado	SELECT Regiao, COUNT(*)
6	ORDER BY	Ordena o resultado final	ORDER BY COUNT(*) DESC
7	LIMIT	Limita número de linhas (quando disponível)	LIMIT 10

EXEMPLO

SELECT Regiao, COUNT(*) AS Vendas, AVG(Valor_Total) AS Ticket_Medio FROM VENDAS WHERE Data_Venda >= '2024-01-16' GROUP BY Regiao HAVING COUNT(*) >= 2 ORDER BY AVG(Valor_Total) DESC;

Execução passo a passo:

- 1. FROM VENDAS:** carrega a tabela VENDAS
- 2. WHERE Data_Venda >= '2024-01-16':** filtra vendas a partir de 16/01
- 3. GROUP BY Regiao:** agrupa por região
- 4. HAVING COUNT(*) >= 2:** mantém apenas regiões com 2+ vendas
- 5. SELECT:** projeta colunas especificadas

6. ORDER BY: ordena por ticket médio decrescente

Resultado:

Regiao	Vendas	Ticket_Medio
Centro	2	1600.00
Norte	2	1500.00

RELACIONAMENTOS ENTRE TABELAS: JOINS E OPERAÇÕES DE CONJUNTO

1. Produto Cartesiano

Antes de abordar os JOINS propriamente ditos, é fundamental compreender o conceito de produto cartesiano, que constitui a base teórica para todos os tipos de junções em SQL.

O produto cartesiano representa a combinação de cada linha de uma tabela com cada linha de outra tabela, resultando em um conjunto que pode ser muito maior que as tabelas originais.

O produto cartesiano entre duas tabelas A e B **resulta em uma nova tabela que contém todas as combinações possíveis entre as linhas de A e as linhas de B**. Se a tabela A possui m linhas e a tabela B possui n linhas, o produto cartesiano resultará em $m \times n$ linhas.

Em SQL, o produto cartesiano é indicado com o uso de vírgulas entre as tabelas desejadas.

`SELECT tabela1.coluna1, tabela2.coluna2, ... FROM tabela1, tabela2 WHERE condição;`

EXEMPLO

Para demonstrar este conceito, utilizaremos duas tabelas simples:

a) Tabela DEPARTAMENTOS:

`CREATE TABLE DEPARTAMENTOS ( ID_Dept INT PRIMARY KEY, Nome_Dept VARCHAR(50), Localizacao VARCHAR(50) );`

Dados da Tabela DEPARTAMENTOS:

ID_Dept	Nome_Dept	Localizacao
1	Vendas	São Paulo
2	Marketing	Rio de Janeiro
3	TI	Belo Horizonte

b) Tabela FUNCIONARIOS:

`CREATE TABLE FUNCIONARIOS ( ID_Func INT PRIMARY KEY, Nome_Func VARCHAR(50), ID_Dept INT, Salario DECIMAL(10,2) );`

Dados da Tabela FUNCIONARIOS:

ID_Func	Nome_Func	ID_Dept	Salario
101	João	1	5000.00
102	Maria	2	6000.00
103	Pedro	1	4500.00
104	Ana	NULL	5500.00

c) Consulta com Produto Cartesiano

SELECT * FROM FUNCIONARIOS, DEPARTAMENTOS;

ou equivalentemente:

SELECT * FROM FUNCIONARIOS CROSS JOIN

Resultado:

ID_Func	Nome_Func	ID_Dept	Salario	ID_Dept	Nome_Dept	Localizacao
101	João	1		1	Vendas	São Paulo
101	João	1		2	Marketing	Rio de Janeiro
101	João	1		3	TI	Belo Horizonte
102	Maria	2		1	Vendas	São Paulo
102	Maria	2		2	Marketing	Rio de Janeiro
102	Maria	2		3	TI	Belo Horizonte
103	Pedro	1		1	Vendas	São Paulo
103	Pedro	1		2	Marketing	Rio de Janeiro
103	Pedro	1		3	TI	Belo Horizonte
104	Ana	NULL		1	Vendas	São Paulo
104	Ana	NULL		2	Marketing	Rio de Janeiro
104	Ana	NULL		3	TI	Belo Horizonte

Análise do resultado:

Tabela FUNCIONARIOS: 4 linhas

Tabela DEPARTAMENTOS: 3 linhas

Produto Cartesiano: $4 \times 3 = 12$ linhas

d) Problemas do Produto Cartesiano

Problema	Descrição	Exemplo
Volume excessivo	Número de linhas cresce exponencialmente	$1000 \times 1000 = 1.000.000$ linhas
Dados irrelevantes	Combinações sem sentido lógico	João do Vendas com Depto TI
Performance degradada	Consultas lentas e consumo de memória	Timeout em tabelas grandes
Resultados incorretos	Informações distorcidas para análises	Totais multiplicados incorretamente

2. Tabelas de Junção (JOINS)

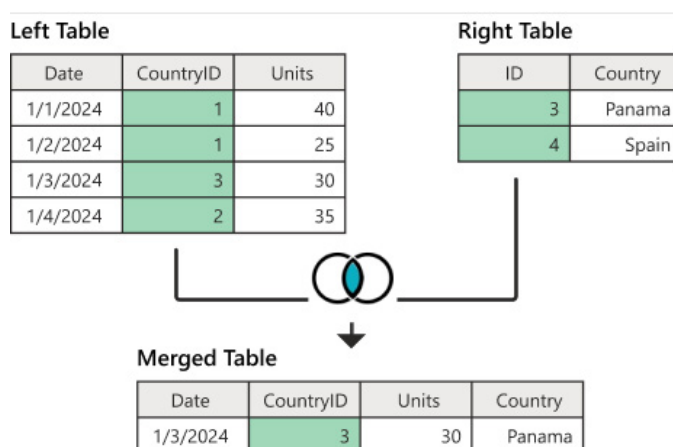
As **tabelas de junção** **permitem combinar dados provenientes de duas ou mais tabelas com base em colunas relacionadas entre si**. Essas junções são definidas por meio da cláusula **JOIN**, que pode ser aplicada em diferentes formas:

- **INNER JOIN (ou simplesmente JOIN)**: retorna apenas os registros que possuem **valores correspondentes em ambas as tabelas (as interseções)**;
- **LEFT JOIN (ou LEFT OUTER JOIN)**: retorna todos os **registros da tabela à esquerda e os registros correspondentes da tabela à direita**. Campos sem correspondência na tabela da direita são preenchidos com NULL;
- **RIGHT JOIN (ou RIGHT OUTER JOIN)**: retorna todos os **registros da tabela à direita e os registros correspondentes da tabela à esquerda**. Campos sem correspondência na tabela da esquerda são preenchidos com NULL;
- **FULL OUTER JOIN**: retorna **todos os registros**, independentemente de haver correspondência entre as tabelas. Campos sem correspondência em qualquer das tabelas são preenchidos com NULL;
- **CROSS JOIN**: retorna todos os **registros da primeira tabela combinados com todos os registros da segunda**. Representa o produto cartesiano;
- **SELF JOIN**: realiza a junção de **uma tabela com ela mesma**.

2.1. INNER JOIN

O **INNER JOIN** é o tipo de junção mais utilizado em SQL e resolve o problema do produto cartesiano ao **combinar apenas as linhas que possuem correspondência em ambas as tabelas com base em uma condição específica**.

O **INNER JOIN** retorna **apenas os registros que têm valores correspondentes em ambas as tabelas, ou seja, a interseção entre as tabelas**. Registros sem correspondência são automaticamente excluídos do resultado.



Fonte: <https://learn.microsoft.com/pt-br/power-query>.

Sintaxe básica:

SELECT colunas FROM tabela1 INNER JOIN tabela2 ON tabela1.coluna = tabela2.coluna;

Sintaxe alternativa (mais antiga):

SELECT colunas FROM tabela1, tabela2 WHERE tabela1.coluna = tabela2.coluna;

2.1.1. Características

Característica	Descrição	Exemplo
Apenas correspondências	Só retorna linhas com match em ambas tabelas	Ana (NULL) é excluída
Mais restritivo	Menor número de linhas no resultado	3 linhas ao invés de 12
Performance otimizada	SGBD pode usar índices eficientemente	Consultas mais rápidas
Padrão implícito	JOIN sem especificação = INNER JOIN	JOIN = INNER JOIN

EXEMPLO

SELECT f.Nome_Func, f.Salario, d.Nome_Dept, d.Localizacao FROM FUNCIONARIOS f INNER JOIN DEPARTAMENTOS d ON f.ID_Dept = d.ID_Dept;

Resultado:

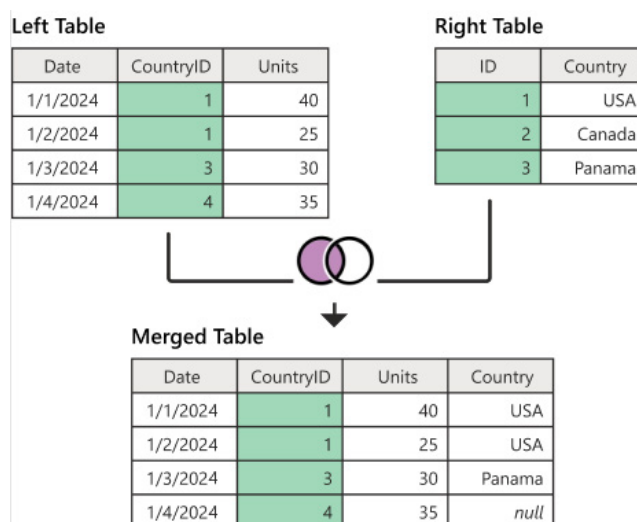
Nome_Func	Salario	Nome_Dept	Localizacao
João	5000.00	Vendas	São Paulo
Maria	6000.00	Marketing	Rio de Janeiro
Pedro	4500.00	Vendas	São Paulo

Obs.: Note que Ana não aparece no resultado porque seu ID_Dept é NULL, não havendo correspondência com nenhum departamento.

2.2. LEFT JOIN

O **LEFT JOIN** (ou **LEFT OUTER JOIN**) retorna **todas as linhas da tabela da esquerda e as linhas correspondentes da tabela da direita. Quando não há correspondência, os campos da tabela da direita são preenchidos com NULL.**

O **LEFT JOIN** é **utilizado quando queremos preservar todos os registros da primeira tabela (esquerda)**, independentemente de haver correspondência na segunda tabela (direita). Este tipo de junção é fundamental **para análises que precisam incluir registros “órfãos”.**



Fonte: <https://learn.microsoft.com/pt-br/power-query>.

Sintaxe do LEFT JOIN:

SELECT colunas FROM tabela1 LEFT JOIN tabela2 ON tabela1.coluna = tabela2.coluna;

EXEMPLO

SELECT f.Nome_Func, f.Salario, d.Nome_Dept, d.Localizacao FROM FUNCIONARIOS f LEFT JOIN DEPARTAMENTOS d ON f.ID_Dept = d.ID_Dept;

Resultado:

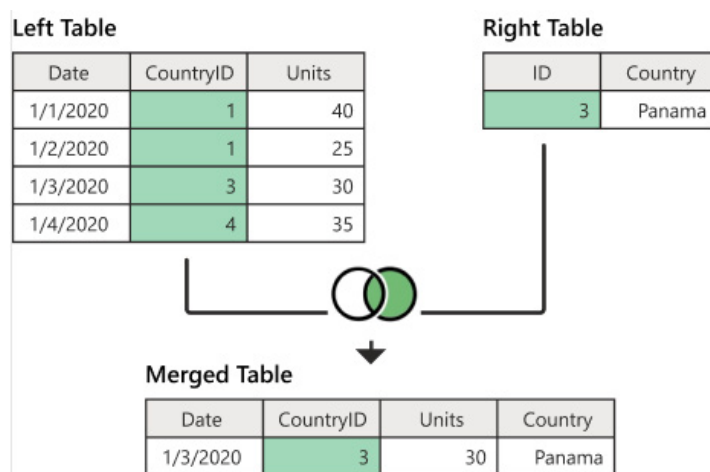
Nome_Func	Salario	Nome_Dept	Localizacao
João		Vendas	São Paulo
Maria		Marketing	Rio de Janeiro
Pedro		Vendas	São Paulo
Ana		NULL	NULL

2.2.1. Casos de Uso

Cenário	Descrição	Exemplo de Consulta
Funcionários sem depto	Identificar funcionários não alocados	LEFT JOIN para encontrar NULLs
Clientes sem pedidos	Clientes que nunca compraram	Clientes LEFT JOIN Pedidos
Produtos sem vendas	Produtos que não foram vendidos	Produtos LEFT JOIN Vendas
Relatórios completos	Incluir todos os registros principais	Sempre mostrar funcionários

2.3. RIGHT JOIN

O **RIGHT JOIN** (ou **RIGHT OUTER JOIN**) é o **espelho do LEFT JOIN**, retornando **todas as linhas da tabela da direita e as linhas correspondentes da tabela da esquerda**. Quando não há correspondência, os campos da tabela da esquerda são preenchidos com **NULL**.



Fonte: <https://learn.microsoft.com/pt-br/power-query>.

Sintaxe:

SELECT colunas FROM tabela1 RIGHT JOIN tabela2 ON tabela1.coluna = tabela2.coluna;

EXEMPLO

```
ELECT f.Nome_Func, f.Salario, d.Nome_Dept, d.Localizacao FROM FUNCIONARIOS
f RIGHT JOIN DEPARTAMENTOS d ON f.ID_Dept = d.ID_Dept;
```

Resultado:

Nome_Func	Salario	Nome_Dept	Localizacao
João		Vendas	São Paulo
Pedro		Vendas	São Paulo
Maria		Marketing	Rio de Janeiro
NULL	NULL	TI	Belo Horizonte

2.3.1. Equivalência LEFT ↔ RIGHT

RIGHT JOIN pode sempre ser reescrito como LEFT JOIN invertendo a ordem das tabelas:

Estas consultas são equivalentes:

```
SELECT * FROM A RIGHT JOIN B ON A.id = B.id;
```

e

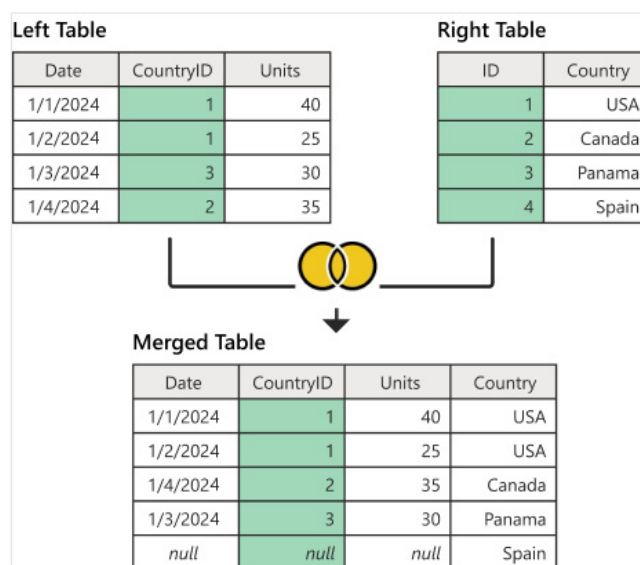
```
SELECT * FROM B LEFT JOIN A ON A.id = B.id;
```

2.4. FULL OUTER JOIN

O **FULL OUTER JOIN** combina os resultados de **LEFT JOIN** e **RIGHT JOIN**, retornando **todas as linhas de ambas as tabelas**. Quando não há correspondência, os campos da **tabela sem correspondência são preenchidos com NULL**.

Sintaxe:

```
SELECT colunas FROM tabela1 FULL OUTER JOIN tabela2 ON tabela1.coluna =
tabela2.coluna;
```



Fonte: <https://learn.microsoft.com/pt-br/power-query>.

ATENÇÃO !

Nem todos os SGBDs suportam FULL OUTER JOIN (por exemplo, MySQL não suporta nativamente).

EXEMPLO

```
SELECT f.Nome_Func, f.Salario, d.Nome_Dept, d.Localizacao FROM FUNCIONARIOS
f FULL OUTER JOIN DEPARTAMENTOS d ON f.ID_Dept = d.ID_Dept;
```

Resultado:

Nome_Func	Salario	Nome_Dept	Localizacao
João		Vendas	São Paulo
Pedro		Vendas	São Paulo
Maria		Marketing	Rio de Janeiro
Ana		NULL	NULL
NULL	NULL	TI	Belo Horizonte

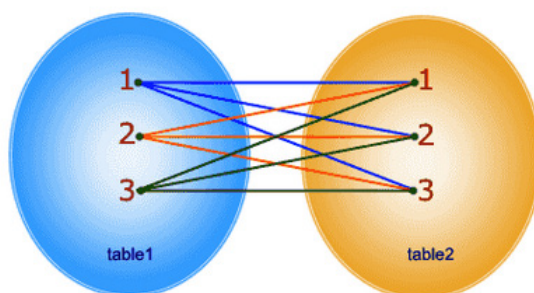
Obs.: Aparecem todos os funcionários (incluindo Ana sem departamento) e todos os departamentos (incluindo TI sem funcionários).

2.5. CROSS JOIN

O **CROSS JOIN** é a **implementação formal do produto cartesiano em SQL, permitindo gerar todas as combinações possíveis entre duas tabelas de forma explícita e controlada**. Embora seja menos comum em aplicações práticas, o CROSS JOIN é importante para compreender a teoria dos relacionamentos e pode ser útil em cenários específicos.

O **CROSS JOIN** produz o produto cartesiano entre duas tabelas, **combinando cada linha da primeira tabela com cada linha da segunda tabela**. Não requer nenhuma condição de junção, pois o objetivo é exatamente gerar todas as combinações possíveis.

`SELECT * FROM table1 CROSS JOIN table2;`



In CROSS JOIN, each row from 1st table joins with all the rows of another table.
If 1st table contain x rows and y rows in 2nd one the result set will be x * y rows.

Fonte: <https://www.w3resource.com/sql/joins>.

Sintaxe:

`SELECT colunas FROM tabela1 CROSS JOIN tabela2;`

ou

`SELECT colunas FROM tabela1, tabela2;`

EXEMPLO

`SELECT f.Nome_Func, d.Nome_Dept FROM FUNCIONARIOS f CROSS JOIN DEPARTAMENTOS
d ORDER BY f.Nome_Func, d.Nome_Dept;`

Resultado:

Nome_Func	Nome_Dept
Ana	Marketing
Ana	TI
Ana	Vendas
João	Marketing
João	TI
João	Vendas
Maria	Marketing
Maria	TI
Maria	Vendas
Pedro	Marketing
Pedro	TI
Pedro	Vendas

Análise: 4 funcionários × 3 departamentos = 12 combinações

2.5.1. Casos de Uso

Cenário	Descrição	Exemplo
Geração de combinações	Criar todas as possibilidades	Produtos x Cores x Tamanhos
Calendários e horários	Combinar datas com horários	Dias x Horários disponíveis
Testes de configuração	Testar todas as combinações de parâmetros	Parâmetros x Valores
Relatórios de matriz	Criar estruturas de relatório em grade	Regiões x Períodos

2.5.2. Cuidados com CROSS JOIN

Aspecto	Descrição	Recomendação
Volume de dados	Crescimento exponencial do resultado	Usar apenas com tabelas pequenas
Performance	Pode ser muito lento com tabelas grandes	Considerar alternativas
Memória	Consome muita memória para resultados grandes	Monitorar uso de recursos
Utilidade prática	Raramente necessário em aplicações reais	Avaliar se realmente precisa

2.6. SELF JOIN

SELF JOIN é uma técnica especial onde **uma tabela é relacionada consigo mesma, permitindo comparar registros da mesma tabela ou navegar por estruturas hierárquicas.**

Não é um tipo de JOIN diferente, mas sim uma aplicação dos JOINS tradicionais (INNER, LEFT, etc.) onde ambas as tabelas são a mesma.

No **SELF JOIN**, **utilizamos alias obrigatórios para distinguir as duas “instâncias” da mesma tabela**. Isso permite comparar linhas da tabela com outras linhas da mesma tabela, criando relacionamentos internos.

2.6.1. Casos de Uso

Cenário	Descrição	Exemplo de Consulta
Estrutura hierárquica	Funcionários e supervisores	Organograma da empresa
Comparações internas	Comparar registros da mesma tabela	Produtos similares
Sequências temporais	Relacionar eventos consecutivos	Vendas mês atual vs anterior
Árvores genealógicas	Relacionamentos familiares	Pais e filhos
Redes sociais	Relacionamentos entre usuários	Amigos e seguidores

2.6.2. Características Importantes

Característica	Descrição	Observação
Alias obrigatórios	Deve usar alias para distinguir instâncias	FROM tabela t1 JOIN tabela t2
Mesma tabela	Ambos os lados do JOIN são a mesma tabela	Não é um tipo novo de JOIN
Chave estrangeira interna	Relacionamento dentro da própria tabela	ID_Supervisor referencia ID_Func
Recursão limitada	SQL padrão não suporta recursão infinita	Use CTE recursivo quando disponível

2.7. Visualização dos JOINS

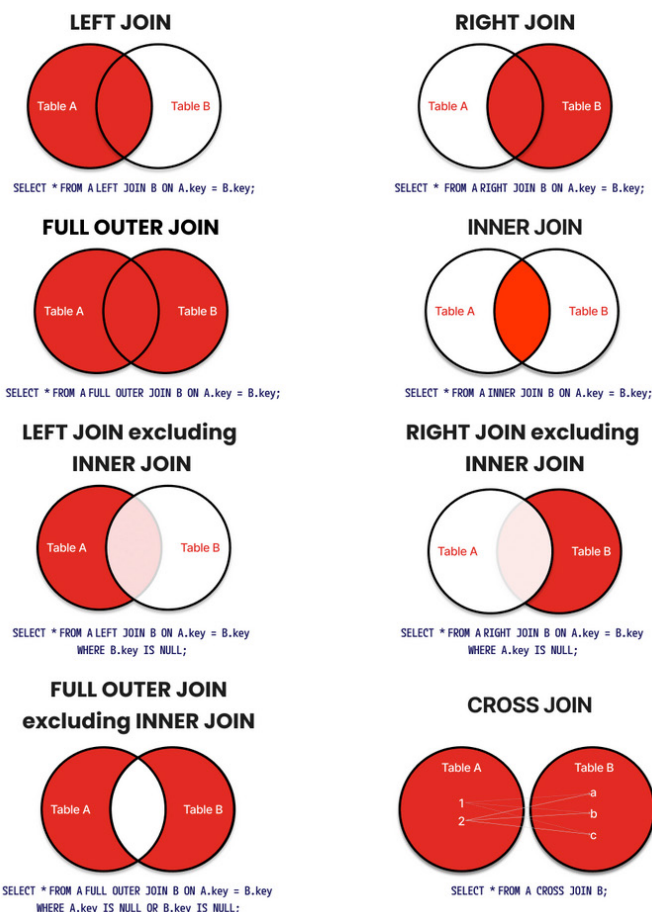
Para facilitar a compreensão dos diferentes tipos de JOIN, utilizamos **Diagramas de Venn** que representam visualmente quais registros são incluídos em cada tipo de junção.

Tipo de JOIN	Área do Diagrama de Venn	Registros Incluídos
INNER JOIN	Apenas a interseção ($A \cap B$)	Só correspondências mútuas
LEFT JOIN	Círculo A completo + interseção	Todos de A + matches de B
RIGHT JOIN	Círculo B completo + interseção	Todos de B + matches de A
FULL OUTER JOIN	União completa ($A \cup B$)	Todos de A + Todos de B
CROSS JOIN	Produto cartesiano completo	Todas as combinações possíveis
SELF JOIN	Relacionamento interno (A com A)	Registros da mesma tabela relacionados

2.8. Resumo Comparativo dos JOINS

Aspecto	INNER JOIN	LEFT JOIN	RIGHT JOIN	FULL OUTER JOIN
Linhas de A	Só matches	Todas	Só matches	Todas
Linhas de B	Só matches	Só matches	Todas	Todas
NULLs de A	Não	Não	Sim	Sim
NULLs de B	Não	Sim	Não	Sim
Tamanho resultado	Menor	Médio	Médio	Maior
Uso mais comum	Sim	Comum	Raro	Específico

SQL JOINS Visualized with Venn Diagrams



Fonte: <https://red9.com/blog/sql-joins-venn-diagram/>.

2.9. JOINS Múltiplos

Em cenários reais, frequentemente precisamos combinar três ou mais tabelas em uma única consulta. Os JOINS múltiplos seguem a mesma lógica dos JOINS simples, mas requerem atenção especial à ordem de execução e condições de junção.

EXEMPLO

Vamos adicionar uma terceira tabela ao nosso exemplo:

Tabela PROJETOS:

```
CREATE TABLE PROJETOS ( ID_Projeto INT PRIMARY KEY, Nome_Projeto
  VARCHAR(50), ID_Responsavel INT, Orcamento DECIMAL(12,2) );
```

Dados da Tabela PROJETOS:

ID_Projeto	Nome_Projeto	ID_Responsavel	Orcamento
1	Sistema Vendas	101	100000.00
2	App Mobile	102	75000.00
3	Data Warehouse	105	150000.00

Consulta com JOINS Múltiplos

```
SELECT f.Nome_Func, d.Nome_Dept, p.Nome_Projeto, p.Orcamento FROM
  FUNCIONARIOS f INNER JOIN DEPARTAMENTOS d ON f.ID_Dept = d.ID_Dept LEFT
  JOIN PROJETOS p ON f.ID_Func = p.ID_Responsavel ORDER BY f.Nome_Func;
```

Resultado:

Nome_Func	Nome_Dept	Nome_Projeto	Orcamento
João	Vendas	Sistema Vendas	100000.00
Maria	Marketing	App Mobile	75000.00
Pedro	Vendas	NULL	NULL

Ordem de Execução dos JOINS Múltiplos

Passo	Operação	Resultado Intermediário
1	FROM FUNCIONARIOS f	Tabela base (4 linhas)
2	INNER JOIN DEPARTAMENTOS d	Só funcionários com depto (3 linhas)
3	LEFT JOIN PROJETOS p	Inclui funcionários sem projeto (3 linhas)
4	SELECT e ORDER BY	Resultado final ordenado

3. Operações de Conjunto

As operações de conjunto permitem **combinar resultados de duas ou mais consultas SELECT, seguindo os princípios da teoria de conjuntos**. Estas operações são especialmente úteis para consolidar dados de diferentes fontes ou realizar análises comparativas.

3.1. Regras Fundamentais das Operações de Conjunto

Regra	Descrição	Exemplo de Erro
Mesmo número de colunas	Todas as consultas devem ter igual qtd cols	SELECT a,b UNION SELECT c

Regra	Descrição	Exemplo de Erro
Tipos compatíveis	Colunas correspondentes devem ser compatíveis	INT UNION VARCHAR pode falhar
Nomes das colunas	Resultado usa nomes da primeira consulta	Primeira consulta define nomes
ORDER BY apenas no final	Ordenação só na consulta final	ORDER BY no meio gera erro

3.2. UNION (União de Conjuntos)

O operador **UNION** combina os resultados de duas ou mais consultas **SELECT**, eliminando automaticamente as linhas duplicadas.

Sintaxe:

```
SELECT colunas FROM tabela1 UNION SELECT colunas FROM tabela2;
```

EXEMPLO

Vamos criar duas consultas que queremos combinar:

Funcionários do departamento de Vendas:

```
SELECT Nome_Func, 'Vendas' AS Origem FROM FUNCIONARIOS WHERE ID_Dept = 1 UNION
```

Funcionários com salário acima de 5500:

```
SELECT Nome_Func, 'Alto Salário' AS Origem FROM FUNCIONARIOS WHERE Salario > 5500;
```

Resultado:

Nome_Func	Origem
João	Vendas
Pedro	Vendas
Maria	Alto Salário
Ana	Alto Salário

3.3. UNION ALL (União com Duplicatas)

O **UNION ALL** funciona como o **UNION**, mas preserva todas as linhas, incluindo duplicatas.

3.3.1. Diferença UNION vs UNION ALL

Aspecto	UNION	UNION ALL
Duplicatas	Remove automaticamente	Preserva todas as linhas
Performance	Mais lenta (precisa verificar)	Mais rápida (não verifica)

Aspecto	UNION	UNION ALL
Uso recomendado	Quando não quer duplicatas	Quando quer todas as linhas
Ordenação implícita	Pode ordenar para remover	Mantém ordem original

3.4. INTERSECT (Interseção de Conjuntos)

O operador **INTERSECT** retorna **apenas as linhas que aparecem em ambas as consultas**.

Sintaxe:

```
SELECT colunas FROM tabela1 INTERSECT SELECT colunas FROM tabela2;
```

Obs.: Nem todos os SGBDs suportam INTERSECT (MySQL não suporta nativamente).

3.5. EXCEPT (Diferença de Conjuntos)

O operador **EXCEPT** (ou **MINUS** em alguns SGBDs) retorna as **linhas da primeira consulta que não aparecem na segunda consulta**.

Sintaxe:

```
SELECT colunas FROM tabela1 EXCEPT SELECT colunas FROM tabela2;
```

4. Subconsultas (Consultas Dentro de Consultas)

As **subconsultas** (ou **subqueries**) são **consultas SELECT aninhadas dentro de outras consultas SQL**. Elas permitem resolver problemas complexos dividindo-os em partes menores e mais gerenciáveis.

Tipos de subconsultas:

Tipo	Descrição	Exemplo de Uso
Subconsulta Escalar	Retorna um único valor (1 linha, 1 coluna)	SELECT com comparação
Subconsulta de Linha	Retorna uma linha com múltiplas colunas	WHERE com tupla
Subconsulta de Tabela	Retorna múltiplas linhas e colunas	FROM, EXISTS, IN
Subconsulta Correlacionada	Referencia tabela da consulta externa	Comparações linha por linha
Subconsulta Não Correlacionada	Independente da consulta externa	Filtros baseados em agregação

EXEMPLO

Tabelas Base para Exemplos

Para demonstrar subconsultas, utilizaremos tabelas relacionadas:

Tabela FUNCIONARIOS_SUB:

```
CREATE TABLE FUNCIONARIOS_SUB ( ID_Func INT PRIMARY KEY, Nome VARCHAR(50), ID_Dept INT, Salario DECIMAL(10,2), Data_Admissao DATE );
```

Dados:

ID_Func		ID_Dept	Salario	Data_Admissao
1	João	1		2020-01-15
2	Maria	2		2019-03-20
3	Pedro	1		2021-06-10
4	Ana	3		2018-11-05
5		2		2020-09-12

Tabela DEPARTAMENTOS_SUB:

```
CREATE TABLE DEPARTAMENTOS_SUB ( ID_Dept INT PRIMARY KEY, Nome_Dept
    VARCHAR(50), Orcamento DECIMAL(12,2) );
```

Dados:

ID_Dept	Nome_Dept	Orcamento
1	Vendas	100000.00
2	Marketing	150000.00
3	TI	200000.00

4.1. Subconsultas Escalares

Retornam um único valor e podem ser usadas em qualquer lugar onde um valor único é esperado.

EXEMPLO

Exemplo 1 – Funcionários com Salário Acima da Média

```
SELECT Nome, Salario FROM FUNCIONARIOS_SUB WHERE Salario > ( SELECT
    AVG(Salario) FROM FUNCIONARIOS_SUB );
```

Resultado:

	Salario
Maria	6000.00
Ana	7000.00
	5500.00

Exemplo 2 – Departamento com Maior Orçamento

```
SELECT Nome_Dept, Orcamento FROM DEPARTAMENTOS_SUB WHERE Orcamento = (
    SELECT MAX(Orcamento) FROM DEPARTAMENTOS_SUB );
```

Resultado:

Nome_Dept	Orcamento
TI	200000.00

4.2. Subconsultas de Tabela

Retornam **múltiplas linhas** e são usadas com **operadores como IN, EXISTS, ANY, ALL**.

EXEMPLO

Exemplo 1 – Funcionários de Departamentos com Orçamento > 120000

```
SELECT Nome, Salario FROM FUNCIONARIOS_SUB WHERE ID_Dept IN ( SELECT
  ID_Dept FROM DEPARTAMENTOS_SUB WHERE Orcamento > 120000 );
```

Resultado:

	Salario
Maria	6000.00
Ana	7000.00
	5500.00

Exemplo 2 – Departamentos que Têm Funcionários

```
SELECT Nome_Dept FROM DEPARTAMENTOS_SUB WHERE ID_Dept IN ( SELECT
  DISTINCT ID_Dept FROM FUNCIONARIOS_SUB );
```

4.3. Subconsultas Correlacionadas

Fazem referência à tabela externa e são executadas uma vez para cada linha da consulta externa.

EXEMPLO

Exemplo 1 – Funcionários com Maior Salário em seu Departamento

```
SELECT f1.Nome, f1.Salario, f1.ID_Dept FROM FUNCIONARIOS_SUB f1 WHERE
  f1.Salario = ( SELECT MAX(f2.Salario) FROM FUNCIONARIOS_SUB f2 WHERE
  f2.ID_Dept = f1.ID_Dept );
```

Resultado:

	Salario	ID_Dept
João		1
Maria		2
Ana		3

Exemplo 2 – Funcionários Admitidos Antes da Média de seu Departamento

```
SELECT Nome, Data_Admissao, ID_Dept FROM FUNCIONARIOS_SUB f1 WHERE
  Data_Admissao < ( SELECT AVG(f2.Data_Admissao) FROM FUNCIONARIOS_SUB
  f2 WHERE f2.ID_Dept = f1.ID_Dept );
```

4.4. Subconsultas no FROM (Tabelas Derivadas)

Subconsultas podem ser usadas na cláusula FROM, criando tabelas temporárias.

EXEMPLO

```
SELECT dept_stats.Nome_Dept, dept_stats.Qtd_Funcionarios, dept_stats.
Media_Salarial FROM ( SELECT d.Nome_Dept, COUNT(f.ID_Func) AS Qtd_
Funcionarios, AVG(f.Salario) AS Media_Salarial FROM DEPARTAMENTOS_SUB
d LEFT JOIN FUNCIONARIOS_SUB f ON d.ID_Dept = f.ID_Dept GROUP BY d.ID_
Dept, d.Nome_Dept ) AS dept_stats WHERE dept_stats.Qtd_Funcionarios
> 1;
```

Resultado:

Nome_Dept	Qtd_Funcionarios	Media_Salarial
Vendas	2	4750.00
Marketing	2	5750.00

5. Cláusulas Especiais (EXISTS, ANY e ALL)

5.1. EXISTS (Verificação de Existência)

O operador **EXISTS** é usado para **testar se uma subconsulta retorna pelo menos um registro**.

Características do EXISTS:

Característica	Descrição	Vantagem
Retorna TRUE/FALSE	Não importa quantos registros, só se existe	Performance otimizada
Para na primeira linha	Não processa toda a subconsulta	Mais rápido que IN
Ignora NULL	NULL não afeta o resultado	Mais seguro que NOT IN
Permite correlação	Pode referenciar tabela externa	Mais flexível

EXEMPLO

Exemplo 1 – Departamentos que Têm Funcionários (EXISTS)

```
SELECT Nome_Dept FROM DEPARTAMENTOS_SUB d WHERE EXISTS ( SELECT 1 FROM
FUNCIONARIOS_SUB f WHERE f.ID_Dept = d.ID_Dept );
```

Exemplo 2 – Departamentos sem Funcionários (NOT EXISTS)

```
SELECT Nome_Dept FROM DEPARTAMENTOS_SUB d WHERE NOT EXISTS ( SELECT 1
FROM FUNCIONARIOS_SUB f WHERE f.ID_Dept = d.ID_Dept );
```

5.1.1. EXISTS vs IN

Aspecto	EXISTS	IN
Performance	Melhor	Pior (processa tudo)
Comportamento com NULL	Ignora	Problemático com NOT IN
Correlação	Suporta	Limitado
Legibilidade	Menos intuitivo	Mais intuitivo
Uso recomendado	Subconsultas grandes	Listas pequenas

5.2. ANY (Qualquer Valor)

O operador **ANY** (ou **SOME**) **compara um valor com qualquer valor retornado por uma subconsulta.**

Sintaxe:

valor operador ANY (subconsulta)

EXEMPLO

Exemplo 1 – Funcionários com Salário Maior que Qualquer Funcionário de TI

```
SELECT Nome, Salario FROM FUNCIONARIOS_SUB WHERE Salario > ANY ( SELECT
    Salario FROM FUNCIONARIOS_SUB f JOIN DEPARTAMENTOS_SUB d ON f.ID_Dept
    = d.ID_Dept WHERE d.Nome_Dept = 'TI' );
```

Explicação: retorna funcionários com salário > 7000.00 (qualquer salário de TI).

Exemplo 2 – Equivalência ANY com IN

- Estas consultas são equivalentes: `SELECT * FROM tabela WHERE coluna = ANY (subconsulta);` `SELECT * FROM tabela WHERE coluna IN (subconsulta);`

5.3. ALL (Todos os Valores)

O operador ALL compara um valor com todos os valores retornados por uma subconsulta.

EXEMPLO

Exemplo 1 – Funcionários com Salário Maior que Todos os Funcionários de Vendas

```
SELECT Nome, Salario FROM FUNCIONARIOS_SUB WHERE Salario > ALL ( SELECT
    Salario FROM FUNCIONARIOS_SUB f JOIN DEPARTAMENTOS_SUB d ON f.ID_Dept
    = d.ID_Dept WHERE d.Nome_Dept = 'Vendas' );
```

Explicação: retorna funcionários com salário > 5000.00 (maior que TODOS os salários de Vendas).

Exemplo 2 – Equivalências com ALL

- Estas consultas são equivalentes: WHERE coluna > ALL (subconsulta) WHERE coluna > (SELECT MAX(coluna) FROM subconsulta) WHERE coluna < ALL (subconsulta) WHERE coluna < (SELECT MIN(coluna) FROM subconsulta)

5.3.1. Tabela Comparativa ANY vs ALL

Operador	Significado	Equivalência	Exemplo
= ANY	Igual a qualquer	IN	WHERE id = ANY (1,2,3)
> ANY	Maior que o menor	> MIN()	WHERE sal > ANY (1000,2000)
< ANY	Menor que o maior	< MAX()	WHERE sal < ANY (1000,2000)
= ALL	Igual a todos (raro)	Valor único	WHERE id = ALL (5)
> ALL	Maior que o maior	> MAX()	WHERE sal > ALL (1000,2000)
< ALL	Menor que o menor	< MIN()	WHERE sal < ALL (1000,2000)

6. Controle de Resultado (TOP, LIMIT e OFFSET)

6.1. Limitação de Registros

Diferentes SGBDs utilizam sintaxes distintas para limitar o número de registros retornados por uma consulta.

SGBD	Sintaxe	Exemplo	Observações
SQL Server	TOP n	SELECT TOP 5 * FROM tabela	No início do SELECT
MySQL	LIMIT n	SELECT * FROM tabela LIMIT 5	No final da consulta
PostgreSQL	LIMIT n	SELECT * FROM tabela LIMIT 5	Padrão SQL
Oracle	ROWNUM (legado)	WHERE ROWNUM <= 5	Método antigo
Oracle 12c+	FETCH FIRST n ROWS ONLY	FETCH FIRST 5 ROWS ONLY	Padrão SQL moderno
SQLite	LIMIT n	SELECT * FROM tabela LIMIT 5	Similar ao MySQL

EXEMPLO

Exemplo 1 – Top 3 Funcionários com Maiores Salários

- SQL Server SELECT TOP 3 Nome, Salario FROM FUNCIONARIOS_SUB ORDER BY Salario DESC;
- MySQL/PostgreSQL SELECT Nome, Salario FROM FUNCIONARIOS_SUB ORDER BY Salario DESC LIMIT 3;
- Oracle 12c+ SELECT Nome, Salario FROM FUNCIONARIOS_SUB ORDER BY Salario DESC FETCH FIRST 3 ROWS ONLY;

Resultado:

	Salario
Ana	7000.00
Maria	6000.00
	5500.00

6.2. OFFSET (Paginação de Resultados)

OFFSET permite pular um número específico de registros, essencial para paginação.

Sintaxe padrão SQL:

SELECT colunas FROM tabela ORDER BY coluna LIMIT quantidade OFFSET inicio;

EXEMPLO

Paginação (página 2, 2 registros por página)

- MySQL/PostgreSQL SELECT Nome, Salario FROM FUNCIONARIOS_SUB ORDER BY Nome LIMIT 2 OFFSET 2; -- SQL Server (2012+) SELECT Nome, Salario FROM FUNCIONARIOS_SUB ORDER BY Nome OFFSET 2 ROWS FETCH NEXT 2 ROWS ONLY;

Resultado (registros 3-4):

	Salario
Maria	6000.00
Pedro	4500.00

6.3. Casos de Uso Práticos

Tabela de aplicações comuns:

Caso de Uso	Sintaxe Exemplo	Aplicação
Top N registros	SELECT TOP 10 * FROM vendas ORDER BY valor	Melhores vendedores
Paginação web	LIMIT 20 OFFSET 40	Página 3 de resultados
Amostragem	SELECT * FROM clientes LIMIT 100	Teste com subset
Ranking	TOP 5 WITH TIES ORDER BY pontuacao	Classificação com empates
Performance	LIMIT 1	Verificar existência

7. Estruturas Condicionais (CASE WHEN)

7.1. Introdução ao CASE WHEN

A estrutura **CASE WHEN** permite **criar lógica condicional dentro de consultas SQL**, similar a estruturas if-else em linguagens de programação.

Duas sintaxes principais:

- **CASE simples:**

CASE expressao WHEN valor1 THEN resultado1 WHEN valor2 THEN resultado2 ELSE resultado_padrao END

- **CASE pesquisado (mais comum):**

CASE WHEN condicao1 THEN resultado1 WHEN condicao2 THEN resultado2 ELSE resultado_padrao END

EXEMPLO

Exemplo 1 – Classificação de Salários

```
SELECT Nome, Salario, CASE WHEN Salario >= 6000 THEN 'Alto' WHEN Salario
    >= 5000 THEN 'Médio' ELSE 'Baixo' END AS Categoria_Salario FROM
    FUNCIONARIOS_SUB;
```

Resultado:

	Salario	Categoria_Salario
João		Médio
Maria		Alto
Pedro		Baixo
Ana		Alto
		Médio

Exemplo 2 – CASE com Agregação

```
SELECT d.Nome_Dept, COUNT(f.ID_Func) AS Total_Funcionarios, SUM(CASE
    WHEN f.Salario > 5000 THEN 1 ELSE 0 END) AS Funcionarios_Alto_Salario,
    AVG(CASE WHEN f.Salario > 5000 THEN f.Salario END) AS Media_Alto_Salario
    FROM DEPARTAMENTOS_SUB d LEFT JOIN FUNCIONARIOS_SUB f ON d.ID_Dept =
    f.ID_Dept GROUP BY d.ID_Dept, d.Nome_Dept;
```

Resultado:

Nome_Dept	Total_Funcionarios	Funcionarios_Alto_Salario	Media_Alto_Salario
Vendas	2	1	5000.00
Marketing	2	2	5750.00
TI	1	1	7000.00

7.2. CASE em Different Cláusulas

7.2.1. No SELECT (mais comum)

```
SELECT nome, CASE WHEN idade < 30 THEN 'Jovem' ELSE 'Experiente' END AS
    perfil FROM funcionarios;
```

7.2.2. No WHERE

```
SELECT * FROM funcionarios WHERE CASE WHEN departamento = 'TI' THEN salario > 6000 ELSE salario > 4000 END;
```

7.2.3. No ORDER BY

```
SELECT nome, departamento, salario FROM funcionarios ORDER BY CASE departamento WHEN 'TI' THEN 1 WHEN 'Vendas' THEN 2 ELSE 3 END, salario DESC;
```

7.3. CASE vs Outras Estruturas

Estrutura	Uso Principal	Vantagem	Limitação
CASE WHEN	Lógica complexa, múltiplas conds	Padrão SQL, muito flexível	Sintaxe mais verbosa
COALESCE	Tratamento de NULL	Simples para NULL	Apenas NULL
NULLIF	Converter valor específico em NULL	Específico para um caso	Uso limitado
IIF	Condição simples (SQL Server)	Sintaxe concisa	Não padrão SQL
DECODE	Múltiplas comparações (Oracle)	Conciso para igualdades	Específico do Oracle

8. Comparação: JOINS vs Subconsultas

8.1. Quando Usar Cada Abordagem

Cenário	Recomendação	Justificativa
Combinar dados de tabelas	JOIN	Performance e legibilidade
Filtrar baseado em existência	EXISTS	Melhor performance que IN
Filtrar baseado em valores	IN/ANY/ALL	Mais legível para listas
Cálculos complexos	Subconsulta	Isolamento da lógica
Múltiplas tabelas	JOIN	Controle melhor das relações
Condições correlacionadas	EXISTS	Flexibilidade e performance

EXEMPLO

Problema: funcionários de departamentos com orçamento > 120000

Solução com JOIN:

```
SELECT DISTINCT f.Nome, f.Salario
FROM FUNCIONARIOS_SUB f
INNER JOIN DEPARTAMENTOS_SUB d ON f.ID_Dept = d.ID_Dept
WHERE d.Orcamento > 120000;
```

Solução com subconsulta:

```
SELECT Nome, Salario
FROM FUNCIONARIOS_SUB
WHERE ID_Dept IN (
SELECT ID_Dept
FROM DEPARTAMENTOS_SUB
WHERE Orcamento > 120000
);
```

Análise:

JOIN: mais eficiente, permite combinar dados;

Subconsulta: mais legível, lógica isolada.

COMANDOS DE MANIPULAÇÃO: INSERT, UPDATE E DELETE

Os **comandos de manipulação de dados** constituem o núcleo operacional da DML, permitindo **modificar efetivamente o conteúdo das tabelas no banco de dados**. Enquanto o **SELECT permite consultar dados**, os comandos **INSERT, UPDATE e DELETE permitem alterar o estado persistente das informações**.

Características fundamentais:

Aspecto	INSERT	UPDATE	DELETE
Função principal	Adiciona registros	Modifica registros	Remove registros
Afeta estrutura da tabela	Não	Não	Não
Pode usar WHERE	Não	Sim (recomendado)	Sim (recomendado)
Pode usar subconsultas	Sim	Sim	Sim
Reversibilidade	Com transação	Com transação	Com transação
Impacto na performance	Baixo	Médio	Médio
Risco de erro	Baixo	Alto	Muito Alto

1. INSERT (Inserção de Novos Registros)

O comando **INSERT** é utilizado para **adicionar novos registros a uma tabela existente**. É o comando mais seguro entre os três, pois não modifica dados existentes, apenas adiciona novos.

EXEMPLO

Para demonstrar os comandos de manipulação, utilizaremos uma tabela PRODUTOS:

Estrutura da Tabela PRODUTOS:

```
CREATE TABLE PRODUTOS ( ID_Produto INT PRIMARY KEY, Nome_Produto
    VARCHAR(100) NOT NULL, Categoria VARCHAR(50), Preco DECIMAL(10,2),
    Estoque INT DEFAULT 0, Data_Cadastro DATE, Ativo BOOLEAN DEFAULT TRUE
    );
```

Dados Iniciais da Tabela PRODUTOS:

ID_Produto	Nome_Produto	Categoria	Preco	Estoque	Data_Cadastro	
1	Notebook Dell	Informática	2500.00	10	2024-01-15	TRUE
2	Mouse Logitech	Informática	50.00	25	2024-01-16	TRUE
3	Cadeira Office	Móveis	300.00	5	2024-01-17	TRUE

1.1. INSERT com Valores Explícitos

A forma mais comum do INSERT especifica valores explícitos para as colunas da tabela.

Sintaxe básica:

```
INSERT INTO tabela (coluna1, coluna2, coluna3) VALUES (valor1, valor2, valor3);
```

EXEMPLO

Exemplo 1 – INSERT Completo

```
INSERT INTO PRODUTOS (ID_Produto, Nome_Produto, Categoria, Preco, Estoque,
    Data_Cadastro, Ativo) VALUES (4, 'Teclado Mecânico', 'Informática',
    150.00, 15, '2024-01-20', TRUE);
```

Resultado após INSERT:

ID_Produto	Nome_Produto	Categoria	Preco	Estoque	Data_Cadastro	
1	Notebook Dell			10	2024-01-15	TRUE
2	Mouse Logitech		50.00	25	2024-01-16	TRUE
3	Cadeira Office	Móveis	300.00	5	2024-01-17	TRUE
4	Teclado Mecânico		150.00	15	2024-01-20	TRUE

Exemplo 2 – INSERT Parcial (usando defaults)

```
INSERT INTO PRODUTOS (ID_Produto, Nome_Produto, Categoria, Preco) VALUES
    (5, 'Monitor 24"', 'Informática', 400.00);
```

Obs.: As colunas não especificadas (Estoque, Data_Cadastro, Ativo) receberão valores DEFAULT ou NULL.

EXEMPLO

Exemplo 3 – INSERT sem Especificar Colunas

```
INSERT INTO PRODUTOS VALUES (6, 'Mesa Escritório', 'Móveis', 250.00,
    3, '2024-01-21', TRUE);
```

Obs.: Esta sintaxe requer valores para todas as colunas na ordem exata da definição da tabela.

1.2. INSERT Múltiplo

Muitos SGBDs suportam inserção de múltiplos registros em um único comando:

```
INSERT INTO PRODUTOS (ID_Produto, Nome_Produto, Categoria, Preço, Estoque)
VALUES (7, 'Impressora HP', 'Informática', 300.00, 8), (8, 'Scanner Canon',
'Informática', 200.00, 12), (9, 'Luminária LED', 'Iluminação', 80.00, 20);
```

Vantagens do INSERT Múltiplo:

- **Performance:** mais rápido que múltiplos INSERTs individuais;
- **Transação:** todos os registros são inseridos ou nenhum (atomicidade);
- **Menos código:** sintaxe mais concisa.

1.3. INSERT com Subconsulta

O INSERT pode utilizar subconsultas para inserir dados baseados em consultas existentes.

Sintaxe:

```
INSERT INTO tabela (colunas) SELECT colunas FROM outra_tabela WHERE condição;
```

EXEMPLO

Vamos criar uma tabela de produtos em promoção:

```
CREATE TABLE PRODUTOS_PROMOCAO ( ID_Produto INT, Nome_Produto VARCHAR(100),
Preco_Original DECIMAL(10,2), Preco_Promocional DECIMAL(10,2) );
INSERT INTO PRODUTOS_PROMOCAO (ID_Produto, Nome_Produto, Preco_Original,
Preco_Promocional) SELECT ID_Produto, Nome_Produto, Preço, Preço *
0.8 FROM PRODUTOS WHERE Categoria = 'Informática' AND Preço > 100.00;
```

Resultado:

ID_Produto	Nome_Produto	Preco_Original	Preco_Promocional
1	Notebook Dell	2500.00	2000.00
4	Teclado Mecânico	150.00	120.00
5	Monitor 24"	400.00	320.00

1.4. Regras e Restrições do INSERT

Regra	Descrição	Exemplo de Erro
Chave primária única	Não pode inserir ID já existente	INSERT ID_Produto = 1 (já existe)
Colunas NOT NULL	Deve fornecer valor para colunas obrigatórias	Nome_Produto não pode ser NULL

Regra	Descrição	Exemplo de Erro
Tipos de dados compatíveis	Valores devem corresponder ao tipo da coluna	'ABC' em coluna numérica
Restrições de integridade	Chaves estrangeiras devem existir	ID_Categoria inexistente
Valores DEFAULT	Colunas omitidas recebem valor padrão	Estoque = 0 se não especificado

2. UPDATE (Modificação de Dados Existentes)

O comando **UPDATE** é utilizado para **modificar dados existentes em uma ou mais linhas de uma tabela**. É um comando muito útil, mas perigoso, pois pode alterar muitos registros simultaneamente.

2.1. UPDATE Básico com WHERE

A forma mais segura do UPDATE sempre inclui uma cláusula WHERE para limitar quais registros serão modificados.

Sintaxe básica:

```
UPDATE tabela SET coluna1 = valor1, coluna2 = valor2 WHERE condição;
```

EXEMPLO

Exemplo 1 – UPDATE de um Registro Específico

```
UPDATE PRODUTOS SET Preço = 2400.00, Estoque = 8 WHERE ID_Produto = 1;
```

Antes:

ID_Produto	Nome_Produto	Preço	Estoque
1	Notebook Dell		10

Depois:

ID_Produto	Nome_Produto	Preço	Estoque
1	Notebook Dell		8

Exemplo 2 – UPDATE de Múltiplos Registros

```
UPDATE PRODUTOS SET Preço = Preço * 1.1 WHERE Categoria = 'Informática';
```

Obs.: Este comando aumenta o preço em 10% para todos os produtos de informática.

EXEMPLO

Exemplo 3 – UPDATE com Cálculos

```
UPDATE PRODUTOS SET Preço = CASE WHEN Estoque > 20 THEN Preço * 0.95
-- Desconto para alto estoque WHEN Estoque < 5 THEN Preço * 1.05 --
```

Aumento para baixo estoque ELSE Preco -- Mantém preço atual END WHERE Categoria = 'Informática';

2.2. UPDATE com Subconsultas

O **UPDATE** pode utilizar **subconsultas tanto na cláusula SET quanto na cláusula WHERE**.

EXEMPLO

Exemplo 1 – UPDATE Baseado em Outra Tabela

- Assumindo uma tabela FORNECEDORES com novos preços UPDATE PRODUTOS SET Preco = (SELECT Novo_Preco FROM FORNECEDORES f WHERE f.ID_Produto = PRODUTOS.ID_Produto) WHERE EXISTS (SELECT 1 FROM FORNECEDORES f WHERE f.ID_Produto = PRODUTOS.ID_Produto);

Exemplo 2 – UPDATE com Agregação

- Atualizar estoque baseado na média da categoria UPDATE PRODUTOS SET Estoque = (SELECT AVG(Estoque) FROM PRODUTOS p2 WHERE p2.Categoria = PRODUTOS.Categoria) WHERE Estoque < 5;

2.3. UPDATE com JOINS (Quando Suportado)

Alguns SGBDs permitem UPDATE com JOINS.

- Sintaxe MySQL/SQL Server UPDATE PRODUTOS p JOIN CATEGORIAS c ON p.Categoria = c.Nome_Categoria SET p.Preco = p.Preco * c.Fator_Ajuste WHERE c.Ativo = TRUE;

2.4. Cuidados Especiais com UPDATE

Cuidado	Descrição	Prevenção
UPDATE sem WHERE	Modifica TODOS os registros da tabela	Sempre usar WHERE
Condição WHERE incorreta	Modifica registros não intencionados	Testar com SELECT primeiro
Perda de dados original	Valores antigos são perdidos	Backup antes de grandes UPDATES
Deadlocks em transações	Bloqueios mútuos entre transações	Ordem consistente de UPDATES
Performance degradada	UPDATE de muitos registros pode ser lento	Processar em lotes menores

3. DELETE (Remoção de Registros)

O comando **DELETE** é utilizado para **remover registros de uma tabela**. É o comando mais perigoso da DML, pois pode eliminar dados permanentemente.

3.1. DELETE Básico com WHERE

Sintaxe básica:

DELETE FROM tabela WHERE condição;

EXEMPLO

Exemplo 1 – DELETE de um Registro Específico

DELETE FROM PRODUTOS WHERE ID_Produto = 9;

Exemplo 2 – DELETE com Múltiplas Condições

DELETE FROM PRODUTOS WHERE Categoria = 'Informática' AND Preço < 100.00
AND Estoque = 0;

Exemplo 3 – DELETE com Subconsulta

DELETE FROM PRODUTOS WHERE ID_Produto IN (SELECT ID_Produto FROM
PRODUTOS_DESCONTINUADOS);

3.2. DELETE vs TRUNCATE vs DROP

Aspecto	DELETE	TRUNCATE	DROP
Tipo de comando	DML	DDL	DDL
Remove dados	Sim	Sim	Sim
Remove estrutura	Não	Não	Sim
Pode usar WHERE	Sim	Não	Não
Pode ser revertido	Sim (com transação)	Não (em alguns SGBDs)	Não
Velocidade	Lenta	Rápida	Rápida
Dispara triggers	Sim	Não	Não
Libera espaço	Não imediatamente	Sim	Sim
Reseta AUTO_INCREMENT	Não	Sim	Sim

EXEMPLO

DELETE:

DELETE FROM PRODUTOS WHERE Categoria = 'Móveis'; -- Remove apenas
produtos de móveis, mantém estrutura da tabela

TRUNCATE:

TRUNCATE TABLE PRODUTOS; -- Remove TODOS os registros, mantém estrutura,
reseta contadores

DROP:

DROP TABLE PRODUTOS; -- Remove a tabela completamente (dados + estrutura)

3.3. DELETE com JOINS

Alguns SGBDs permitem DELETE com JOINS para remoções baseadas em relacionamentos.

- Sintaxe MySQL DELETE p FROM PRODUTOS p JOIN CATEGORIAS c ON p.Categoria = c.Nome_Categoria WHERE c.Descontinuada = TRUE;
- Sintaxe SQL Server DELETE PRODUTOS FROM PRODUTOS p JOIN CATEGORIAS c ON p.Categoria = c.Nome_Categoria WHERE c.Descontinuada = TRUE;

3.4. Estratégias de Segurança para DELETE

Estratégia	Descrição	Exemplo
Sempre usar WHERE	Nunca execute DELETE sem condição	DELETE FROM tabela WHERE id = 1
Testar com SELECT	Verificar quais registros serão afetados	SELECT * FROM tabela WHERE condição
Usar transações	Permitir rollback em caso de erro	BEGIN; DELETE...; COMMIT/ROLLBACK;
Backup antes de grandes DELETES	Proteger contra perda acidental	Backup da tabela ou database
Usar LIMIT quando possível	Limitar número de registros removidos	DELETE FROM tabela WHERE... LIMIT 100

4. Transações e Controle de Integridade

4.1. Conceito de Transações

Uma **transação** é uma **unidade lógica de trabalho que agrupa uma ou mais operações DML**, garantindo que todas sejam executadas com sucesso ou todas sejam desfeitas.

Propriedades ACID:

Propriedade	Nome	Descrição	Exemplo
A	Atomicidade	Tudo ou nada – operação indivisível	Transferência bancária completa
C	Consistência	Dados sempre em estado válido	Salos nunca negativos
I	Isolamento	Transações não interferem entre si	Leituras consistentes
D	Durabilidade	Mudanças persistem após confirmação	Dados salvos permanentemente

4.2. Comandos de Controle de Transação

```
-- Iniciar transação (implícito em muitos SGBDs) BEGIN TRANSACTION; -- ou
START TRANSACTION; -- Operações DML INSERT INTO PRODUTOS (...) VALUES
(...); UPDATE PRODUTOS SET... WHERE...; DELETE FROM PRODUTOS WHERE...; --
Confirmar mudanças COMMIT; -- OU desfazer mudanças ROLLBACK;
```

EXEMPLO

```
BEGIN TRANSACTION; -- Transferir estoque entre produtos UPDATE PRODUTOS
SET Estoque = Estoque - 5 WHERE ID_Produto = 1; UPDATE PRODUTOS SET
Estoque = Estoque + 5 WHERE ID_Produto = 2; -- Verificar se ambas as
operações foram bem-sucedidas -- Se alguma falhou, fazer ROLLBACK --
Se ambas foram bem-sucedidas, fazer COMMIT COMMIT;
```

LÓGICA DE TRÊS ESTADOS E TRATAMENTO DE NULL

No SQL, o **tratamento de valores NULL** introduz uma **lógica de três estados**, diferente da lógica booleana tradicional, que possui apenas TRUE e FALSE.

Isso ocorre porque um valor NULL representa um dado desconhecido, e qualquer operação que envolva NULL pode resultar em um terceiro estado, denominado UNKNOWN (desconhecido).

Os três estados dessa lógica são:

- **TRUE (T – Verdadeiro):** quando a condição é verdadeira;
- **FALSE (F – Falso):** quando a condição é falsa;
- **UNKNOWN (? – Desconhecido):** quando o resultado é incerto, geralmente por envolver valores **NULL**.

A seguir, a tabela-verdade que demonstra como o SQL avalia expressões lógicas com esses três estados:

NOT A	A	B	A AND B	A OR B
F	T	T	T	T
F	T	F	F	T
T	F	?	?	T
T	F	F	F	F
?	?	?	?	?
?	?	?	?	?

1. Conceito Fundamental de NULL

NULL não é um valor, mas sim a ausência de valor. Representa informação desconhecida, não aplicável ou indefinida. Esta distinção é crucial para compreender seu comportamento peculiar em operações SQL.

Conceito	Descrição	Exemplo
NULL não é valor	Representa ausência de informação	Idade desconhecida
NULL não é zero	Zero é um valor, NULL é ausência	0 ≠ NULL
NULL não é string vazia	"" é um valor, NULL é ausência	"" ≠ NULL
NULL não é espaço	" " é um valor, NULL é ausência	" " ≠ NULL
NULL é contagioso	Operações com NULL geralmente retornam NULL	5 + NULL = NULL

EXEMPLO

Tabela base para exemplos

Para demonstrar o comportamento de NULL, utilizaremos uma tabela FUNCIONARIOS_NULL:

Estrutura da Tabela FUNCIONARIOS_NULL:

```
CREATE TABLE FUNCIONARIOS_NULL ( ID_Func INT PRIMARY KEY, Nome VARCHAR(50)
NOT NULL, Idade INT, Salario DECIMAL(10,2), Comissao DECIMAL(10,2),
Telefone VARCHAR(20), Email VARCHAR(100) );
```

Dados da Tabela FUNCIONARIOS_NULL:

ID_Func	Nome	Idade	Salario	Comissao	Telefone	Email
1	João	30	5000.00	500.00	11999999999	joao@empresa.com
2	Maria	NULL	6000.00	NULL	11888888888	maria@empresa.com
3	Pedro	25	4500.00	200.00	NULL	pedro@empresa.com
4	Ana	35	NULL	NULL	11777777777	NULL
5	Carlos	NULL	5500.00	300.00	NULL	carlos@empresa.com

2. NULL em Comparações e Operadores Lógicos

2.1. Comportamento de NULL em Comparações

O comportamento de NULL em comparações é contraintuitivo. A regra fundamental é: **qualquer comparação com NULL resulta em UNKNOWN.**

2.1.1. Comparações Básicas:

Expressão	Resultado	Explicação
NULL = NULL	UNKNOWN	Não sabemos se dois valores desconhecidos são iguais
NULL <> NULL	UNKNOWN	Não sabemos se dois valores desconhecidos são diferentes
NULL > 5	UNKNOWN	Não sabemos se valor desconhecido é maior que 5
5 < NULL	UNKNOWN	Não sabemos se 5 é menor que valor desconhecido
NULL IS NULL	TRUE	Teste específico para verificar se é NULL
NULL IS NOT NULL	FALSE	Teste específico para verificar se não é NULL

EXEMPLO

Exemplo 1 – Comparação Direta (INCORRETA)

SELECT * FROM FUNCIONARIOS_NULL WHERE Idade = NULL;

Resultado: nenhum registro retornado (mesmo que existam idades NULL)

Exemplo 2 – Teste Correto para NULL

SELECT * FROM FUNCIONARIOS_NULL WHERE Idade IS NULL;

Resultado:

ID_Func		Idade	Salario	Comissao	Telefone	Email
2	Maria	NULL		NULL	11888888888	maria@empresa.com
5		NULL		300.00	NULL	carlos@empresa.com

2.2. NULL em Operadores Lógicos

Os **operadores lógicos AND, OR e NOT têm comportamento específico com NULL.**

2.2.1. Operador AND

A	B	A AND B
TRUE	TRUE	TRUE
TRUE	FALSE	FALSE
TRUE	UNKNOWN	UNKNOWN
FALSE	TRUE	FALSE
FALSE	FALSE	FALSE
FALSE	UNKNOWN	FALSE
UNKNOWN	TRUE	UNKNOWN
UNKNOWN	FALSE	FALSE
UNKNOWN	UNKNOWN	UNKNOWN

2.2.2. Operador OR

A	B	A OR B
TRUE	TRUE	TRUE
TRUE	FALSE	TRUE
TRUE	UNKNOWN	TRUE
FALSE	TRUE	TRUE
FALSE	FALSE	FALSE
FALSE	UNKNOWN	UNKNOWN
UNKNOWN	TRUE	TRUE
UNKNOWN	FALSE	UNKNOWN
UNKNOWN	UNKNOWN	UNKNOWN

2.2.3. Operador NOT

A	NOT A
TRUE	FALSE
FALSE	TRUE
UNKNOWN	UNKNOWN

EXEMPLO

Exemplo 1 – AND com NULL

SELECT Nome, Idade, Salario FROM FUNCIONARIOS_NULL WHERE Idade > 30 AND Salario > 5000;

Resultado: apenas registros onde AMBAS as condições são TRUE (exclui NULLs).

Exemplo 2 – OR com NULL

SELECT Nome, Idade, Comissao FROM FUNCIONARIOS_NULL WHERE Idade IS NULL OR Comissao IS NULL;

Resultado:

	Idade	Comissao
Maria	NULL	NULL
Ana	35	NULL
	NULL	300.00

2.3. NULL em Operadores Especiais

EXEMPLO

1. BETWEEN com NULL

```
SELECT * FROM FUNCIONARIOS_NULL WHERE Idade BETWEEN 25 AND NULL;
```

Resultado: nenhum registro (BETWEEN com NULL sempre retorna UNKNOWN)

2. IN com NULL

```
SELECT * FROM FUNCIONARIOS_NULL WHERE Idade IN (25, 30, NULL);
```

Resultado: apenas registros com Idade = 25 ou Idade = 30 (NULL é ignorado)

3. LIKE com NULL

```
SELECT * FROM FUNCIONARIOS_NULL WHERE Nome LIKE NULL;
```

Resultado: nenhum registro (LIKE com NULL sempre retorna UNKNOWN)

3. NULL em Funções Agregadas

As funções agregadas têm comportamento específico com NULL. A regra geral é: **funções agregadas ignoram valores NULL, exceto COUNT(*)**.

3.1. Comportamento Detalhado por Função

Função	Comportamento com NULL	Exemplo
COUNT(*)	Conta TODAS as linhas, incluindo NULLs	COUNT(*) = 5 (conta tudo)
COUNT(coluna)	Conta apenas valores NÃO NULL	COUNT(Idade) = 3 (ignora NULLs)
SUM(coluna)	Soma apenas valores NÃO NULL	SUM(Comissao) ignora NULLs
AVG(coluna)	Média apenas de valores NÃO NULL	AVG(Idade) = (30+25+35)/3
MAX(coluna)	Maior valor NÃO NULL	MAX(Salario) ignora NULLs
MIN(coluna)	Menor valor NÃO NULL	MIN(Idade) ignora NULLs

EXEMPLO

Exemplo 1 – Diferença entre COUNT(*) e COUNT(coluna)

```
SELECT COUNT(*) AS Total_Funcionarios, COUNT(Idade) AS Funcionarios_
Com_Idade, COUNT(Comissao) AS Funcionarios_Com_Comissao FROM
FUNCIONARIOS_NULL;
```

Resultado:

Total_Funcionarios	Funcionarios_Com_Idade	Funcionarios_Com_Comissao
5	3	3

Exemplo 2 – SUM e AVG ignorando NULL

```
SELECT SUM(Salario) AS Soma_Salarios, AVG(Salario) AS Media_Salarios,
SUM(Comissao) AS Soma_Comissoes, AVG(Comissao) AS Media_Comissoes
FROM FUNCIONARIOS_NULL;
```

Resultado:

Soma_Salarios	Media_Salarios	Soma_Comissoes	Media_Comissoes
21000.00	5250.00	1000.00	333.33

Observação crítica:

AVG(Salario) = $21000 \div 4 = 5250$ (Ana com salário NULL é ignorada);

AVG(Comissao) = $1000 \div 3 = 333.33$ (Maria e Ana com comissão NULL são ignoradas).

Exemplo 3 – MAX e MIN com NULL

SELECT MAX(Idade) AS Maior_Idade, MIN(Idade) AS Menor_Idade, MAX(Salario) AS Maior_Salario, MIN(Salario) AS Menor_Salario FROM FUNCIONARIOS_NULL;

Resultado:

Maior_Idade	Menor_Idade	Maior_Salario	Menor_Salario
35	25	6000.00	4500.00

3.2. NULL em GROUP BY

O **GROUP BY** trata NULL como um valor específico, agrupando todos os NULLs juntos.

EXEMPLO

SELECT Idade, COUNT(*) AS Quantidade, AVG(Salario) AS Media_Salario FROM FUNCIONARIOS_NULL GROUP BY Idade ORDER BY Idade;

Resultado:

Idade	Quantidade	Media_Salario
NULL	2	5750.00
25	1	4500.00
30	1	5000.00
35	1	NULL

Obs.: NULL forma seu próprio grupo, contendo Maria e Carlos.

4. NULL em JOINS e Subconsultas

4.1. NULL em Condições de JOIN

NULL em condições de JOIN pode produzir resultados inesperados. A regra fundamental:
NULL não faz match com NULL em condições de JOIN.

EXEMPLO

Exemplo com Tabelas Relacionadas

Tabela DEPARTAMENTOS_NULL:

```
CREATE TABLE DEPARTAMENTOS_NULL ( ID_Dept INT PRIMARY KEY, Nome_Dept
  VARCHAR(50), ID_Gerente INT );
```

Dados:

ID_Dept	Nome_Dept	ID_Gerente
1	Vendas	1
2	Marketing	NULL
3	TI	3

Exemplo 1 – INNER JOIN com NULL

```
SELECT f.Nome, d.Nome_Dept FROM FUNCIONARIOS_NULL f INNER JOIN
  DEPARTAMENTOS_NULL d ON f.ID_Func = d.ID_Gerente;
```

Resultado:

	Nome_Dept
João	Vendas
Pedro	TI

Obs.: Departamento Marketing (ID_Gerente = NULL) não aparece, mesmo que existam funcionários.

EXEMPLO

Exemplo 2 – LEFT JOIN mostrando NULLs

```
SELECT d.Nome_Dept, f.Nome AS Gerente FROM DEPARTAMENTOS_NULL d LEFT
  JOIN FUNCIONARIOS_NULL f ON d.ID_Gerente = f.ID_Func;
```

Resultado:

Nome_Dept	Gerente
Vendas	João
Marketing	NULL
TI	Pedro

4.2. NULL em Subconsultas

EXEMPLO

Exemplo 1 – IN com NULL

```
SELECT Nome FROM FUNCIONARIOS_NULL WHERE ID_Func IN (SELECT ID_Gerente
  FROM DEPARTAMENTOS_NULL);
```


Resultado:

Nome
João
Pedro

Obs.: NULL na subconsulta é ignorado pelo IN.

EXEMPLO

Exemplo 2 – NOT IN com NULL (PEGADINHA CRÍTICA)

```
SELECT Nome FROM FUNCIONARIOS_NULL WHERE ID_Func NOT IN (SELECT ID_Gerente FROM DEPARTAMENTOS_NULL);
```

Resultado: nenhum registro (pegadinha comum!)

Explicação: NOT IN com NULL na subconsulta sempre retorna FALSE para todos os registros.

Solução Correta para NOT IN com NULL:

```
SELECT Nome FROM FUNCIONARIOS_NULL WHERE ID_Func NOT IN ( SELECT ID_Gerente FROM DEPARTAMENTOS_NULL WHERE ID_Gerente IS NOT NULL );
```

5. Funções para Tratamento de NULL

5.1. Funções Padrão SQL

EXEMPLO

1. COALESCE – Primeira Não NULL

```
SELECT Nome, COALESCE(Telefone, Email, 'Sem contato') AS Contato_Principal FROM FUNCIONARIOS_NULL;
```

Resultado:

	Contato_Principal
João	11999999999
Maria	11888888888
Pedro	pedro@empresa.com
Ana	11777777777
	carlos@empresa.com

2. NULLIF – Retorna NULL se Iguais

```
SELECT Nome, NULLIF(Comissao, 0) AS Comissao_Valida FROM FUNCIONARIOS_NULL;
```

5.2. Funções Específicas por SGBD

SGBD	Função	Sintaxe	Descrição
SQL Server	ISNULL	ISNULL(campo, substituto)	Substitui NULL por valor
MySQL	IFNULL	IFNULL(campo, substituto)	Substitui NULL por valor
Oracle	NVL	NVL(campo, substituto)	Substitui NULL por valor
PostgreSQL	COALESCE	COALESCE(campo1, campo2)	Primeira não NULL
Padrão SQL	CASE	CASE WHEN campo IS NULL...	Controle condicional

RESUMO

INTRODUÇÃO

Conceitos Fundamentais:

- SQL como linguagem declarativa padrão para bancos relacionais;
- Divisão da SQL em sublinguagens especializadas.

Sublinguagens da SQL:

- DML: manipulação de dados (INSERT, UPDATE, DELETE, SELECT*);
- DDL: definição de estruturas (CREATE, ALTER, DROP, TRUNCATE);
- DCL: controle de acesso (GRANT, REVOKE, DENY);
- DTL: controle de transações (COMMIT, ROLLBACK, SAVEPOINT);
- DQL: consulta de dados (SELECT*) – classificação moderna.

Controvérsia do SELECT:

- Classificação tradicional: SELECT como DML;
- Classificação moderna: SELECT como DQL.

Comandos DML:

Características comuns: transacionais, respeitam integridade, podem ser condicionais.

SQL (DML): FUNDAMENTOS DA CONSULTA DE DADOS

Conceitos Fundamentais:

- SELECT realiza operação de PROJEÇÃO (especifica colunas);
- WHERE realiza operação de SELEÇÃO (filtra linhas);
- Diferença conceitual entre SQL e álgebra relacional.

Sintaxe e Variações:

- Estrutura básica: SELECT, FROM, WHERE;
- Seleção específica vs. SELECT *;
- Consultas com e sem condições.

Eliminação de Duplicatas:

- DISTINCT elimina valores duplicados;
- Avaliação considera todos os valores retornados;
- Comportamento com múltiplas colunas.

Alias (Apelidos):

- Melhora legibilidade das consultas;
- Útil para funções agregadas;
- Sintaxe com e sem AS.

Operadores de Comparação:

- Básicos: =, <>, >, <, >=, <=;
- Lógicos: AND, OR, NOT;
- Padrão <> para “diferente”.

Operadores Especiais:

- BETWEEN: intervalos (inclusivo);
- IN: múltiplos valores específicos;
- LIKE: busca por padrões (% e _);
- IS NULL: valores nulos.

Ordenação:

- ORDER BY para organizar resultados;
- ASC (crescente) e DESC (decrecente);
- Ordenação por múltiplas colunas;
- Diferença entre ORDER BY e GROUP BY.

ANÁLISE DE DADOS: FUNÇÕES AGREGADAS E AGRUPAMENTOS**Funções Agregadas Principais:**

- COUNT: contagem (diferença crítica entre COUNT(*) e COUNT(coluna));
- SUM: soma de valores numéricos;
- AVG: média aritmética;
- MAX/MIN: valores máximo e mínimo;
- Tratamento de NULL: todas ignoram valores NULL (exceto COUNT(*)).

GROUP BY – Agrupamento:

- Função: divide registros em grupos para análise;
- Regra fundamental: colunas não agregadas devem estar no GROUP BY;
- Múltiplas colunas: permite agrupamento hierárquico;
- Não aceita alias: deve usar nome original da coluna.

HAVING – Filtragem de Grupos:

- Diferença do WHERE: filtra grupos, não linhas individuais;
- Momento: executa após GROUP BY e funções agregadas;
- Uso: condições com funções agregadas;
- Combinação: pode ser usado junto com WHERE.

Ordem de Execução:

1. FROM: define tabelas;
2. WHERE: filtra linhas;

3. GROUP BY: agrupa registros;
4. HAVING: filtra grupos;
5. SELECT: projeta colunas;
6. ORDER BY: ordena resultado.

Pegadinhas Críticas:

- COUNT(*) vs COUNT(coluna): comportamento com NULL;
- Colunas sem GROUP BY: erro comum em consultas;
- WHERE vs HAVING: momento e propósito diferentes;
- NULL em agregações: valores ignorados automaticamente;
- Ordem lógica: sequência de execução das cláusulas.

Aplicações Práticas:

- Relatórios gerenciais: vendas por região, período, vendedor;
- Análises estatísticas: médias, totais, contagens;
- Filtros complexos: combinação WHERE + HAVING;
- Agrupamentos hierárquicos.

RELACIONAMENTOS ENTRE TABELAS: JOINS E OPERAÇÕES DE CONJUNTO**Produto Cartesiano:**

- Conceito base: combinação de cada linha de A com cada linha de B;
- Problema: volume excessivo ($m \times n$ linhas);
- Solução: usar JOINS com condições adequadas;
- Identificação: consultas sem condição de junção.

Tipos de JOIN:

- INNER JOIN: apenas correspondências mútuas (mais comum);
- LEFT JOIN: preserva tabela da esquerda + correspondências;
- RIGHT JOIN: preserva tabela da direita + correspondências;
- FULL OUTER JOIN: preserva ambas as tabelas (união completa).

JOINS Múltiplos:

- Ordem de execução: da esquerda para a direita;
- Condições múltiplas: uma condição por JOIN;
- Performance: atenção à ordem das tabelas;
- Complexidade: pode envolver 3+ tabelas.

Operações de Conjunto:

- UNION: remove duplicatas automaticamente;
- UNION ALL: preserva todas as linhas;

- INTERSECT: apenas linhas comuns (nem todos SGBDs);
- EXCEPT: diferença entre conjuntos (nem todos SGBDs).

Regras Críticas:

- Compatibilidade: mesmo número e tipo de colunas;
- NULL em JOINS: NULL não faz match com NULL;
- Performance: INNER JOIN > LEFT/RIGHT > FULL OUTER;
- Sintaxe: ON para condições, WHERE para filtros.

Subconsultas:

- 5 tipos: escalar, linha, tabela, correlacionada, não correlacionada;
- Posicionamento: SELECT, FROM, WHERE, HAVING;
- Performance: correlacionadas vs não correlacionadas;
- Casos de uso: filtros complexos, cálculos avançados.

Cláusulas Especiais:

- EXISTS: “cláusula querida da FGV”, otimizada para existência;
- ANY/ALL: comparações com conjuntos de valores;
- Equivalências: EXISTS vs IN, ANY vs MIN/MAX, ALL vs MIN/MAX.

Controle de Resultado:

- Limitação: TOP, LIMIT, FETCH FIRST (por SGBD);
- Paginação: OFFSET para navegação de resultados;
- Casos de uso: ranking, amostragem, performance.

Estruturas Condicionais:

- CASE WHEN: lógica condicional em SQL;
- Duas sintaxes: simples vs pesquisado;
- Aplicações: SELECT, WHERE, ORDER BY, agregações.

Comparações Estratégicas:

- JOINS vs Subconsultas: quando usar cada abordagem;
- EXISTS vs IN: performance e comportamento com NULL;
- ANY vs ALL: significados e equivalências.

Pegadinhas Principais:

- JOIN sem condição: gera produto cartesiano;
- Confusão LEFT/RIGHT: qual tabela é preservada;
- NULL behavior: NULL ≠ NULL em comparações;
- UNION compatibility: regras de compatibilidade;
- WHERE vs ON: momento da aplicação do filtro;
- NOT IN com NULL: resultado vazio inesperado;

- EXISTS vs IN: diferenças de performance;
- CASE sem ELSE: resultado NULL para casos não cobertos.

COMANDOS DE MANIPULAÇÃO: INSERT, UPDATE E DELETE

INSERT – Inserção de Dados:

- Sintaxe básica: INSERT INTO tabela (colunas) VALUES (valores);
- INSERT múltiplo: múltiplos registros em um comando;
- INSERT com subconsulta: dados baseados em outras tabelas;
- Regras: chave primária única, NOT NULL, tipos compatíveis;
- Segurança: comando mais seguro, não modifica dados existentes.

UPDATE – Modificação de Dados:

- Sintaxe básica: UPDATE tabela SET coluna=valor WHERE condição;
- UPDATE com cálculos: operações matemáticas e CASE;
- UPDATE com subconsultas: valores baseados em outras consultas;
- Cuidados críticos: sempre usar WHERE, testar com SELECT primeiro;
- Risco: alto potencial de modificar dados não intencionados.

DELETE – Remoção de Dados:

- Sintaxe básica: DELETE FROM tabela WHERE condição;
- DELETE vs TRUNCATE vs DROP: diferenças fundamentais;
- DELETE com subconsultas: remoções baseadas em outras tabelas;
- Estratégias de segurança: WHERE obrigatório, backup, transações;
- Risco: muito alto, pode eliminar dados permanentemente.

Diferenças Críticas:

- DELETE: DML, usa WHERE, reversível, dispara triggers;
- TRUNCATE: DDL, remove tudo, rápido, reseta contadores;
- DROP: DDL, remove estrutura, irreversível.

Transações:

- Propriedades ACID: Atomicidade, Consistência, Isolamento, Durabilidade;
- Comandos: BEGIN, COMMIT, ROLLBACK;
- Importância: garantir integridade e permitir recuperação;
- Boas práticas: sempre finalizar com COMMIT ou ROLLBACK.

Pegadinhas Principais:

- UPDATE/DELETE sem WHERE: afeta todos os registros;
- Confusão entre DELETE/TRUNCATE/DROP: tipos e comportamentos;
- INSERT sem especificar colunas: ordem e obrigatoriedade;

- Violações de integridade: chaves estrangeiras e restrições;
- Transações incompletas: falta de COMMIT.

LÓGICA DE TRÊS ESTADOS E TRATAMENTO DE NULL

Conceitos Fundamentais:

- NULL não é valor: representa ausência de informação;
- Três estados: TRUE, FALSE, UNKNOWN (NULL);
- NULL é contagioso: operações com NULL geralmente retornam NULL;
- Diferenças críticas: NULL ≠ 0, NULL ≠ "", NULL ≠ ' '.

Comparações com NULL:

- NULL = NULL: sempre UNKNOWN (nunca TRUE);
- NULL IS NULL: sempre TRUE (teste correto);
- Operadores lógicos: tabelas de verdade com três estados;
- Operadores especiais: BETWEEN, IN, LIKE com NULL.

Funções Agregadas:

- COUNT(*): inclui linhas com NULL;
- COUNT(coluna): ignora valores NULL;
- SUM, AVG, MAX, MIN: ignoram valores NULL;
- GROUP BY: agrupa NULLs juntos.

JOINS e Subconsultas:

- JOIN: NULL não faz match com NULL;
- IN: ignora NULLs na lista;
- NOT IN: retorna vazio se houver NULL na subconsulta;
- EXISTS: alternativa segura para NOT IN.

Funções de Tratamento:

- COALESCE: primeira não NULL (padrão SQL);
- ISNULL: SQL Server;
- IFNULL: MySQL;
- NVL: Oracle;
- CASE: controle condicional universal.

Top 10 Pegadinhas:

1. NULL = NULL (sempre UNKNOWN);
2. COUNT(*) vs COUNT(coluna) (comportamento diferente);
3. NOT IN com NULL (resultado vazio);
4. Cálculos com NULL (resultado NULL);

5. ORDER BY com NULL (posição específica);
6. HAVING com NULL (filtro inadequado);
7. JOIN com NULL (sem correspondência);
8. UNION com NULL (valor distinto);
9. BETWEEN com NULL (sempre UNKNOWN);
10. CASE com NULL (tratamento inadequado).

EXERCÍCIOS

001. (FGV/INPE/TECNOLOGISTA JÚNIOR/ÁREA OPERAÇÃO DE SISTEMAS ESPACIAIS/2024) SQL (Structured Query Language) é uma linguagem usada para interagir com bancos de dados relacionais. Relativamente a SQL, analise as afirmativas a seguir e assinale (V) para a verdadeira e (F) para a falsa.

- () A cláusula WHERE só é utilizada associada ao comando SELECT.
- () O comando CREATE TABLE cria uma nova tabela no banco de dados e o comando INSERT INTO permite popular essa tabela por meio da inserção de registros.
- () O comando INNER JOIN é utilizado para unir os resultados de duas ou mais consultas SELECT e o seu resultado é um conjunto de linhas provenientes das tabelas envolvidas.
- () Os comandos ALTER TABLE e UPDATE são comandos usados para atualizar registros em tabelas de um banco de dados.
- () SQL suporta o conceito de chaves primárias e chaves estrangeiras para manter a integridade dos dados e as relações entre as tabelas.

As afirmativas são, respectivamente:

- a) F – F – F – V – F
- b) F – V – F – F – V
- c) V – V – V – F – V
- d) V – F – V – V – V
- e) F – F – V – F – F

002. (FGV/TJ-MS/TÉCNICO DE NÍVEL SUPERIOR/ANALISTA DE SISTEMAS COMPUTACIONAIS/WEB DESIGNER/2024) Observe o script SQL a seguir:

SQL

```
SELECT COUNT(*) AS [Quantidade], Tipo_Processo FROM Processo GROUP BY Tipo_Processo;
```

O resultado da execução desse script é:

- a) a lista dos registros da tabela quantidade
- b) a quantidade de processos por tipo
- c) a contagem dos registros da tabela de tipos de processos
- d) o agrupamento de processos que realizam contagem
- e) a contagem dos processos relacionados à quantidade de valores

003. (IBFC/SAEB/TÉCNICO EM TECNOLOGIA DA INFORMAÇÃO/ÁREA DESENVOLVIMENTO/2023) A linguagem SQL permite diversas operações sob um conjunto de dados de uma ou mais

tabelas, as necessidades do projeto de software ditando quais cláusulas deverão ser utilizadas, sejam para recuperação, atualização, inserção ou remoção de algum dado. Ao lidarmos com recuperação, os dados que iremos recuperar podem ser filtrados, agrupados etc. Assinale a alternativa que apresenta qual a cláusula SQL responsável para filtrar registros em uma consulta de uma tabela:

- a) GROUP BY
- b) SELECT
- c) FILTER
- d) WHERE

004. (FCC/TJ-CE/ANALISTA JUDICIÁRIO/2022) Uma tabela chamada cliente possui os campos abaixo

id – int (Primary Key) nome – varchar(70) cidade – varchar(40) estado – varchar(40)

Em condições ideais, para exibir os dados de todos os clientes, cujo nome da cidade não seja igual ao nome do estado, utiliza-se a instrução SQL:

SELECT * FROM cliente WHERE

- a) cidade != estado;
- b) cidade!EQUALS estado;
- c) cidade UNLIKE(estado);
- d) cidade <> estado;
- e) cidade UNLIKE estado;

005. (IF SUL RIO-GRANDENSE/IF SUL RIO-GRANDENSE/PROFESSOR EBTT/INFORMAÇÃO E COMUNICAÇÃO I/2025) Os comandos SQL INSERT, DELETE e UPDATE são utilizados para manipular dados em um banco de dados. Com relação ao uso desses comandos, é INCORRETO afirmar que o comando:

- a) INSERT permite adicionar registros a uma tabela, independentemente das restrições de integridade impostas pelo banco de dados.
- b) INSERT pode ser usado para adicionar múltiplos registros a uma tabela em um único comando.
- c) DELETE, quando utilizado sem a cláusula WHERE, remove todos os registros de uma tabela, mas a definição da tabela permanece no banco de dados.
- d) UPDATE permite modificar os valores dos atributos de vários registros em uma tabela, e a cláusula SET define os novos valores dos atributos.

006. (FUNCAB/CÂMARA DE VITÓRIA-ES/TÉCNICO EM TECNOLOGIA DA INFORMAÇÃO/2014) Na linguagem SQL, o comando `DELETE FROM nome_tabela`, implementado com a condição omitida:

- a) apaga todos os registros da primeira linha da tabela.
- b) altera a tabela, removendo apenas alguns registros da primeira linha.
- c) apaga completamente a tabela, inclusive sua estrutura.
- d) remove as permissões para acesso à tabela de todos os usuários.
- e) apaga todos os registros da tabela, porém mantém sua estrutura.

007. (INSTITUTO CONSULPLAN/HEMOBRÁS/ANALISTA ADMINISTRATIVO DE ASSUNTOS CORPORATIVOS/TECNOLOGIA DA INFORMAÇÃO/2025) Uma instituição pública que fornece medicamentos de biotecnologia para o Sistema Único de Saúde (SUS) está implementando um novo sistema de gestão para rastrear a produção e a distribuição de insumos. Durante a fase de desenvolvimento do banco de dados, um analista de TI precisa executar diferentes comandos SQL para estruturar e manipular os dados armazenados. Dentre os comandos SQL apresentados, qual pertence à categoria DML (Data Manipulation Language), sendo utilizado para modificar registros existentes em uma tabela?

- a) `DROP TABLE medicamento`
- b) `ALTER TABLE medicamento ADD COLUMN fabricante VARCHAR(100)`
- c) `UPDATE medicamento SET data_validade = '2026-05-30' WHERE id = 10`
- d) `CREATE TABLE medicamento (id INT PRIMARY KEY, nome VARCHAR(100), data_validade DATE)`

008. (INSTITUTO ACCESS/UFAC/TÉCNICO EM TECNOLOGIA DA INFORMAÇÃO/2025) Em SQL, operações se referem às diversas ações que podem ser realizadas em um banco de dados para manipular dados. Julgue as seguintes operações SQL:

I – `SELECT` é usado para consultar dados em uma tabela

II – `INSERT` adiciona novos registros em uma tabela

III – `DELETE` altera dados já existentes em uma tabela

IV – `UPDATE` modifica valores em registros existentes

Qual alternativa está correta?

- a) I, III e IV apenas
- b) II, III e IV apenas
- c) III e IV apenas
- d) I e II apenas
- e) I, II e IV apenas

009. (FGV/CÂMARA MUNICIPAL DE FORTALEZA/ANALISTA LEGISLATIVO/2024) Em um banco de dados relacional, considere a tabela a seguir, que possui informações sobre diferentes tipos de produtos, incluindo eletrônicos, roupas, eletrodomésticos, entre outros:

Produto (ID, Nome, Tipo, Preço, Fabricante)

Assinale a alternativa que corresponde à consulta que retornará o nome e o preço dos produtos que possuem a palavra “Smart” em seu tipo, somente do fabricante “Banana Inc.” e preço abaixo de R\$2000,00.

- a) `SELECT Nome
FROM Produto
WHERE Fabricante = 'Banana Inc.' AND Nome = 'Smart'
AND Preço < 2000;`
- b) `SELECT Nome, Preço FROM Produto WHERE Tipo = 'Smart'
AND Fabricante = 'Banana Inc.' AND Preço < 2000;`
- c) `SELECT Nome, Preço FROM Produto
WHERE Tipo LIKE '%Smart%' AND Fabricante = 'Banana Inc.' AND Preço < 2000;`
- d) `SELECT Fabricante, Preço FROM Produto
WHERE Fabricante = 'Banana Inc.' AND Tipo = 'Smart'
AND Preço < 2000;`
- e) `SELECT Tipo, Preço
FROM Produto
WHERE Tipo LIKE '%Smart%'
AND Fabricante = 'Banana Inc.' AND Preço > 2000;`

010. (CESPE/CEBRASPE/SUSEP/ANALISTA TÉCNICO/ÁREA TECNOLOGIA DA INFORMAÇÃO E CIÊNCIA DE DADOS/2025) O comando UPDATE pode ser utilizado para alterar a definição de colunas em uma tabela, como o tipo de dado de uma coluna data_nascimento, por exemplo, substituindo VARCHAR por DATE.

011. (FGV/TJ-MS/TÉCNICO DE NÍVEL SUPERIOR/2024) João está escrevendo uma consulta que envolve várias tabelas e precisa garantir que todas as suas linhas sejam incluídas no resultado, mesmo que não haja correspondências entre elas.

Para tanto, João deverá utilizar o seguinte operador de junção:

- a) LEFT JOIN
- b) INNER JOIN
- c) RIGHT JOIN
- d) CROSS JOIN
- e) FULL OUTER JOIN

012. (CESPE/CEBRASPE/DATAPREV/ANALISTA DE TECNOLOGIA DA INFORMAÇÃO/2023) Com relação a SQL, julgue o item a seguir.

O comando UNION é utilizado para combinar as linhas de duas tabelas, mesmo que as colunas dessas tabelas sejam de tipos e tamanhos diferentes.

013. (CESPE/CEBRASPE/DATAPREV/ANALISTA DE PROCESSAMENTO/2023) Em relação às linguagens de banco de dados SQL, DDL e DML, julgue o item a seguir.

Somente são possíveis os seguintes quatro tipos de JOIN em SQL, segundo o padrão ANSI: INNER JOIN, LEFT JOIN, RIGHT JOIN e CROSS JOIN.

014. (CESPE/CEBRASPE/TELEBRAS/ESPECIALISTA EM GESTÃO DE TELECOMUNICAÇÕES/2022) Julgue o seguinte item, pertinentes a bancos de dados.

Conforme os conceitos de SQL (ANSI), em uma expressão SQL o produto cartesiano resulta que algumas linhas da primeira tabela são unidas a todas as linhas da segunda tabela.

015. (FUNDATEC/CAU-RS/ANALISTA SUPERIOR/2023) Assinale a alternativa que corresponde ao resultado esperado pela execução do comando SQL abaixo:

```
SELECT *  
INTO BKP_VENDAS  
FROM VENDAS;
```

a) Será criada uma nova tabela, nomeada como BKP_VENDAS, com todos os dados da tabela VENDAS

b) Serão atualizados apenas os registros da tabela BKP_VENDAS que tiverem chave correspondente na tabela VENDAS

c) A tabela VENDAS será renomeada para BKP_VENDAS

d) Será criada uma nova tabela vazia, nomeada como BKP_VENDAS

e) Serão copiados os dados da tabela BKP_VENDAS para a tabela VENDAS

016. (INSTITUTO QUADRIX/CRM-MS/ANALISTA DE INFORMÁTICA/2025) Para excluir, por exemplo, o médico de id_medico = 5 da tabela Medicos, garantindo que não existam atendimentos vinculados a ele, pode-se usar o seguinte comando:

```
SQL  
DELETE FROM Medicos WHERE id_medico = 5 AND NOT EXISTS (SELECT 1 FROM  
Atendimentos WHERE id_medico = 5);
```

017. (INÉDITA/2025) Julgue o item a seguir como CERTO ou ERRADO:

Assinale a opção em que é apresentada a palavra-chave, em SQL, que deve ser incluída em uma instrução de SELECT para evitar a apresentação de resultados duplicados da tabela."

- a) UNIQUE
- b) NOTDUPLICATE
- c) NOTALL
- d) DISTINCT
- e) ONLY

018. (IADES/CRMV-PI/ANALISTA/ÁREA SUPORTE E GESTÃO DA TECNOLOGIA DA INFORMAÇÃO/2025) Considere um banco de dados relacional com uma tabela chamada “Funcionarios” que possui os campos: “id”, “nome”, “departamento” e “salario”. Para aumentar o salário de todos os funcionários do departamento “Vendas” em 10%, qual comando SQL deve ser utilizado?

- a) `MODIFY Funcionarios SET salario = salario + 10 WHERE departamento = 'Vendas'`
- b) `ALTER TABLE Funcionarios SET salario = salario * 1.1 WHERE departamento = 'Vendas'`
- c) `UPDATE Funcionarios SET salario = salario * 1.1 WHERE departamento = 'Vendas'`
- d) `INSERT INTO Funcionarios (salario) VALUES (salario * 1.1) WHERE departamento = 'Vendas'`
- e) `SELECT salario * 1.1 FROM Funcionarios WHERE departamento = 'Vendas'`

019. (VUNESP/TJ-RS/ANALISTA DE TECNOLOGIA DA INFORMAÇÃO/ÁREA ANÁLISE DE SISTEMAS/2023) Considere a seguinte tabela de um banco de dados relacional:

Aluno (ID, Nome, Curso)

O comando SQL para obter o nome e o curso dos alunos, com a condição de que o curso se inicie com a palavra Física é:

- a) `SELECT Nome, Curso`
`FROM Aluno`
`WHERE Curso == 'Física%';`
- b) `SELECT Nome, Curso`
`FROM Aluno`
`WHERE Curso.'Física%';`
- c) `SELECT Nome, Curso`
`FROM Aluno`
`WHERE Curso LIKE 'Física%';`
- d) `SELECT Nome, Curso`
`FROM Aluno`
`WHERE Curso('Física%');`

```
e) SELECT Nome, Curso  
FROM Aluno  
WHERE Curso = 'Física%';
```

020. (INSTITUTO QUADRIX/CONSELHO REGIONAL DE ENFERMAGEM-PR/ANALISTA DE TECNOLOGIA DA INFORMAÇÃO/2024) Caso se deseje excluir da base de dados apenas o enfermeiro com ID 121, é suficiente executar o comando SQL a seguir:

```
SQL  
DELETE * FROM Enfermeiro WHERE ID = 121;
```

021. (FGV/TJ-SE/ANALISTA JUDICIÁRIO/2023) Considere uma tabela relacional TAB, com colunas A e B. A coluna A constitui a chave primária de TAB. A instância de TAB contém 100 linhas, e em todas as linhas o valor da coluna B é 10. Nesse contexto, analise o comando SQL a seguir.

```
select * from TAB t where not exists  
(select * from TAB tt  
where t.B = tt.B and t.A > tt.A)
```

Além da linha de títulos, o número de linhas produzidas pelo comando acima é:

- a) 0;
- b) 1;
- c) 98;
- d) 99;
- e) 100.

022. (FGV/TCE-PA/AUDITOR DE CONTROLE EXTERNO/ÁREA ADMINISTRATIVA/2024) Em um processo, o funcionário responsável foi substituído por outro funcionário da mesma organização. Tal substituição deve ser registrada no banco de dados do departamento de pessoal. A cláusula SQL mais indicada para implementar essa alteração, mantendo a integridade do banco, é denominada:

- a) ALTER
- b) INSERT
- c) JOIN
- d) UPDATE
- e) WITH

023. (CESPE/CEBRASPE/SEPLAN-RR/ANALISTA DE PLANEJAMENTO E ORÇAMENTO/2023) Com pertinência à linguagem SQL, julgue o item abaixo. Considere-se o seguinte script SQL.


```
select report_code, year, month, day, wind_speed, case  
when wind_speed >= 40 then 'HIGH'  
when wind_speed >= 30 then 'MODERATE' else 'LOW'  
end as wind_severity from station_data
```

O resultado da execução do script resultará em erro, pois, caso haja, na tabela `station_data`, algum registro no campo `wind_speed` com valor superior a 40, não será possível prever se o valor da variável `wind_severity` será igual a 'HIGH'.

024. (CESPE/CEBRASPE/FINEP/ANALISTA/2024) Assinale a opção que apresenta a cláusula SQL que permite extrair dados em determinado intervalo.

- a) find in
- b) like
- c) alias
- d) between
- e) ranges

025. (FCC/COPERGÁS/ANALISTA/2023) Em uma tabela chamada `user` de um banco de dados aberto e em condições ideais, para selecionar todos os registros que possuem nomes (campo `nome`) iniciados com a letra E e terminados com a letra I utiliza-se a instrução SQL `SELECT * FROM user`

- a) `LIKE = 'E*I'`;
- b) `WHERE nome = 'E*I'`;
- c) `LIKE nome CONTAINS 'E*I'`;
- d) `WHERE nome LIKE 'E*I'`;
- e) `WHERE nome LIKE 'E%I'`;

026. (FCC/PGE AM/TÉCNICO EM GESTÃO PROCURATORIAL/2022) Para excluir da tabela `Advogado_Processo` todas as linhas nas quais consta o valor 4378 no campo `oabAdvogado`, utiliza-se a instrução

- a) `DELETE * FROM Advogado_Processo AS adp WHERE adp.oabAdvogado=4378;`
- b) `DELETE FROM Advogado_Processo (SELECT oabAdvogado=4378);`
- c) `DELETE * FROM Advogado_Processo WHERE oabAdvogado=4378;`
- d) `DELETE FROM Advogado_Processo WHERE oabAdvogado=4378;`
- e) `DELETE FROM Advogado_Processo (SELECT * FROM Advogado WHERE oabAdvogado=4378);`

027. (CESPE/CEBRASPE/POLC-AL/PERITO CRIMINAL/2023) Com relação aos componentes de um computador, aos barramentos de E/S, à aritmética computacional e à linguagem SQL, julgue o próximo item.

Em SQL, para que não haja erro de construção (sintaxe), as cláusulas GROUP BY e HAVING, quando usadas, devem ser definidas sempre antes da cláusula WHERE.

028. (IFSUL/IFSUL/PROFESSOR/ÁREA INFORMAÇÃO E COMUNICAÇÃO/2025) Os comandos SQL INSERT, DELETE e UPDATE são utilizados para manipular dados em um banco de dados. Com relação ao uso desses comandos, é INCORRETO afirmar que o comando:

- a) INSERT permite adicionar registros a uma tabela, independentemente das restrições de integridade impostas pelo banco de dados
- b) INSERT pode ser usado para adicionar múltiplos registros a uma tabela em um único comando
- c) DELETE, quando utilizado sem a cláusula WHERE, remove todos os registros de uma tabela, mas a definição da tabela permanece no banco de dados
- d) UPDATE permite modificar os valores dos atributos de vários registros em uma tabela, e a cláusula SET define os novos valores dos atributos

029. (FGV/TJ-MS/TÉCNICO DE NÍVEL SUPERIOR/ÁREA WEB DESIGNER/2024) Observe o script SQL a seguir:

SQL

```
SELECT COUNT(*) AS [Quantidade], Tipo_Processo FROM Processo GROUP BY Tipo_Processo;
```

O resultado da execução desse script é:

- a) a lista dos registros da tabela quantidade
- b) a quantidade de processos por tipo
- c) a contagem dos registros da tabela de tipos de processos
- d) o agrupamento de processos que realizam contagem
- e) a contagem dos processos relacionados à quantidade de valores

030. (FGV/DPE-RS/TÉCNICO/2023) A programadora Fabiana elaborou um relatório com o nome do autor dos processos utilizando o seguinte comando SQL:

```
SELECT Autor FROM tab_processo
```

Ao ver o relatório gerado, o chefe da Fabiana solicitou um relatório contendo apenas os autores que possuem o primeiro nome Elizabeth. Contudo, Fabiana observou que havia diferentes grafias, como: Elisabeth, Elizabete etc.

No MySQL, para garantir que qualquer Autor cujo nome comece pelas letras “eli” sejam recuperados, Fabiana deve complementar o comando SQL com a cláusula WHERE Autor LIKE:

- a) 'eli%'
- b) '%eli%'
- c) 'eli_'
- d) '_eli'
- e) '_eli%'

031. (FGV/TJ-MS/TÉCNICO DE NÍVEL SUPERIOR/ÁREA WEB DESIGNER/2024) SQL (Structured Query Language) é uma linguagem usada para interagir com bancos de dados relacionais. Relativamente a SQL, analise as afirmativas a seguir e assinale (V) para a verdadeira e (F) para a falsa:

- () A cláusula WHERE só é utilizada associada ao comando SELECT
- () O comando CREATE TABLE cria uma nova tabela no banco de dados e o comando INSERT INTO permite popular essa tabela por meio da inserção de registros
- () O comando INNER JOIN é utilizado para unir os resultados de duas ou mais consultas SELECT e o seu resultado é um conjunto de linhas provenientes das tabelas envolvidas
- () Os comandos ALTER TABLE e UPDATE são comandos usados para atualizar registros em tabelas de um banco de dados
- () SQL suporta o conceito de chaves primárias e chaves estrangeiras para manter a integridade dos dados e as relações entre as tabelas

As afirmativas são, respectivamente:

- a) F – F – F – V – F
- b) F – V – F – F – V
- c) V – V – V – F – V
- d) V – F – V – V – V
- e) F – F – V – F – F

categoria	modelo	valor
1	A	40
1	A	18
1	B	76
2	B	97

032. (CESPE/CEBRASPE/CNPQ/ANALISTA EM CIÊNCIA E TECNOLOGIA I/2024) Tendo como referência a tabela precedente, de nome dados, julgue o item subsequente, com relação à SQL. As expressões SQL a seguir, após serem executadas, gerarão o mesmo resultado.
insert into dados values (2,'C',40);

```
insert into dados (categoria, modelo, valor)
values (2, 'C', 40);
```

033. (FGV/TJ-MS/TÉCNICO DE NÍVEL SUPERIOR/ÁREA WEB DESIGNER/2023) A linguagem SQL permite diversas operações sob um conjunto de dados de uma ou mais tabelas, as necessidades do projeto de software ditando quais cláusulas deverão ser utilizadas, sejam para recuperação, atualização, inserção ou remoção de algum dado. Ao lidarmos com recuperação, os dados que iremos recuperar podem ser filtrados, agrupados etc. Assinale a alternativa que apresenta qual a cláusula SQL responsável para filtrar registros em uma consulta de uma tabela:

- a) GROUP BY
- b) SELECT
- c) FILTER
- d) WHERE

034. (FGV/TJ-MS/TÉCNICO DE NÍVEL SUPERIOR/ÁREA WEB DESIGNER/2016) Considere um banco de dados criado no MySQL 5.7 com os seguintes comandos:

SQL

```
CREATE DATABASE controle; USE controle; CREATE TABLE PAGAMENTOS ( ID INT
PRIMARY KEY AUTO_INCREMENT, NOME VARCHAR(50), VALOR DOUBLE ); INSERT INTO
PAGAMENTOS(NOME,VALOR) VALUES ( 'Mary',1500); INSERT INTO PAGAMENTOS(NOME,VALOR)
VALUES ( 'Albert',300); INSERT INTO PAGAMENTOS(NOME,VALOR) VALUES ( 'Mary',450);
INSERT INTO PAGAMENTOS(NOME,VALOR) VALUES ( 'Lucas',3450); INSERT INTO
PAGAMENTOS(NOME,VALOR) VALUES ( 'Mary',180); INSERT INTO PAGAMENTOS(NOME,VALOR)
VALUES ( 'Mary',520); INSERT INTO PAGAMENTOS(NOME,VALOR) VALUES ( 'Albert',840);
INSERT INTO PAGAMENTOS(NOME,VALOR) VALUES ( 'Lucas',110); INSERT INTO
PAGAMENTOS(NOME,VALOR) VALUES ( 'Mary',3300);
```

Logo após, a seguinte consulta foi executada:

SQL

```
SELECT NOME, SUM(VALOR) AS TOTAL FROM PAGAMENTOS GROUP BY NOME HAVING TOTAL
> ( SELECT SUM(VALOR) / COUNT(DISTINCT NOME) FROM PAGAMENTOS );
```

Assinale a alternativa que contém o número de linhas retornadas pela consulta anterior.

- a) 0
- b) 1
- c) 2
- d) 3
- e) 9

035. (DIRENS AERONÁUTICA/SARGENTO DA AERONÁUTICA/ESPECIALIDADE INFORMÁTICA/2024) SQL (Structured Query Language) é uma linguagem usada para interagir com bancos de dados relacionais. Relativamente a SQL, analise as afirmativas a seguir e assinale a alternativa correta:

- I – O comando SELECT é utilizado para recuperar dados de uma ou mais tabelas
 - II – O comando INSERT é utilizado para adicionar novas linhas a uma tabela
 - III – O comando UPDATE é utilizado para modificar dados existentes em uma tabela
 - IV – O comando DELETE é utilizado para remover linhas de uma tabela
- a) Apenas as afirmativas I e II estão corretas
 - b) Apenas as afirmativas I, II e III estão corretas
 - c) Apenas as afirmativas I, III e IV estão corretas
 - d) Apenas as afirmativas II, III e IV estão corretas
 - e) Todas as afirmativas estão corretas

036. (FGV/TJ-MS/TÉCNICO DE NÍVEL SUPERIOR/2024) Observe script SQL a seguir.

```
SELECT COUNT(*) AS [Quantidade], Tipo_Processo  
FROM Processo  
GROUP BY Tipo_Processo;
```

O resultado da execução desse script é:

- a) a lista dos registros da tabela quantidade;
- b) a quantidade de processos por tipo;
- c) a contagem dos registros da tabela de tipos de processos;
- d) o agrupamento de processos que realizam contagem;
- e) a contagem dos processos relacionados à quantidade de valores.

037. (DIRENSAERONÁUTICA/SARGENTO DAAERONÁUTICA/ESPECIALIDADEINFORMÁTICA/2024) Em SQL, qual comando é utilizado para atualizar registros existentes em uma tabela?

- a) INSERT
- b) DELETE
- c) SELECT
- d) UPDATE
- e) CREATE

038. (DIRENSAERONÁUTICA/SARGENTO DAAERONÁUTICA/ESPECIALIDADEINFORMÁTICA/2024) Qual comando SQL DML é utilizado para remover uma ou mais linhas de uma tabela?

- a) TRUNCATE
- b) DROP

- c) DELETE
- d) REMOVE
- e) ERASE

039. (CESPE/CEBRASPE/PETROBRAS/PROFISSIONAL PETROBRAS DE NÍVEL SUPERIOR/2022)

Julgue o item abaixo, a respeito da linguagem SQL.

O comando delete alunos permite apagar uma tabela de nome alunos.

040. (CESPE/CEBRASPE/CNPQ/ANALISTA EM CIÊNCIA E TECNOLOGIA I/2024) Acerca de técnicas de modelagem de BI (business intelligence), de big data e de linguagem de manipulação de dados (DML), julgue o item que se segue.

A DML não procedimental exige que um usuário especifique quais dados são necessários sem especificar como obtê-los.

041. (FCC/TRT 12ª REGIÃO/ANALISTA JUDICIÁRIO/2023) Considere uma tabela denominada Cidadao e as colunas Nome_Cidadao e Valor_Recebido.

Para obter a média dos valores maiores que 200 recebidos pelos cidadãos, um Analista deve utilizar o comando

- a) `SELECT AVG (Valor_Recebido > 200) FROM Cidadao;`
- b) `SELECT AVG (Valor_Recebido) FROM Cidadao WHERE Valor_Recebido > 200;`
- c) `SELECT AVG (Valor_Recebido) > 200 FROM Cidadao;`
- d) `SELECT FROM Cidadao AVG (Valor_Recebido) > 200;`
- e) `SELECT Nome Cidadao and AVG (Valor_Recebido) FROM Cidadao WHERE Valor_Recebido > 200;`

042. (CESPE/CEBRASPE/MPU/ANALISTA DE INFORMÁTICA/2010) Julgue o item seguinte acerca de DML (data manipulation language) e DDL (data definition language).

As DML devem ser utilizadas incorporadas a outras linguagens, chamadas de linguagens hospedeiras, uma vez que sentenças escritas em DML devem estar embutidas em código escrito em uma linguagem hospedeira para que possam ser executadas.

043. (FCC/SEGEP-MA/ANALISTA EXECUTIVO/PROGRAMADOR DE SISTEMAS/2018) Um Programador de Sistemas recuperou todos os dados dos países Brasil, Argentina e Peru gravados no campo Pais da tabela Cliente, abaixo especificada:

Tabela Cliente:

IdCliente NomeCliente Endereco CEP

Cidade SiglaUF Pais

A sintaxe SQL correta que ele usou para realizar essa atividade foi `SELECT * FROM Cliente`

- a) WHERE Pais IN ('Argentina', 'Peru', 'Brasil');
- b) WHEN Pais = ('Argentina', 'Peru', 'Brasil');
- c) WHERE Pais = ('Argentina', 'Peru', 'Brasil');
- d) WHEN Pais IN ('Argentina', 'Peru', 'Brasil');
- e) WHERE Pais BETWEEN ('Argentina', 'Peru', 'Brasil');

044. (INÉDITA/2025) Considere a seguinte consulta SQL:

SQL

```
SELECT DISTINCT Nome, Pais FROM Clientes WHERE Cidade LIKE 'M%' ORDER BY Pais DESC;
```

Sobre esta consulta, é CORRETO afirmar que:

- a) Retorna nomes e países únicos de clientes de cidades que terminam com 'M'
- b) Retorna nomes e países de clientes de cidades que começam com 'M', eliminando duplicatas e ordenando por país decrescente
- c) A cláusula DISTINCT não funcionará corretamente pois há duas colunas selecionadas
- d) O ORDER BY deve vir antes do WHERE para funcionar corretamente
- e) O operador LIKE está sendo usado incorretamente

045. (FUNDATEC/CAU-RS/ANALISTA SUPERIOR/2023) Na linguagem SQL, um alias para uma coluna ou tabela pode ser criado utilizando a palavra-chave:

- a) HOW
- b) AS
- c) FOR
- d) ON
- e) LIKE

046. (CESPE/CEBRASPE/AGER-MT/ANALISTA REGULADOR/2023) Em linguagem de manipulação de dados DML, o operador SQL BETWEEN serve para

- a) delimitar o valor de uma coluna na cláusula WHERE.
- b) delimitar as colunas a serem apresentadas na cláusula WHERE.
- c) delimitar os limites de um campo para a cláusula INSERT.
- d) restringir a quantidade de linhas a serem recuperadas na cláusula UPDATE.
- e) restringir a quantidade de campos na cláusula DELETE.

047. (CESPE/CEBRASPE/TCE-SC/ANALISTA DE CONTAS PÚBLICAS/2022) Acerca do conceito de view, da modelagem dimensional, do modelo de referência CRISP- DM e da linguagem SQL, julgue o item subsequente.

O comando a seguir tem a finalidade de mostrar o nome e o email de todos os procuradores que recebem salários acima de R\$ 30.000,00.

```
SELECT nome, email  
FROM procurador  
WHEN salario >= 30.000,00;
```

048. (INÉDITA/2025) Julgue o item a seguir como CERTO ou ERRADO:

Em uma consulta SQL, a cláusula FROM corresponde à seleção do predicado que envolve atributos da relação determinada pela cláusula SELECT.

049. (CESPE/CEBRASPE/CTI/TECNOLOGISTA JÚNIOR/2024) Considerando as linguagens e os fundamentos de bancos de dados relacionais, julgue o item subsequente.

No DML (data manipulation language), a instrução TRUNCATE elimina todas as linhas de uma tabela simultaneamente, enquanto a instrução DELETE oferece a possibilidade de excluir dados específicos ou todos os dados.

050. (VUNESP/TJM-SP/ANALISTA DE BANCO DE DADOS JUDICIÁRIO/2025) Considere a linguagem SQL no que se refere a comandos para manipulação de dados (DML). Suponha uma tabela de um banco de dados relacional, denominada Tab1, que contenha os campos Cod e Nome. Assinale a alternativa que indica o comando para atualizar o campo Cod para o valor 25, apenas para valores do campo Nome diferentes da string Teste:

- a) UPDATE Tab1 Cod = 25 HAVING Nome <> 'Teste'
- b) UPDATE Tab1 MAKE Cod = 25 FOR Nome <> 'Teste'
- c) UPDATE Tab1 SET Cod = 25 WHERE Nome <> 'Teste'
- d) UPDATE Tab1 SET Cod = 25 IN CASE Nome <> 'Teste'
- e) UPDATE Tab1 USE Cod = 25 AT Nome <> 'Teste'

GABARITO

1. b	18. c	35. e
2. b	19. c	36. b
3. d	20. E	37. d
4. d	21. b	38. c
5. a	22. d	39. E
6. e	23. E	40. C
7. c	24. d	41. b
8. e	25. e	42. E
9. c	26. d	43. a
10. E	27. E	44. b
11. e	28. a	45. b
12. E	29. b	46. a
13. E	30. a	47. E
14. E	31. b	48. E
15. a	32. C	49. E
16. C	33. d	50. c
17. d	34. c	

GABARITO COMENTADO

001. (FGV/INPE/TECNOLOGISTA JÚNIOR/ÁREA OPERAÇÃO DE SISTEMAS ESPACIAIS/2024) SQL (Structured Query Language) é uma linguagem usada para interagir com bancos de dados relacionais. Relativamente a SQL, analise as afirmativas a seguir e assinale (V) para a verdadeira e (F) para a falsa.

- () A cláusula WHERE só é utilizada associada ao comando SELECT.
- () O comando CREATE TABLE cria uma nova tabela no banco de dados e o comando INSERT INTO permite popular essa tabela por meio da inserção de registros.
- () O comando INNER JOIN é utilizado para unir os resultados de duas ou mais consultas SELECT e o seu resultado é um conjunto de linhas provenientes das tabelas envolvidas.
- () Os comandos ALTER TABLE e UPDATE são comandos usados para atualizar registros em tabelas de um banco de dados.
- () SQL suporta o conceito de chaves primárias e chaves estrangeiras para manter a integridade dos dados e as relações entre as tabelas.

As afirmativas são, respectivamente:

- a) F – F – F – V – F
- b) F – V – F – F – V
- c) V – V – V – F – V
- d) V – F – V – V – V
- e) F – F – V – F – F



(F) A cláusula WHERE só é utilizada associada ao comando SELECT.

A cláusula WHERE é usada para filtrar registros em uma consulta. Embora seja comumente usada com o comando SELECT, ela também pode ser utilizada com outros comandos, como UPDATE e DELETE.

(V) O comando CREATE TABLE cria uma nova tabela no banco de dados e o comando INSERT INTO permite popular essa tabela por meio da inserção de registros.

O comando CREATE TABLE é utilizado para criar uma nova tabela no banco de dados. Após criar a tabela, o comando INSERT INTO é empregado para inserir registros nela.

(F) O comando INNER JOIN é utilizado para unir os resultados de duas ou mais consultas SELECT e o seu resultado é um conjunto de linhas provenientes das tabelas envolvidas.

O comando INNER JOIN é utilizado para combinar registros de duas ou mais tabelas com base em uma condição de junção. Porém, ele não une resultados de várias consultas SELECT, mas sim registros das tabelas envolvidas diretamente na consulta.

(F) Os comandos ALTER TABLE e UPDATE são comandos usados para atualizar registros em tabelas de um banco de dados.

O comando UPDATE é usado para atualizar registros em uma tabela. Já o comando ALTER TABLE é utilizado para modificar a estrutura de uma tabela, como adicionar ou remover colunas, mas não é usado para atualizar os registros da tabela.

(V) SQL suporta o conceito de chaves primárias e chaves estrangeiras para manter a integridade dos dados e as relações entre as tabelas.

SQL realmente suporta o uso de chaves primárias (primary keys) e chaves estrangeiras (foreign keys) para garantir a integridade referencial e manter as relações entre tabelas.

Letra b.

002. (FGV/TJ-MS/TÉCNICO DE NÍVEL SUPERIOR/ANALISTA DE SISTEMAS COMPUTACIONAIS/WEB DESIGNER/2024) Observe o script SQL a seguir:

SQL

```
SELECT COUNT(*) AS [Quantidade], Tipo_Processo FROM Processo GROUP BY
Tipo_Processo;
```

O resultado da execução desse script é:

- a) a lista dos registros da tabela quantidade
- b) a quantidade de processos por tipo
- c) a contagem dos registros da tabela de tipos de processos
- d) o agrupamento de processos que realizam contagem
- e) a contagem dos processos relacionados à quantidade de valores



Analisando o script SQL fornecido:

SQL

```
SELECT COUNT(*) AS [Quantidade], Tipo_Processo
FROM Processo
GROUP BY Tipo_Processo;
```

1. FROM Processo: indica que a consulta está sendo feita na tabela chamada Processo.
2. GROUP BY Tipo_Processo: determina que os resultados serão agrupados com base nos valores únicos encontrados na coluna Tipo_Processo.
3. SELECT COUNT(*): para cada grupo formado, a função de agregação COUNT(*) conta o número de linhas (registros) dentro daquele grupo.
4. AS [Quantidade]: dá um nome de coluna ao resultado da contagem, neste caso, Quantidade.
5. SELECT..., Tipo_Processo: retorna, para cada grupo, o valor do Tipo_Processo que foi usado para agrupar e a contagem correspondente.

O resultado final será uma lista onde cada linha mostra um valor distinto de Tipo_Processo e, ao lado, o total de registros (processos) que pertencem àquele tipo. Portanto, o script conta quantos processos existem para cada tipo de processo diferente.

- a) Errada. A coluna [Quantidade] é um resultado da contagem, não uma tabela.)
- b) Certa. O script agrupa os processos por Tipo_Processo e conta quantos existem em cada grupo.)
- c) Errada. Conta os registros da tabela Processo agrupados por tipo, não “da tabela de tipos de processos” (que nem é mencionada).
- d) Errada. A contagem é o resultado do agrupamento.
- e) Errada. A contagem é de processos por tipo.

Letra b.

003. (IBFC/SAEB/TÉCNICO EM TECNOLOGIA DA INFORMAÇÃO/ÁREA DESENVOLVIMENTO/2023)

A linguagem SQL permite diversas operações sob um conjunto de dados de uma ou mais tabelas, as necessidades do projeto de software ditando quais cláusulas deverão ser utilizadas, sejam para recuperação, atualização, inserção ou remoção de algum dado. Ao lidarmos com recuperação, os dados que iremos recuperar podem ser filtrados, agrupados etc. Assinale a alternativa que apresenta qual a cláusula SQL responsável para filtrar registros em uma consulta de uma tabela:

- a) GROUP BY
- b) SELECT
- c) FILTER
- d) WHERE



- a) Errada. É usada para agrupar linhas que têm os mesmos valores em colunas especificadas. É usada em conjunto com funções de agregação (como COUNT, SUM, AVG).
- b) Errada. É a palavra-chave fundamental usada para iniciar uma consulta de recuperação de dados e para especificar as colunas que serão retornadas.
- c) Errada. Não é uma cláusula padrão do SQL para filtragem de registros em uma tabela. A cláusula WHERE é usada para esse fim. (Em alguns dialetos SQL, como PostgreSQL, FILTER é usada em conjunto com funções de agregação, mas não para filtrar registros de uma tabela inteira).
- d) Certa. É a cláusula correta, usada para filtrar os registros que satisfazem uma condição específica antes que o resultado seja retornado ou antes que o agrupamento (GROUP BY) seja aplicado.

Letra d.

004. (FCC/TJ-CE/ANALISTA JUDICIÁRIO/2022) Uma tabela chamada cliente possui os campos abaixo

id – int (Primary Key) nome – varchar(70) cidade – varchar(40) estado – varchar(40)

Em condições ideais, para exibir os dados de todos os clientes, cujo nome da cidade não seja igual ao nome do estado, utiliza-se a instrução SQL:

SELECT * FROM cliente WHERE

- a) cidade != estado;
- b) cidade!EQUALS estado;
- c) cidade UNLIKE(estado);
- d) cidade <> estado;
- e) cidade UNLIKE estado;



a) Errada. O operador de desigualdade padrão em SQL não é !=. Embora alguns SGBDs (como MySQL e PostgreSQL) aceitem o operador != como sinônimo, o padrão SQL e o operador mais universalmente aceito (inclusive no SQL Server e Oracle) é o <>.

b) Errada. Não é uma sintaxe padrão em SQL.

c) Errada. A sintaxe correta para o operador de negação de padrão é NOT LIKE, e UNLIKE não é padrão.

d) Certa. O operador < > é o operador de desigualdade (diferente de / *not equal to*) padrão ANSI SQL, sendo aceito por praticamente todos os SGBDs e considerado a forma mais correta e universal de expressar essa condição.

e) Errada. A sintaxe correta para o operador de negação de padrão é NOT LIKE.

Letra d.

005. (IF SUL RIO-GRANDENSE/IF SUL RIO-GRANDENSE/PROFESSOR EBTT/INFORMAÇÃO E COMUNICAÇÃO I/2025) Os comandos SQL INSERT, DELETE e UPDATE são utilizados para manipular dados em um banco de dados. Com relação ao uso desses comandos, é INCORRETO afirmar que o comando:

- a) INSERT permite adicionar registros a uma tabela, independentemente das restrições de integridade impostas pelo banco de dados.
- b) INSERT pode ser usado para adicionar múltiplos registros a uma tabela em um único comando.
- c) DELETE, quando utilizado sem a cláusula WHERE, remove todos os registros de uma tabela, mas a definição da tabela permanece no banco de dados.
- d) UPDATE permite modificar os valores dos atributos de vários registros em uma tabela, e a cláusula SET define os novos valores dos atributos.



Analisando as afirmações sobre os comandos SQL INSERT, DELETE e UPDATE:

- a) Errada. Os comandos INSERT (assim como UPDATE) NÃO permitem adicionar dados que violem as restrições de integridade (como PRIMARY KEY, FOREIGN KEY, NOT NULL ou CHECK) impostas pelo banco de dados. Se o INSERT violar uma restrição, o comando falhará e o registro não será adicionado.
- b) Certa. A sintaxe padrão INSERT INTO tabela (colunas) VALUES (valor1, valor2,...), (valor1, valor2,...); permite a inserção de múltiplos registros em um único comando.
- c) Certa. DELETE FROM Tabela; remove todas as linhas (registros) da tabela, mas mantém a estrutura (definição) da tabela, incluindo colunas, índices e restrições.
- d) Certa. O UPDATE é usado para modificar dados existentes. A cláusula SET é obrigatória para especificar quais colunas terão seus valores alterados, e a cláusula WHERE (opcional) define quais registros serão afetados.

Letra a.

006. (FUNCAB/CÂMARA DE VITÓRIA-ES/TÉCNICO EM TECNOLOGIA DA INFORMAÇÃO/2014) Na linguagem SQL, o comando DELETE FROM nome_tabela, implementado com a condição omitida:

- a) apaga todos os registros da primeira linha da tabela.
- b) altera a tabela, removendo apenas alguns registros da primeira linha.
- c) apaga completamente a tabela, inclusive sua estrutura.
- d) remove as permissões para acesso à tabela de todos os usuários.
- e) apaga todos os registros da tabela, porém mantém sua estrutura.



- a) Errada. Este comando não possui filtro (WHERE), portanto, ele não se restringe apenas à primeira linha. Se a cláusula WHERE for omitida, o comando age sobre todos os registros da tabela.
- b) Errada. O comando DELETE não é usado para “alterar” a estrutura da tabela (isso é função do ALTER TABLE). Além disso, sem o WHERE, ele remove todos os registros, e não “apenas alguns”.
- c) Errada. O comando que apaga a tabela e sua estrutura (incluindo metadados, índices e definições) é o DROP TABLE nome_tabela. O DELETE é um comando DML (Linguagem de Manipulação de Dados) e apenas remove o conteúdo (os dados).
- d) Errada. O comando responsável por gerenciar permissões é o REVOKE (para remover) ou GRANT (para conceder), que pertencem à DCL (Linguagem de Controle de Dados). O DELETE manipula dados.

e) Certa. Ao omitir a cláusula WHERE em um comando DELETE FROM, o SGBD aplica a remoção a todas as linhas da tabela. No entanto, como DELETE é um comando DML, ele não afeta a definição (estrutura ou schema) da tabela.

Letra e.

007. (INSTITUTO CONSULPLAN/HEMOBRÁS/ANALISTA ADMINISTRATIVO DE ASSUNTOS CORPORATIVOS/TECNOLOGIA DA INFORMAÇÃO/2025) Uma instituição pública que fornece medicamentos de biotecnologia para o Sistema Único de Saúde (SUS) está implementando um novo sistema de gestão para rastrear a produção e a distribuição de insumos. Durante a fase de desenvolvimento do banco de dados, um analista de TI precisa executar diferentes comandos SQL para estruturar e manipular os dados armazenados. Dentre os comandos SQL apresentados, qual pertence à categoria DML (Data Manipulation Language), sendo utilizado para modificar registros existentes em uma tabela?

- a) DROP TABLE medicamento
- b) ALTER TABLE medicamento ADD COLUMN fabricante VARCHAR(100)
- c) UPDATE medicamento SET data_validade = '2026-05-30' WHERE id = 10
- d) CREATE TABLE medicamento (id INT PRIMARY KEY, nome VARCHAR(100), data_validade DATE)



- a) Errada. Este comando pertence à DDL (Data Definition Language). Ele é usado para apagar completamente a estrutura (a própria tabela, incluindo todos os seus dados, índices e definições) do banco de dados.
- b) Errada. Este comando também pertence à DDL (Data Definition Language). Ele é usado para modificar a estrutura (esquema ou schema) de uma tabela existente, neste caso, adicionando uma nova coluna (fabricante).
- c) Certa. Este comando pertence à DML (Data Manipulation Language). Ele é especificamente usado para modificar (atualizar) os registros (dados) que já existem na tabela, alterando o valor da coluna data_validade para o registro onde o id é igual a 10.
- d) Errada. Este comando pertence à DDL (Data Definition Language). Ele é usado para criar a estrutura (definição) de uma nova tabela no banco de dados.

Letra c.

008. (INSTITUTO ACCESS/UFAC/TÉCNICO EM TECNOLOGIA DA INFORMAÇÃO/2025) Em SQL, operações se referem às diversas ações que podem ser realizadas em um banco de dados para manipular dados. Julgue as seguintes operações SQL:

I – SELECT é usado para consultar dados em uma tabela

- II – INSERT adiciona novos registros em uma tabela
- III – DELETE altera dados já existentes em uma tabela
- IV – UPDATE modifica valores em registros existentes

Qual alternativa está correta?

- a) I, III e IV apenas
- b) II, III e IV apenas
- c) III e IV apenas
- d) I e II apenas
- e) I, II e IV apenas



I – Certa. O SELECT é o comando DML fundamental para a recuperação (consulta) de dados em um banco de dados.

II – Certa. O INSERT é o comando DML usado para adicionar novas linhas (registros) a uma tabela.

III – Errada. O DELETE é usado para remover (apagar) registros de uma tabela. O comando usado para alterar dados existentes é o UPDATE.

IV – Certa. O UPDATE é o comando DML usado para modificar (atualizar) os valores em linhas (registros) que já existem na tabela.

Letra e.

009. (FGV/CÂMARA MUNICIPAL DE FORTALEZA/ANALISTA LEGISLATIVO/2024) Em um banco de dados relacional, considere a tabela a seguir, que possui informações sobre diferentes tipos de produtos, incluindo eletrônicos, roupas, eletrodomésticos, entre outros:

Produto (ID, Nome, Tipo, Preço, Fabricante)

Assinale a alternativa que corresponde à consulta que retornará o nome e o preço dos produtos que possuem a palavra “Smart” em seu tipo, somente do fabricante “Banana Inc.” e preço abaixo de R\$2000,00.

- a) SELECT Nome
FROM Produto
WHERE Fabricante = 'Banana Inc.' AND Nome = 'Smart'
AND Preço < 2000;
- b) SELECT Nome, Preço FROM Produto WHERE Tipo = 'Smart'
AND Fabricante = 'Banana Inc.' AND Preço < 2000;
- c) SELECT Nome, Preço FROM Produto
WHERE Tipo LIKE '%Smart%' AND Fabricante = 'Banana Inc.' AND Preço < 2000;
- d) SELECT Fabricante, Preço FROM Produto


```
WHERE Fabricante = 'Banana Inc.' AND Tipo = 'Smart'
AND Preço < 2000;
e) SELECT Tipo, Preço
FROM Produto
WHERE Tipo LIKE '%Smart%'
AND Fabricante = 'Banana Inc.' AND Preço > 2000;
```



A questão exige a aplicação de múltiplos filtros e a utilização correta do operador de busca por padrões (LIKE) para atender a todas as condições.

A consulta SQL deve:

- Retornar: nome e preço;
- Filtrar por Tipo: conter a palavra "Smart" em qualquer parte do campo Tipo;
- Filtrar por Fabricante: ser exatamente igual a "Banana Inc.";
- Filtrar por Preço: ser abaixo de R\$ 2000,00 (< 2000).

A alternativa que satisfaz todas essas condições é:

```
SELECT Nome, Preço FROM Produto WHERE Tipo LIKE '%Smart%' AND Fabricante =
'Banana Inc.' AND Preço < 2000;
```

a) Errada. Retorna apenas o Nome (deveria retornar Nome, Preço). Filtra o Nome (deveria ser o Tipo) e usa = (deveria ser LIKE '%Smart%').

b) Errada. O filtro de Tipo usa o operador de igualdade (= 'Smart'), o que só retornaria produtos cujo tipo é exatamente "Smart", e não aqueles que contêm a palavra "Smart" (ex.: "SmartTV").

c) Certa. O SELECT Nome, Preço atende ao requisito de exibição. O LIKE '%Smart%' usa o caractere curinga % antes e depois, garantindo que o filtro encontre "Smart" em qualquer posição do campo Tipo. As demais condições (Fabricante e Preço) estão corretas.

d) Errada. O select retorna Fabricante (deveria ser Nome).

e) Errada. O select retorna Tipo (deveria ser Nome). O filtro de preço está invertido (> 2000 em vez de < 2000).

Letra c.

010. (CESPE/CEBRASPE/SUSEP/ANALISTA TÉCNICO/ÁREA TECNOLOGIA DA INFORMAÇÃO E CIÊNCIA DE DADOS/2025) O comando UPDATE pode ser utilizado para alterar a definição de colunas em uma tabela, como o tipo de dado de uma coluna data_nascimento, por exemplo, substituindo VARCHAR por DATE.



O comando UPDATE pertence à DML (Data Manipulation Language) e é utilizado exclusivamente para modificar os *valores* dos dados (registros) dentro de uma tabela.

Para alterar a definição de colunas em uma tabela, como mudar o tipo de dado (VARCHAR para DATE), o comando correto a ser utilizado é o ALTER TABLE, que pertence à DDL (Data Definition Language).

O comando correto para a alteração seria algo como:

```
ALTER TABLE nome_tabela ALTER COLUMN data_nascimento DATE;
```

Errado.

011. (FGV/TJ-MS/TÉCNICO DE NÍVEL SUPERIOR/2024) João está escrevendo uma consulta que envolve várias tabelas e precisa garantir que todas as suas linhas sejam incluídas no resultado, mesmo que não haja correspondências entre elas.

Para tanto, João deverá utilizar o seguinte operador de junção:

- a) LEFT JOIN
- b) INNER JOIN
- c) RIGHT JOIN
- d) CROSS JOIN
- e) FULL OUTER JOIN



a) Errada. LEFT JOIN retorna todos os registros da tabela da esquerda, e os registros relacionados da tabela da direita. Logo, não garante que todas as linhas serão incluídas no resultado, pois não trará os registros da tabela da direita que não tiverem relação com os registros da tabela da esquerda.

b) Errada. INNER JOIN retorna somente os registros relacionados, logo os registros de ambas as tabelas que não tiverem relação não serão retornados.

c) Errada. RIGHT JOIN retorna todos os registros da tabela da direita, e os registros relacionados da tabela da esquerda. Logo, não garante que todas as linhas serão incluídas no resultado, pois não trará os registros da tabela da esquerda que não tiverem relação com os registros da tabela da direita.

d) Errada. CROSS JOIN retorna o produto cartesiano, ou seja, todas as combinações de linhas das tabelas de entrada. Embora esse comando garanta que todas as linhas sejam retornadas, ele não é o mais adequado para o objetivo da questão, pois não exibirá as relações entre as linhas quando elas existirem, trazendo um simples cruzamento de todas as linhas.

e) Certa. FULL OUTER JOIN retorna todos os registros das tabelas independente de relação, trazendo na mesma linha aqueles que tiverem relação.

Letra e.

012. (CESPE/CEBRASPE/DATAPREV/ANALISTA DE TECNOLOGIA DA INFORMAÇÃO/2023) Com relação a SQL, julgue o item a seguir.

O comando UNION é utilizado para combinar as linhas de duas tabelas, mesmo que as colunas dessas tabelas sejam de tipos e tamanhos diferentes.



Função do UNION: o operador UNION é usado para combinar os conjuntos de resultados de duas ou mais consultas SELECT em um único conjunto de resultados.

Regras de Compatibilidade Estritas: para que o UNION funcione, o padrão SQL (e a maioria dos SGBDs) exige que os conjuntos de resultados das consultas obedeçam a regras estritas de compatibilidade, sendo as duas principais:

- Número de Colunas: o número de colunas selecionadas em cada consulta SELECT deve ser o mesmo;
- Tipos de Dados: as colunas devem ter tipos de dados compatíveis (embora o tamanho do campo possa ser ajustado pelo SGBD, os tipos de dados básicos, como número, texto ou data, devem ser coerentes na mesma posição).

Se as colunas das tabelas a serem combinadas forem de tipos e tamanhos muito diferentes e incompatíveis (por exemplo, tentar combinar um campo DATE com um campo INT), o comando UNION falhará.

Portanto, a afirmação de que o UNION funciona mesmo que as colunas dessas tabelas sejam de tipos e tamanhos diferentes está incorreta.

Errado.

013. (CESPE/CEBRASPE/DATAPREV/ANALISTA DE PROCESSAMENTO/2023) Em relação às linguagens de banco de dados SQL, DDL e DML, julgue o item a seguir.

Somente são possíveis os seguintes quatro tipos de JOIN em SQL, segundo o padrão ANSI: INNER JOIN, LEFT JOIN, RIGHT JOIN e CROSS JOIN.



O padrão ANSI SQL (a partir da revisão ANSI-92) especifica cinco tipos principais de JOIN, e não apenas quatro:

1. INNER JOIN (Junção Interna);
2. LEFT JOIN (ou LEFT OUTER JOIN) (Junção Externa Esquerda);
3. RIGHT JOIN (ou RIGHT OUTER JOIN) (Junção Externa Direita);
4. CROSS JOIN (Produto Cartesiano);
5. FULL JOIN (ou FULL OUTER JOIN) (Junção Externa Completa).

O tipo de JOIN que foi omitido na lista da questão é o FULL (OUTER) JOIN, que combina o resultado do LEFT JOIN e do RIGHT JOIN, retornando todos os registros de ambas as tabelas, com valores NULL onde não há correspondência.

Portanto, a afirmação de que *somente* são possíveis os quatro tipos listados está incorreta, pois o FULL OUTER JOIN também faz parte do padrão ANSI SQL.

Errado.

014. (CESPE/CEBRASPE/TELEBRAS/ESPECIALISTA EM GESTÃO DE TELECOMUNICAÇÕES/2022)

Julgue o seguinte item, pertinentes a bancos de dados.

Conforme os conceitos de SQL (ANSI), em uma expressão SQL o produto cartesiano resulta que algumas linhas da primeira tabela são unidas a todas as linhas da segunda tabela.



A descrição fornecida está incorreta sobre o funcionamento do Produto Cartesiano (obtido pelo CROSS JOIN em SQL, ou pela omissão da cláusula WHERE ou JOIN explícito entre duas tabelas no padrão SQL-89).

O Conceito de Produto Cartesiano:

O produto cartesiano entre duas tabelas (Tabela A e Tabela B) é uma operação que combina cada linha da primeira tabela com todas as linhas da segunda tabela.

- Afirmação incorreta: “algumas linhas da primeira tabela são unidas a todas as linhas da segunda tabela.”;
- Correta: todas as linhas da primeira tabela são unidas a todas as linhas da segunda tabela.

Se a Tabela A tiver ‘M’ linhas e a Tabela B tiver ‘N’ linhas, o produto cartesiano resultará em um novo conjunto com linhas.

Errado.

015. (FUNDATEC/CAU-RS/ANALISTA SUPERIOR/2023) Assinale a alternativa que corresponde ao resultado esperado pela execução do comando SQL abaixo:

```
SELECT *
INTO BKP_VENDAS
FROM VENDAS;
```

- Será criada uma nova tabela, nomeada como BKP_VENDAS, com todos os dados da tabela VENDAS
- Serão atualizados apenas os registros da tabela BKP_VENDAS que tiverem chave correspondente na tabela VENDAS

- c) A tabela VENDAS será renomeada para BKP_VENDAS
- d) Será criada uma nova tabela vazia, nomeada como BKP_VENDAS
- e) Serão copiados os dados da tabela BKP_VENDAS para a tabela VENDAS



Este comando é uma sintaxe específica, amplamente utilizada em ambientes como SQL Server (Transact-SQL) e Microsoft Access, que combina duas ações: a definição de dados (criação da tabela) e a manipulação de dados (inserção dos registros).

- a) Certa. Esta é exatamente a função do comando SELECT... INTO. Ele cria a nova tabela (BKP_VENDAS) com a estrutura da tabela de origem (VENDAS) e a popula com todos os registros (SELECT *).
- b) Errada. O comando SELECT INTO não é um comando de atualização (UPDATE). Além disso, BKP_VENDAS não existia antes da execução.
- c) Errada. O comando para renomear uma tabela é RENAME TABLE (ou sp_rename em SQL Server). O comando em questão cria uma *cópia*, não renomeia.
- d) Errada. A sintaxe que cria apenas a estrutura vazia, sem copiar os dados, seria SELECT * INTO BKP_VENDAS FROM VENDAS WHERE 1 = 0; (ou a sintaxe padrão DDL CREATE TABLE BKP_VENDAS LIKE VENDAS;). O comando original copia os dados.
- e) Errada. O fluxo da cópia é o inverso: os dados são copiados *da* VENDAS *para* a BKP_VENDAS (FROM VENDAS -> INTO BKP_VENDAS).

Letra a.

016. (INSTITUTO QUADRIX/CRM-MS/ANALISTA DE INFORMÁTICA/2025) Para excluir, por exemplo, o médico de id_medico = 5 da tabela Medicos, garantindo que não existam atendimentos vinculados a ele, podese usar o seguinte comando:

SQL

```
DELETE FROM Medicos WHERE id_medico = 5 AND NOT EXISTS (SELECT 1 FROM
Atendimentos WHERE id_medico = 5);
```



O comando busca deletar um registro da tabela Medicos somente se uma condição específica não for satisfeita em outra tabela (Atendimentos).

1. DELETE FROM Medicos: inicia a operação de remoção de registros da tabela Medicos.
2. WHERE id_medico = 5: filtra para considerar a exclusão apenas do médico com o id_medico igual a 5.
3. AND NOT EXISTS (...): esta é a condição principal de segurança. A exclusão só será realizada se o resultado da subconsulta for falso (ou seja, se a subconsulta não retornar nenhuma linha).

4. (SELECT 1 FROM Atendimentos WHERE id_medico = 5): a subconsulta verifica se há qualquer registro na tabela Atendimentos onde o id_medico seja 5. O SELECT 1 é uma prática comum para verificar a existência de linhas, pois é mais eficiente do que retornar dados reais. Portanto, o comando efetivamente exclui o médico de id_medico = 5 apenas se não houver nenhum registro correspondente na tabela Atendimentos, garantindo assim a integridade dos dados e evitando atendimentos “órfãos” (sem médico vinculado).

Certo.

017. (INÉDITA/2025) Julgue o item a seguir como CERTO ou ERRADO:

Assinale a opção em que é apresentada a palavra-chave, em SQL, que deve ser incluída em uma instrução de SELECT para evitar a apresentação de resultados duplicados da tabela.”

- a) UNIQUE
- b) NOTDUPLICATE
- c) NOTALL
- d) DISTINCT
- e) ONLY



- a) Errada. Usado em definições de esquema, não em SELECT.
- b) Errada. Não existe em SQL.
- c) Errada. Não é uma palavra-chave válida.
- d) Certa. Elimina duplicatas em SELECT.
- e) Errada. Não é usado para esse propósito.

Letra d.

018. (IADES/CRMV-PI/ANALISTA/ÁREA SUPORTE E GESTÃO DA TECNOLOGIA DA INFORMAÇÃO/2025) Considere um banco de dados relacional com uma tabela chamada “Funcionarios” que possui os campos: “id”, “nome”, “departamento” e “salario”. Para aumentar o salário de todos os funcionários do departamento “Vendas” em 10%, qual comando SQL deve ser utilizado?

- a) MODIFY Funcionarios SET salario = salario + 10 WHERE departamento = ‘Vendas’
- b) ALTER TABLE Funcionarios SET salario = salario * 1.1 WHERE departamento = ‘Vendas’
- c) UPDATE Funcionarios SET salario = salario * 1.1 WHERE departamento = ‘Vendas’
- d) INSERT INTO Funcionarios (salario) VALUES (salario * 1.1) WHERE departamento = ‘Vendas’
- e) SELECT salario * 1.1 FROM Funcionarios WHERE departamento = ‘Vendas’



- a) Errada. MODIFY não é o comando padrão SQL para atualização de dados (DML). O comando DML correto é UPDATE.
- b) Errada. ALTER TABLE é um comando DDL (Linguagem de Definição de Dados) usado para modificar a estrutura da tabela (ex: adicionar uma coluna), e não os dados (registros).
- c) Certa. O comando UPDATE é o DML correto para modificar registros. A cláusula SET define o novo valor (salario + 10\% ou salario * 1.1), e a cláusula WHERE filtra os registros a serem modificados (somente o departamento 'Vendas').
- d) Errada. INSERT INTO é usado para adicionar novos registros, e não para modificar registros existentes.
- e) Errada. SELECT apenas consulta e retorna os valores calculados (os salários aumentados em 10%), mas não modifica efetivamente os dados persistidos no banco de dados.

Letra c.

019. (VUNESP/TJ-RS/ANALISTA DE TECNOLOGIA DA INFORMAÇÃO/ÁREA ANÁLISE DE SISTEMAS/2023) Considere a seguinte tabela de um banco de dados relacional:

Aluno (ID, Nome, Curso)

O comando SQL para obter o nome e o curso dos alunos, com a condição de que o curso se inicie com a palavra Física é:

- a) `SELECT Nome, Curso`
`FROM Aluno`
`WHERE Curso == 'Física%';`
- b) `SELECT Nome, Curso`
`FROM Aluno`
`WHERE Curso.'Física%';`
- c) `SELECT Nome, Curso`
`FROM Aluno`
`WHERE Curso LIKE 'Física%';`
- d) `SELECT Nome, Curso`
`FROM Aluno`
`WHERE Curso('Física%');`
- e) `SELECT Nome, Curso`
`FROM Aluno`
`WHERE Curso = 'Física%';`



- a) Errada. O operador == não é o padrão SQL para comparação de igualdade (que é o =). Além disso, o operador LIKE e o caractere curinga (%) devem ser usados para busca por padrão.
- b) Errada. Esta sintaxe é inválida para operações de filtragem em SQL.
- c) Certa. O operador LIKE é o padrão SQL para busca por padrões. O caractere curinga % é usado para representar zero ou mais caracteres de qualquer tipo. Ao colocar 'Física%', a consulta filtra todos os registros onde o campo Curso começa com "Física" (ex: "Física", "Física Médica", "Física Quântica").
- d) Errada. Esta sintaxe é inválida para operações de filtragem em SQL.
- e) Errada. O operador de igualdade (=) busca uma correspondência exata. Ele só encontraria um registro se o curso fosse literalmente 'Física%', e não se o curso fosse "Física Médica". A busca por padrão requer o operador LIKE.

Letra c.

020. (INSTITUTO QUADRIX/CONSELHO REGIONAL DE ENFERMAGEM-PR/ANALISTA DE TECNOLOGIA DA INFORMAÇÃO/2024) Caso se deseje excluir da base de dados apenas o enfermeiro com ID 121, é suficiente executar o comando SQL a seguir:

SQL

```
DELETE * FROM Enfermeiro WHERE ID = 121;
```



A afirmação de que o comando `DELETE * FROM Enfermeiro WHERE ID = 121;` é suficiente para excluir apenas o enfermeiro com ID 121 da base de dados é Incorreta (F), devido a um erro de sintaxe.

Análise do comando:

- Objetivo: excluir registros condicionalmente (apenas o ID 121). A sintaxe correta para essa operação é o comando `DELETE`;
- Erro de sintaxe: o comando `DELETE` na sintaxe padrão SQL não utiliza o asterisco (*). O * é usado no comando `SELECT` (`SELECT * FROM...`) para indicar todas as colunas;
- Comando incorreto: `DELETE * FROM Enfermeiro WHERE ID = 121;`;
- Comando correto: `DELETE FROM Enfermeiro WHERE ID = 121;`;

Embora o comando fornecido tenha a intenção correta (usar o `DELETE` com o `WHERE` para restringir a exclusão a um único ID), a inclusão do * torna a execução do script inválida na maioria dos SGBDs padrão (como MySQL, PostgreSQL, SQL Server, Oracle).

Portanto, o comando como está escrito não é suficiente pois resultará em um erro de sintaxe.

Errado.

021. (FGV/TJ-SE/ANALISTA JUDICIÁRIO/2023) Considere uma tabela relacional TAB, com colunas A e B. A coluna A constitui a chave primária de TAB. A instância de TAB contém 100 linhas, e em todas as linhas o valor da coluna B é 10. Nesse contexto, analise o comando SQL a seguir.

```
select * from TAB t where not exists
(select * from TAB tt
where t.B = tt.B and t.A > tt.A)
```

Além da linha de títulos, o número de linhas produzidas pelo comando acima é:

- a) 0;
- b) 1;
- c) 98;
- d) 99;
- e) 100.



A questão apresentada envolve a análise de uma subconsulta correlacionada utilizando a cláusula NOT EXISTS e um filtro que depende da chave primária (A). Para determinar o número de linhas retornadas, precisamos entender qual condição a cláusula NOT EXISTS impõe.

Análise do contexto:

1. Tabela TAB: colunas A (Chave Primária, única) e B.
2. Dados: 100 linhas.
3. Coluna B: o valor de B é 10 em todas as 100 linhas.
4. Coluna A: é a Chave Primária, logo, possui 100 valores distintos. Assumindo que os valores são numéricos e ordenáveis (como é comum para chaves primárias), eles variam de um valor mínimo a um valor máximo.

Análise da consulta SQL

```
SQL
SELECT * FROM TAB t
WHERE NOT EXISTS (
SELECT * FROM TAB tt
WHERE t.B = tt.B AND t.A > tt.A
);
```

A consulta principal (SELECT * FROM TAB t) seleciona uma linha (t) da tabela TAB. A cláusula WHERE NOT EXISTS atua como um filtro:

O registro t (da tabela externa) será incluído no resultado SOMENTE SE a subconsulta NÃO retornar nenhuma linha para aquele registro t.

Análise da subconsulta:

A subconsulta busca por registros *tt* (da tabela interna) que satisfaçam as seguintes duas condições:

1. $t.B = tt.B$: a coluna B do registro interno (*tt*) deve ser igual à coluna B do registro externo (*t*). Como todos os valores de B são 10, essa condição é sempre verdadeira para qualquer par de linhas.
2. $t.A > tt.A$: a chave primária A do registro externo (*t*) deve ser maior que a chave primária A do registro interno (*tt*).

Juntando tudo, o WHERE NOT EXISTS é traduzido para:

“Selecione todas as linhas *t* onde NÃO EXISTE nenhuma outra linha *tt* na tabela onde a coluna A é menor que a coluna A de *t*.”

Em outras palavras, a condição NOT EXISTS só é verdadeira para a linha *t* que tem o menor valor na coluna A (a linha onde não existe nenhum outro valor *tt.A* menor).

O comando SQL busca, efetivamente, a linha com o valor mínimo da chave primária A.

Como a coluna A é a chave primária e todos os 100 valores são distintos, haverá exatamente **uma** linha onde a chave A é a menor de todas. Essa única linha fará com que a subconsulta retorne zero linhas, validando o NOT EXISTS.

O número de linhas produzidas pelo comando é **1**.

Letra b.

022. (FGV/TCE-PA/AUDITOR DE CONTROLE EXTERNO/ÁREA ADMINISTRATIVA/2024) Em um processo, o funcionário responsável foi substituído por outro funcionário da mesma organização. Tal substituição deve ser registrada no banco de dados do departamento de pessoal. A cláusula SQL mais indicada para implementar essa alteração, mantendo a integridade do banco, é denominada:

- a) ALTER
- b) INSERT
- c) JOIN
- d) UPDATE
- e) WITH



- a) Errada. (DDL) Usado para modificar a estrutura de um objeto (ex: adicionar uma coluna à tabela), e não o conteúdo dos registros.
- b) Errada. (DML) Usado para adicionar novos registros à tabela.
- c) Errada. (Cláusula SELECT) Usado para combinar dados de duas ou mais tabelas em uma consulta. Não é um comando de manipulação de dados.

d) Certa. (DML) Usado para modificar/atualizar os valores de registros existentes em uma tabela, o que se encaixa perfeitamente na substituição do funcionário responsável.

e) Errada. (Cláusula SELECT/DML em alguns casos) Usada para definir uma Common Table Expression (CTE) temporária, que auxilia em consultas ou comandos mais complexos, mas não é o comando principal de alteração de dados.

Letra d.

023. (CESPE/CEBRASPE/SEPLAN-RR/ANALISTA DE PLANEJAMENTO E ORÇAMENTO/2023) Com pertinência à linguagem SQL, julgue o item abaixo. Considere-se o seguinte script SQL.

```
select report_code, year, month, day, wind_speed, case
when wind_speed >= 40 then 'HIGH'
when wind_speed >= 30 then 'MODERATE' else 'LOW'
end as wind_severity from station_data
```

O resultado da execução do script resultará em erro, pois, caso haja, na tabela station_data, algum registro no campo wind_speed com valor superior a 40, não será possível prever se o valor da variável wind_severity será igual a 'HIGH'.



Vamos avaliar o comando:

```
select report_code, year, month, day, wind_speed,
-- seleciona os campos report_code, year, month, day e wind_speed
case -- percorre as condições
when wind_speed >= 40 then 'HIGH'
se o valor de wind_speed for maior ou igual a 40, então retorna 'HIGH'.
when wind_speed >= 30 then 'MODERATE'
se o valor de wind_speed for maior ou igual a 30, então retorna 'MODERATE'.
else 'LOW'
se nenhuma das condições anteriores for verdadeira, então retorna 'LOW'.
end as wind_severity
finaliza a cláusula case e atribui um apelido wind_severity para o resultado.
from station_data -- a partir da tabela station_data.
```

Em resumo, temos:

wind_speed	wind_severity
1 condição testada: Maior ou igual a 40	HIGH
2 condição testada: Maior ou igual a 30	MODERATE

wind_speed	wind_severity
Se não passou em nenhuma condição anterior	LOW

Portanto a assertiva é falsa, pois se houver na tabela um valor de wind_speed maior que 40, a variável wind_severity será HIGH, pois entrará na primeira condição do CASE.

Errado.

024. (CESPE/CEBRASPE/FINEP/ANALISTA/2024) Assinale a opção que apresenta a cláusula SQL que permite extrair dados em determinado intervalo.

- a) find in
- b) like
- c) alias
- d) between
- e) ranges



- a) Errada. Não é uma cláusula SQL padrão para extrair dados em um intervalo.
- b) Errada. É usada para busca de padrões em *strings* (textos), utilizando caracteres curinga (% , _), e não para definir um intervalo de valores.
- c) Errada. É usada para dar um nome temporário a uma coluna ou tabela (ex.: AS).
- d) Certa. É o operador SQL que define um intervalo, sendo inclusivo (inclui os limites superior e inferior). A sintaxe é WHERE coluna BETWEEN valor_inicial AND valor_final.
- e) Errada. Não é uma cláusula SQL padrão para definir um intervalo de valores.

Letra d.

025. (FCC/COPERGÁS/ANALISTA/2023) Em uma tabela chamada user de um banco de dados aberto e em condições ideais, para selecionar todos os registros que possuem nomes (campo nome) iniciados com a letra E e terminados com a letra I utiliza-se a instrução SQL SELECT * FROM user

- a) LIKE = 'E*I';
- b) WHERE nome = 'E*I';
- c) LIKE nome CONTAINS 'E*I';
- d) WHERE nome LIKE 'E*I';
- e) WHERE nome LIKE 'E%I';



A questão trata da forma correta de realizar uma busca por padrão em *strings* (textos) em SQL, utilizando caracteres curinga, o que é feito com o operador LIKE.

O objetivo é selecionar registros onde o campo nome:

1. Inicia com a letra 'E'.
2. Termina com a letra 'I'.
3. Pode ter zero ou mais caracteres entre 'E' e 'I'.

O caractere curinga padrão SQL que representa zero ou mais caracteres de qualquer tipo é o percentagem (%).

A instrução SQL correta utiliza a cláusula WHERE com o operador LIKE e o caractere curinga %:
WHERE nome LIKE 'E%I';

Letra e.

026. (FCC/PGE AM/TÉCNICO EM GESTÃO PROCURATORIAL/2022) Para excluir da tabela Advogado_Processo todas as linhas nas quais consta o valor 4378 no campo oabAdvogado, utiliza-se a instrução

- a) DELETE * FROM Advogado_Processo AS adp WHERE adp.oabAdvogado=4378;
- b) DELETE FROM Advogado_Processo (SELECT oabAdvogado=4378);
- c) DELETE * FROM Advogado_Processo WHERE oabAdvogado=4378;
- d) DELETE FROM Advogado_Processo WHERE oabAdvogado=4378;
- e) DELETE FROM Advogado_Processo (SELECT * FROM Advogado WHERE oabAdvogado=4378);



A questão pede o comando SQL correto para **excluir registros** de uma tabela com base em uma condição específica. O comando de exclusão em SQL é o DELETE.

O objetivo é: excluir da tabela Advogado_Processo todas as linhas onde o campo oabAdvogado tem o valor 4378.

A sintaxe correta para o comando DELETE com filtro condicional é: DELETE FROM nome_da_tabela WHERE condição;

A instrução SQL correta é:

DELETE FROM Advogado_Processo WHERE oabAdvogado=4378;

- a) Errada. A cláusula AS adp não é a forma padrão de se usar *alias* no comando DELETE (embora possa ser usada em alguns SGBDs que permitem JOIN no DELETE), e o * não é usado no DELETE.
- b) Errada. Sintaxe inválida.
- c) Errada. O * não deve ser usado no comando DELETE.
- d) Certa. É a sintaxe padrão e universalmente aceita para excluir linhas com base em uma condição.
- e) Errada. Sintaxe inválida. A remoção condicional é feita diretamente com a cláusula WHERE.

Letra d.

027. (CESPE/CEBRASPE/POLC-AL/PERITO CRIMINAL/2023) Com relação aos componentes de um computador, aos barramentos de E/S, à aritmética computacional e à linguagem SQL, julgue o próximo item.

Em SQL, para que não haja erro de construção (sintaxe), as cláusulas GROUP BY e HAVING, quando usadas, devem ser definidas sempre antes da cláusula WHERE.



As cláusulas GROUP BY e HAVING devem vir após a cláusula WHERE. A ordem correta segue a seguinte sintaxe:

```
SELECT colunas FROM nome_da_tabela WHERE condição GROUP BY coluna
HAVING condição;
```

Errado.

028. (IFSUL/IFSUL/PROFESSOR/ÁREA INFORMAÇÃO E COMUNICAÇÃO/2025) Os comandos SQL INSERT, DELETE e UPDATE são utilizados para manipular dados em um banco de dados. Com relação ao uso desses comandos, é INCORRETO afirmar que o comando:

- a) INSERT permite adicionar registros a uma tabela, independentemente das restrições de integridade impostas pelo banco de dados
- b) INSERT pode ser usado para adicionar múltiplos registros a uma tabela em um único comando
- c) DELETE, quando utilizado sem a cláusula WHERE, remove todos os registros de uma tabela, mas a definição da tabela permanece no banco de dados
- d) UPDATE permite modificar os valores dos atributos de vários registros em uma tabela, e a cláusula SET define os novos valores dos atributos



a) Errada. O comando INSERT NÃO pode ser executado se violar restrições de integridade (como PRIMARY KEY, FOREIGN KEY, NOT NULL, ou CHECK). O SGBD impedirá a inserção para manter a consistência dos dados.

b) Certa. Muitos SGBDs modernos suportam a sintaxe que permite a inserção de múltiplos conjuntos de valores (VALUES (v1, v2...), (v3, v4...),...) em um único comando INSERT.

c) Certa. O comando DELETE FROM Tabela; remove todas as linhas (registros) da tabela. No entanto, sendo um comando DML, ele não afeta a estrutura (definição, colunas, índices) da tabela.

d) Certa. O UPDATE é o comando usado para modificar dados existentes. A cláusula SET é obrigatória para atribuir os novos valores aos atributos (colunas), e a cláusula WHERE (se omitida) fará com que vários (todos) os registros sejam modificados.

Letra a.

029. (FGV/TJ-MS/TÉCNICO DE NÍVEL SUPERIOR/ÁREA WEB DESIGNER/2024) Observe o script SQL a seguir:

SQL

```
SELECT COUNT(*) AS [Quantidade], Tipo_Processo FROM Processo GROUP BY Tipo_Processo;
```

O resultado da execução desse script é:

- a) a lista dos registros da tabela quantidade
- b) a quantidade de processos por tipo
- c) a contagem dos registros da tabela de tipos de processos
- d) o agrupamento de processos que realizam contagem
- e) a contagem dos processos relacionados à quantidade de valores



Analisando o script SQL fornecido, que utiliza as cláusulas SELECT, COUNT(*), FROM e GROUP BY, podemos determinar sua função no contexto de gestão de dados:

- FROM Processo: indica a tabela de origem dos dados;
- GROUP BY Tipo_Processo: agrupa as linhas da tabela Processo com base nos valores únicos da coluna Tipo_Processo;
- COUNT(*) AS [Quantidade]: conta o número de linhas dentro de cada um desses grupos;
- SELECT... Tipo_Processo: exibe o valor do Tipo_Processo e a contagem de linhas (Quantidade) para aquele grupo.

O resultado final é uma tabela que lista cada tipo de processo único e o número total de processos existentes para cada um.

Portanto, o resultado da execução desse script é: **a quantidade de processos por tipo, que corresponde à resposta da letra b.**

Letra b.

030. (FGV/DPE-RS/TÉCNICO/2023) A programadora Fabiana elaborou um relatório com o nome do autor dos processos utilizando o seguinte comando SQL:

```
SELECT Autor FROM tab_processo
```

Ao ver o relatório gerado, o chefe da Fabiana solicitou um relatório contendo apenas os autores que possuem o primeiro nome Elizabeth. Contudo, Fabiana observou que havia diferentes grafias, como: Elisabeth, Elizabete etc.

No MySQL, para garantir que qualquer Autor cujo nome comece pelas letras “eli” sejam recuperados, Fabiana deve complementar o comando SQL com a cláusula WHERE Autor LIKE:

- a) 'eli%'
- b) '%eli%'

- c) 'eli_'
- d) '_eli'
- e) '_eli%'



A questão foca na necessidade de utilizar o operador de busca por padrão (LIKE) e o caractere curinga correto (%) para atender ao requisito de filtragem de nomes que começam com uma sequência específica de letras.

O objetivo é garantir que qualquer autor cujo nome **comece** pelas letras "eli" seja recuperado. Para isso, é necessário usar:

1. O operador LIKE.
2. A sequência fixa "eli" no início da *string*.
3. O caractere curinga % (que representa zero ou mais caracteres de qualquer tipo) imediatamente após "eli".

A cláusula WHERE correta é: WHERE Autor LIKE 'eli%'

Comentário das alternativas e curingas SQL:

Alternativa	Cláusula	Significado	Atende ao Requisito?
a)	'eli%'	Começa com 'eli' e é seguido por qualquer coisa (ex.: <i>Elizabeth, Eli, Eliane</i>).	Sim
b)	'%eli%'	Contém 'eli' em qualquer posição (ex.: <i>Teli, Nelio, Elizabeth</i>).	Não (Pega nomes que não começam com 'eli')
c)	'eli_'	Começa com 'eli' e é seguido por exatamente um caractere (o curinga _ representa um único caractere). (Ex.: <i>Eliz</i>).	Não (Não permite nomes longos)
d)	'_eli'	Termina com 'eli' e é precedido por exatamente um caractere (ex.: <i>Deli</i>).	Não (Não começa com 'eli')
e)	'_eli%'	Contém 'eli' a partir da segunda posição (ex.: <i>Seliab</i>).	Não (Não começa com 'eli')

Letra a.

031. (FGV/TJ-MS/TÉCNICO DE NÍVEL SUPERIOR/ÁREA WEB DESIGNER/2024) SQL (Structured Query Language) é uma linguagem usada para interagir com bancos de dados relacionais. Relativamente a SQL, analise as afirmativas a seguir e assinale (V) para a verdadeira e (F) para a falsa:

- () A cláusula WHERE só é utilizada associada ao comando SELECT
- () O comando CREATE TABLE cria uma nova tabela no banco de dados e o comando INSERT INTO permite popular essa tabela por meio da inserção de registros

- () O comando INNER JOIN é utilizado para unir os resultados de duas ou mais consultas SELECT e o seu resultado é um conjunto de linhas provenientes das tabelas envolvidas
- () Os comandos ALTER TABLE e UPDATE são comandos usados para atualizar registros em tabelas de um banco de dados
- () SQL suporta o conceito de chaves primárias e chaves estrangeiras para manter a integridade dos dados e as relações entre as tabelas

As afirmativas são, respectivamente:

- a) F – F – F – V – F
- b) F – V – F – F – V
- c) V – V – V – F – V
- d) V – F – V – V – V
- e) F – F – V – F – F



(F) A cláusula WHERE só é utilizada associada ao comando SELECT

A cláusula WHERE é essencialmente utilizada nos comandos de manipulação de dados DELETE e UPDATE para especificar quais registros serão afetados pela operação.

(V) O comando CREATE TABLE cria uma nova tabela no banco de dados e o comando INSERT INTO permite popular essa tabela por meio da inserção de registros

CREATE TABLE é um comando DDL (Data Definition Language) que define a estrutura da tabela, e INSERT INTO é um comando DML (Data Manipulation Language) que adiciona dados a essa estrutura.

(F) O comando INNER JOIN é utilizado para unir os resultados de duas ou mais consultas SELECT e o seu resultado é um conjunto de linhas provenientes das tabelas envolvidas
O INNER JOIN combina linhas de duas ou mais tabelas em uma consulta SELECT, retornando apenas os registros que possuem correspondência nas tabelas.

(F) Os comandos ALTER TABLE e UPDATE são comandos usados para atualizar registros em tabelas de um banco de dados

O comando UPDATE é usado para atualizar/modificar registros (dados nas linhas). O comando ALTER TABLE é usado para modificar a estrutura da tabela (ex: adicionar, remover ou alterar colunas).

(V) SQL suporta o conceito de chaves primárias e chaves estrangeiras para manter a integridade dos dados e as relações entre as tabelas

Chaves primárias (PRIMARY KEY) e chaves estrangeiras (FOREIGN KEY) são recursos fundamentais em bancos de dados relacionais e são usados para impor a integridade referencial.

Letra b.

categoria	modelo	valor
1	A	40
1	A	18
1	B	76
2	B	97

032. (CESPE/CEBRASPE/CNPQ/ANALISTA EM CIÊNCIA E TECNOLOGIA I/2024) Tendo como referência a tabela precedente, de nome dados, julgue o item subsequente, com relação à SQL. As expressões SQL a seguir, após serem executadas, gerarão o mesmo resultado.

```
insert into dados values (2, 'C', 40);
insert into dados (categoria, modelo, valor)
values (2, 'C', 40);
```



O comando INSERT INTO é o principal comando DML (Data Manipulation Language) utilizado para adicionar novos registros (linhas) a uma tabela em um banco de dados relacional. Existem duas sintaxes básicas para o comando, ambas válidas e equivalentes quando a inserção envolve todos os campos da tabela:

1. Sintaxe Completa (Recomendada): Especificação Explícita das Colunas

Esta sintaxe é considerada a melhor prática, pois explicita quais colunas estão recebendo os valores.

SQL

```
INSERT INTO nome_da_tabela (coluna1, coluna2, coluna3, ...)
VALUES (valor1, valor2, valor3, ...);
```

Neste formato, a lista de colunas (coluna1, coluna2,...) deve corresponder à lista de valores (valor1, valor2,...) em número e ordem.

Vantagem: permite que você insira dados em um subconjunto das colunas ou em ordem diferente da definição da tabela, além de tornar o script mais legível e robusto a alterações futuras na estrutura da tabela (DDL).

2. Sintaxe Simplificada (Implícita): Inserção de Registro Completo

Esta sintaxe pode ser usada quando você deseja inserir valores para todas as colunas da tabela.

SQL

```
INSERT INTO nome_da_tabela
VALUES (valor1, valor2, valor3, ...);
```

Ao utilizar este formato, é obrigatório que a lista de valores fornecida (valor1, valor2,...) corresponda rigorosamente à ordem e ao número total de colunas definidas na tabela.

Se a tabela tiver colunas com AUTO_INCREMENT ou valores padrão (DEFAULT), esta sintaxe exige que o valor correspondente seja fornecido (geralmente como NULL ou um valor explícito, dependendo da configuração do SGBD).

As duas sintaxes são equivalentes apenas quando a Sintaxe Simplificada insere um registro completo. A principal diferença reside no controle e na robustez: a primeira sintaxe (explícita) permite maior flexibilidade e garante que os dados sejam atribuídos às colunas corretas, mesmo que a ordem das colunas da tabela seja modificada posteriormente.

Certo.

033. (FGV/TJ-MS/TÉCNICO DE NÍVEL SUPERIOR/ÁREA WEB DESIGNER/2023) A linguagem SQL permite diversas operações sob um conjunto de dados de uma ou mais tabelas, as necessidades do projeto de software ditando quais cláusulas deverão ser utilizadas, sejam para recuperação, atualização, inserção ou remoção de algum dado. Ao lidarmos com recuperação, os dados que iremos recuperar podem ser filtrados, agrupados etc. Assinale a alternativa que apresenta qual a cláusula SQL responsável para filtrar registros em uma consulta de uma tabela:

- a) GROUP BY
- b) SELECT
- c) FILTER
- d) WHERE



- a) Errada. É usada para agrupar linhas que têm os mesmos valores em colunas especificadas, geralmente em conjunto com funções de agregação (COUNT, SUM).
- b) Errada. É o comando principal usado para recuperar dados e para especificar as colunas a serem exibidas.
- c) Errada. Não é a cláusula padrão SQL para filtrar registros de uma tabela. A cláusula WHERE é usada para esse propósito. (Em alguns contextos específicos, como em funções de agregação no PostgreSQL, FILTER é utilizado, mas não para a filtragem geral de linhas).
- d) Certa. Esta é a cláusula fundamental para filtrar registros com base em uma condição, aplicando-a antes do agrupamento (GROUP BY) ou retorno final dos dados.

Letra d.

034. (FGV/TJ-MS/TÉCNICO DE NÍVEL SUPERIOR/ÁREA WEB DESIGNER/2016) Considere um banco de dados criado no MySQL 5.7 com os seguintes comandos:

SQL

```
CREATE DATABASE controle; USE controle; CREATE TABLE PAGAMENTOS ( ID INT
PRIMARY KEY AUTO_INCREMENT, NOME VARCHAR(50), VALOR DOUBLE ); INSERT INTO
PAGAMENTOS(NOME,VALOR) VALUES ( 'Mary',1500); INSERT INTO PAGAMENTOS(NOME,VALOR)
VALUES ( 'Albert',300); INSERT INTO PAGAMENTOS(NOME,VALOR) VALUES ( 'Mary',450);
INSERT INTO PAGAMENTOS(NOME,VALOR) VALUES ( 'Lucas',3450); INSERT INTO
PAGAMENTOS(NOME,VALOR) VALUES ( 'Mary',180); INSERT INTO PAGAMENTOS(NOME,VALOR)
VALUES ( 'Mary',520); INSERT INTO PAGAMENTOS(NOME,VALOR) VALUES ( 'Albert',840);
INSERT INTO PAGAMENTOS(NOME,VALOR) VALUES ( 'Lucas',110); INSERT INTO
PAGAMENTOS(NOME,VALOR) VALUES ( 'Mary',3300);
```

Logo após, a seguinte consulta foi executada:

SQL

```
SELECT NOME, SUM(VALOR) AS TOTAL FROM PAGAMENTOS GROUP BY NOME HAVING TOTAL
> ( SELECT SUM(VALOR) / COUNT(DISTINCT NOME) FROM PAGAMENTOS );
```

Assinale a alternativa que contém o número de linhas retornadas pela consulta anterior.

- a) 0
- b) 1
- c) 2
- d) 3
- e) 9



Para determinar o número de linhas retornadas pela consulta, precisamos primeiro executar mentalmente a inserção de dados e, em seguida, analisar a subconsulta e a consulta principal.

1. Dados Inseridos na Tabela PAGAMENTOS:

A tabela PAGAMENTOS terá os seguintes registros (IDs auto-incrementados):

ID	NOME	VALOR
1	Mary	1500
2	Albert	300
3	Mary	450
4	Lucas	3450
5	Mary	180
6	Mary	520
7	Albert	840
8	Lucas	110
9	Mary	3300

2. Análise da Subconsulta (HAVING TOTAL > (...)):

A subconsulta calcula o valor que será usado como filtro.

- (Soma Total de Todos os Pagamentos):
- (Número de Nomes Únicos): os nomes únicos são: 'Mary', 'Albert', 'Lucas'. Portanto, o total é 3.
- Cálculo da Média de Valores Totais por Pessoa:

O valor de corte para o HAVING é 3550.

3. Análise da Consulta Principal:

Primeiro, precisamos calcular o (TOTAL) para cada NOME (GROUP BY NOME):

NOME	CÁLCULO	TOTAL
Mary	$\$1500 + 450 + 180 + 520 + 3300\$$	$\$ \mathbf{5950}$
Albert	$\$300 + 840\$$	$\$ \mathbf{1140}$
Lucas	$\$3450 + 110\$$	$\$ \mathbf{3560}$

Agora, aplicamos a condição do HAVING:

NOME	TOTAL	TOTAL > 3550	Resultado
Mary	5950	$\$5950 > 3550 \rightarrow \text{Verdadeiro}$	Incluir
Albert	1140	$\$1140 > 3550 \rightarrow \text{Falso}$	Excluir
Lucas	3560	$\$3560 > 3550 \rightarrow \text{Verdadeiro}$	Incluir

A consulta retornará duas linhas (Mary e Lucas). O número de linhas retornadas pela consulta é **2**, correspondendo à resposta da letra c.

Letra c.

035. (DIRENS AERONÁUTICA/SARGENTO DA AERONÁUTICA/ESPECIALIDADE INFORMÁTICA/2024) SQL (Structured Query Language) é uma linguagem usada para interagir com bancos de dados relacionais. Relativamente a SQL, analise as afirmativas a seguir e assinale a alternativa correta:

I – O comando SELECT é utilizado para recuperar dados de uma ou mais tabelas

II – O comando INSERT é utilizado para adicionar novas linhas a uma tabela

III – O comando UPDATE é utilizado para modificar dados existentes em uma tabela

IV – O comando DELETE é utilizado para remover linhas de uma tabela

- Apenas as afirmativas I e II estão corretas
- Apenas as afirmativas I, II e III estão corretas
- Apenas as afirmativas I, III e IV estão corretas

- d) Apenas as afirmativas II, III e IV estão corretas
- e) Todas as afirmativas estão corretas



- I – Certa. O SELECT é o comando principal para consultar e recuperar dados.
- II – Certa. O INSERT é o comando para introduzir novos registros no banco de dados.
- III – Certa. O UPDATE é o comando utilizado para alterar os valores de colunas em registros já existentes.
- IV – Certa. O DELETE é o comando utilizado para apagar registros de uma tabela.

Letra e.

036. (FGV/TJ-MS/TÉCNICO DE NÍVEL SUPERIOR/2024) Observe script SQL a seguir.

```
SELECT COUNT(*) AS [Quantidade], Tipo_Processo
FROM Processo
GROUP BY Tipo_Processo;
```

O resultado da execução desse script é:

- a) a lista dos registros da tabela quantidade;
- b) a quantidade de processos por tipo;
- c) a contagem dos registros da tabela de tipos de processos;
- d) o agrupamento de processos que realizam contagem;
- e) a contagem dos processos relacionados à quantidade de valores.



Este *script* é uma consulta de agregação padrão em SQL, que é fundamental para sumarizar dados, algo que pode ser muito útil em análises de gestão ou forenses.

O *script* é:

```
SQL
SELECT COUNT(*) AS [Quantidade], Tipo_Processo
FROM Processo
GROUP BY Tipo_Processo;
```

Análise da consulta:

1. FROM Processo: define a tabela de origem.
2. GROUP BY Tipo_Processo: agrupa todas as linhas da tabela Processo que têm o mesmo valor na coluna Tipo_Processo.
3. COUNT(*) AS [Quantidade]: conta o número de linhas (registros) dentro de cada grupo criado.

4. SELECT... Tipo_Processo: exibe o nome do grupo (Tipo_Processo) junto com a contagem (Quantidade) para aquele grupo.

O resultado final é uma listagem que mostra quantos processos existem para cada tipo diferente.

Portanto, o resultado da execução desse script é: a quantidade de processos por tipo.

Letra b.

037. (DIRENSAERONÁUTICA/SARGENTO DAAERONÁUTICA/ESPECIALIDADE INFORMÁTICA/2024)

Em SQL, qual comando é utilizado para atualizar registros existentes em uma tabela?

- a) INSERT
- b) DELETE
- c) SELECT
- d) UPDATE
- e) CREATE



- a) Errada. INSERT> Usado para adicionar novos registros.
- b) Errada. DELETE> Usado para remover registros existentes.
- c) Errada. SELECT> Usado para consultar/recuperar dados.
- d) Certa. UPDATE> Usado para modificar valores de registros existentes.
- e) Errada. CREATE> Usado para criar a estrutura de objetos (DDL).

Letra d.

038. (DIRENSAERONÁUTICA/SARGENTO DAAERONÁUTICA/ESPECIALIDADE INFORMÁTICA/2024)

Qual comando SQL DML é utilizado para remover uma ou mais linhas de uma tabela?

- a) TRUNCATE
- b) DROP
- c) DELETE
- d) REMOVE
- e) ERASE



- a) Errada. TRUNCATE é um comando DDL (Data Definition Language). Ele remove *todas* as linhas de uma tabela, geralmente de forma mais rápida que o DELETE, e não permite a cláusula WHERE.
- b) Errada. DROP é um comando DDL. Ele remove a estrutura inteira da tabela e seus dados.

- c) Certa. É um comando DML que permite remover linhas seletivamente (com WHERE) ou todas as linhas (sem WHERE).
- d) Errada. REMOVE não é um comando SQL padrão para remoção de dados de uma tabela.
- e) Errada. ERASE não é um comando SQL padrão para remoção de dados de uma tabela.

Letra c.

039. (CESPE/CEBRASPE/PETROBRAS/PROFISSIONAL PETROBRAS DE NÍVEL SUPERIOR/2022) Julgue o item abaixo, a respeito da linguagem SQL.

O comando delete alunos permite apagar uma tabela de nome alunos.



O comando DELETE não apaga a tabela, mas sim os dados de uma tabela. Para apagar a tabela, deve-se usar o comando DROP.

Errado.

040. (CESPE/CEBRASPE/CNPQ/ANALISTA EM CIÊNCIA E TECNOLOGIA I/2024) Acerca de técnicas de modelagem de BI (business intelligence), de big data e de linguagem de manipulação de dados (DML), julgue o item que se segue.

A DML não procedimental exige que um usuário especifique quais dados são necessários sem especificar como obtê-los.



A linguagem SQL (Structured Query Language), que é a principal Linguagem de Manipulação de Dados (DML) utilizada em bancos de dados relacionais, é classificada como uma linguagem declarativa ou não procedimental (também chamada de não processual).

- Linguagem Não Procedimental (ou Declarativa): o usuário apenas especifica o que deseja (os dados que são necessários) sem precisar detalhar o “como” (a sequência de passos ou o algoritmo que o sistema deve seguir para recuperar os dados). O otimizador do banco de dados se encarrega de determinar o caminho de acesso mais eficiente;

EXEMPLO

O comando SELECT Nome FROM Alunos WHERE Curso = 'Direito' apenas declara a condição e o resultado esperado.

- Linguagem Procedimental: o usuário especifica tanto o que deseja quanto o “como”, ou seja, detalha a sequência de operações para alcançar o resultado.

Portanto, a DML não procedimental (como o SQL) exige que o usuário especifique quais dados são necessários sem especificar como obtê-los.

Certo.

041. (FCC/TRT 12ª REGIÃO/ANALISTA JUDICIÁRIO/2023) Considere uma tabela denominada Cidadao e as colunas Nome_Cidadao e Valor_Recebido.

Para obter a média dos valores maiores que 200 recebidos pelos cidadãos, um Analista deve utilizar o comando

- a) `SELECT AVG (Valor_Recebido > 200) FROM Cidadao;`
- b) `SELECT AVG (Valor_Recebido) FROM Cidadao WHERE Valor_Recebido > 200;`
- c) `SELECT AVG (Valor_Recebido) > 200 FROM Cidadao;`
- d) `SELECT FROM Cidadao AVG (Valor_Recebido) > 200;`
- e) `SELECT Nome Cidadao and AVG (Valor_Recebido) FROM Cidadao WHERE Valor_Recebido > 200;`



a) Errada. A expressão `Valor_Recebido > 200` é uma condição booleana que resulta em TRUE ou FALSE. A função AVG espera uma coluna numérica como argumento, não uma expressão booleana. Tentar calcular a média de uma condição como `Valor_Recebido > 200` resultará em um erro de sintaxe no SQL, pois não é possível calcular a média de valores booleanos.

b) Certa. Vamos analisar:

- `SELECT AVG(Valor_Recebido)`: a função AVG calcula a média dos valores da coluna Valor_Recebido;
- `FROM Cidadao`: especifica a tabela Cidadao como fonte dos dados;
- `WHERE Valor_Recebido > 200`: filtra os registros para incluir apenas aqueles onde Valor_Recebido é maior que 200, antes de calcular a média.

Essa construção atende exatamente ao requisito da questão, que é calcular a média dos valores maiores que 200.

EXEMPLO

Se a tabela Cidadao tiver valores como [100, 300, 400, 150, 250], a cláusula `WHERE Valor_Recebido > 200` seleciona [300, 400, 250], e a média será $(300 + 400 + 250) / 3 = 316.67$.

c) Errada. A expressão `AVG(Valor_Recebido) > 200` implica que a média calculada será comparada com 200, retornando um valor booleano (TRUE ou FALSE), não a média em si. A questão pede a **média dos valores maiores que 200**, não uma verificação se a média é maior que 200. Além disso, essa sintaxe não responde ao objetivo da questão.

d) Errada. A ordem das cláusulas em uma consulta SQL segue a estrutura SELECT... FROM... WHERE.... Colocar AVG(Valor_Recebido) > 200 após o FROM não é válido. Além disso, como na alternativa c, a expressão AVG(Valor_Recebido) > 200 retornaria um booleano, não a média solicitada.

e) Errada. Essa alternativa contém erros de sintaxe e semântica:

- Erro de sintaxe: a expressão Nome Cidadao é inválida. A coluna correta é Nome_Cidadao, e não há espaço ou palavra-chave Cidadao solta. Além disso, a palavra and não é usada para separar colunas no SELECT; as colunas são separadas por vírgulas (,);
- Erro semântico: mesmo corrigindo a sintaxe para SELECT Nome_Cidadao, AVG(Valor_Recebido), a consulta não seria válida para o propósito da questão, pois incluir Nome_Cidadao exigiria uma cláusula GROUP BY Nome_Cidadao para que AVG funcione corretamente (caso contrário, o SQL retornará um erro em bancos de dados padrão, como PostgreSQL ou SQL Server). A questão pede apenas a média, não uma média por cidadão.

Letra b.

042. (CESPE/CEBRASPE/MPU/ANALISTA DE INFORMÁTICA/2010) Julgue o item seguinte acerca de DML (data manipulation language) e DDL (data definition language).

As DML devem ser utilizadas incorporadas a outras linguagens, chamadas de linguagens hospedeiras, uma vez que sentenças escritas em DML devem estar embutidas em código escrito em uma linguagem hospedeira para que possam ser executadas.



- DML como Linguagem Interativa e Standalone (Autônoma): a DML (principalmente o SQL) foi desenvolvida para ser usada de forma interativa e autônoma por usuários, administradores e desenvolvedores diretamente em ambientes de terminal, ferramentas de cliente SQL ou interfaces de banco de dados. Um comando simples como SELECT * FROM Tabela; ou INSERT INTO Tabela... pode ser executado diretamente sem a necessidade de uma linguagem hospedeira;
- Linguagem Hospedeira (Embedded SQL): o conceito de “linguagens hospedeiras” (como C++, Java ou Python) é utilizado quando o código DML (SQL) é embutido (embedded) dentro de um programa aplicativo. Isso permite que o aplicativo manipule dados do banco. Neste caso, o DML está sendo usado de forma procedimental dentro da aplicação, mas isso é apenas uma das formas de uso, e não um requisito obrigatório para a execução do DML;
- DML Não Procedimental: o SQL é uma DML não procedimental/declarativa, o que significa que ela pode ser executada sem um código procedimental detalhando as etapas.

O SQL (DML) pode ser executado de forma autônoma e interativa. A incorporação em linguagens hospedeiras é uma opção para o desenvolvimento de aplicações, mas não é um requisito de execução.

Errado.

043. (FCC/SEGEP-MA/ANALISTA EXECUTIVO/PROGRAMADOR DE SISTEMAS/2018) Um Programador de Sistemas recuperou todos os dados dos países Brasil, Argentina e Peru gravados no campo Pais da tabela Cliente, abaixo especificada:

Tabela Cliente:

```
IdCliente NomeCliente Endereco CEP
Cidade SiglaUF Pais
```

A sintaxe SQL correta que ele usou para realizar essa atividade foi `SELECT * FROM Cliente`

- a) `WHERE Pais IN ('Argentina', 'Peru', 'Brasil');`
- b) `WHEN Pais = ('Argentina', 'Peru', 'Brasil');`
- c) `WHERE Pais = ('Argentina', 'Peru', 'Brasil');`
- d) `WHEN Pais IN ('Argentina', 'Peru', 'Brasil');`
- e) `WHERE Pais BETWEEN ('Argentina', 'Peru', 'Brasil');`



A questão pede o comando SQL correto para selecionar registros com base em uma lista de valores possíveis para uma determinada coluna (Pais).

Para verificar se um campo corresponde a qualquer valor em uma lista, o operador SQL correto é o `IN`.

A sintaxe SQL correta que o Programador de Sistemas usou para recuperar todos os dados dos clientes dos países Brasil, Argentina e Peru é:

```
WHERE Pais IN ('Argentina', 'Peru', 'Brasil');
```

- a) Certa. A cláusula `WHERE` especifica a condição, e o operador `IN` compara o valor do campo Pais com cada valor listado entre parênteses.
- b) Errada. A palavra-chave `WHEN` é usada na estrutura `CASE` e não na cláusula `WHERE`. Além disso, o operador `=` não funciona com uma lista de valores.
- c) Errada. O operador `=` (igualdade) só compara com um único valor e não funciona com uma lista.
- d) Errada. Usa a palavra-chave `WHEN` incorretamente.
- e) Errada. O operador `BETWEEN` é usado para definir um intervalo fechado (geralmente numérico ou de data) e não para testar a inclusão em uma lista de *strings* não sequenciais.

Letra a.

044. (INÉDITA/2025) Considere a seguinte consulta SQL:

SQL

```
SELECT DISTINCT Nome, Pais FROM Clientes WHERE Cidade LIKE 'M%' ORDER BY Pais DESC;
```

Sobre esta consulta, é CORRETO afirmar que:

- a) Retorna nomes e países únicos de clientes de cidades que terminam com 'M'
- b) Retorna nomes e países de clientes de cidades que começam com 'M', eliminando duplicatas e ordenando por país decrescente
- c) A cláusula DISTINCT não funcionará corretamente pois há duas colunas selecionadas
- d) O ORDER BY deve vir antes do WHERE para funcionar corretamente
- e) O operador LIKE está sendo usado incorretamente



- WHERE Cidade LIKE 'M%': filtra cidades que começam com 'M';
- SELECT DISTINCT Nome, Pais: retorna combinações únicas de nome e país;
- ORDER BY Pais DESC: ordena por país em ordem decrescente;
- DISTINCT funciona corretamente com múltiplas colunas, considerando a combinação de valores.

a) Errada. A alternativa afirma que a consulta retorna nomes e países de clientes de cidades que terminam com 'M'. No entanto, o padrão LIKE 'M%' significa que a cidade começa com 'M', não que termina com 'M'. Para cidades que terminam com 'M', o padrão correto seria LIKE '%M'.

b) Certa. Essa alternativa descreve exatamente o comportamento da consulta.

Vamos decompor:

- "Nomes e países de clientes de cidades que começam com 'M'": correto, pois WHERE Cidade LIKE 'M%' filtra cidades que começam com 'M';
- "Eliminando duplicatas": correto, pois DISTINCT garante que as combinações de Nome e Pais sejam únicas;
- "Ordenando por país decrescente": correto, pois ORDER BY Pais DESC ordena os resultados pela coluna Pais em ordem decrescente.

c) Errada. A cláusula DISTINCT funciona perfeitamente com múltiplas colunas. Quando usada com mais de uma coluna (como SELECT DISTINCT Nome, Pais), ela elimina duplicatas com base na combinação única das colunas selecionadas. Por exemplo, se duas linhas têm o mesmo Nome e Pais, apenas uma será retornada. Não há limitação técnica que impeça DISTINCT de funcionar com múltiplas colunas.

d) Errada. Na sintaxe do SQL, a ordem das cláusulas em uma consulta SELECT é fixa: SELECT, FROM, WHERE, ORDER BY (entre outras, como GROUP BY ou HAVING, se aplicáveis). A cláusula

ORDER BY sempre vem após a cláusula WHERE, pois primeiro os dados são filtrados (WHERE) e depois ordenados (ORDER BY). Colocar ORDER BY antes de WHERE resultaria em um erro de sintaxe.

e) Errada. O operador LIKE está sendo usado corretamente na consulta: Cidade LIKE 'M%'. O padrão 'M%' é válido e significa que a coluna Cidade deve começar com 'M' seguido por qualquer sequência de caracteres. Não há erro de sintaxe ou semântica nesse uso. Por exemplo, cidades como "Madrid" ou "Miami" seriam incluídas, enquanto "Boston" ou "Salem" não seriam.

Letra b.

045. (FUNDATEC/CAU-RS/ANALISTA SUPERIOR/2023) Na linguagem SQL, um alias para uma coluna ou tabela pode ser criado utilizando a palavra-chave:

- a) HOW
- b) AS
- c) FOR
- d) ON
- e) LIKE



- a) Errada. Não é uma palavra-chave SQL para criação de alias.
- b) Certa. É o padrão para dar um alias a uma coluna (SELECT nome_coluna AS novo_nome) ou a uma tabela (FROM nome_tabela AS alias_tabela).
- c) Errada. Pode ser usada em contextos específicos de laços ou cursores em algumas extensões SQL procedurais (como PL/SQL), mas não para criar um alias de consulta.
- d) Errada. Usada para especificar a condição de junção entre tabelas (JOIN... ON condição).
- e) Errada. Usada na cláusula WHERE para busca de padrões em strings.

Letra b.

046. (CESPE/CEBRASPE/AGER-MT/ANALISTA REGULADOR/2023) Em linguagem de manipulação de dados DML, o operador SQL BETWEEN serve para

- a) delimitar o valor de uma coluna na cláusula WHERE.
- b) delimitar as colunas a serem apresentadas na cláusula WHERE.
- c) delimitar os limites de um campo para a cláusula INSERT.
- d) restringir a quantidade de linhas a serem recuperadas na cláusula UPDATE.
- e) restringir a quantidade de campos na cláusula DELETE.



O operador SQL BETWEEN pertence à Linguagem de Manipulação de Dados (DML), pois é usado na cláusula WHERE para filtrar dados.

Sua função é definir um intervalo de valores, incluindo os limites (inicial e final), para a filtragem de dados em uma consulta ou comando DML.

a) Certa. O operador BETWEEN é usado na cláusula WHERE para especificar um intervalo de valores para uma única coluna, limitando assim os registros que satisfazem a condição. Ex.: WHERE Preço BETWEEN 10 AND 50.

b) Errada. A cláusula WHERE filtra linhas. As colunas a serem apresentadas são definidas no SELECT.

c) Errada. O INSERT é usado para adicionar dados, não para delimitar valores.

d) Errada. O UPDATE usa o WHERE para restringir as linhas a serem *modificadas*. O BETWEEN não restringe a *quantidade* em si, mas sim quais linhas são selecionadas pela condição de intervalo.

e) Errada. O DELETE remove linhas inteiras, e o BETWEEN é usado no WHERE para restringir as linhas afetadas, não a quantidade de campos.

Letra a.

047. (CESPE/CEBRASPE/TCE-SC/ANALISTA DE CONTAS PÚBLICAS/2022) Acerca do conceito de view, da modelagem dimensional, do modelo de referência CRISP- DM e da linguagem SQL, julgue o item subsequente.

O comando a seguir tem a finalidade de mostrar o nome e o email de todos os procuradores que recebem salários acima de R\$ 30.000,00.

```
SELECT nome, email
FROM procurador
WHEN salario >= 30.000,00;
```



A cláusula correta para definir uma condição no comando é WHERE e não WHEN. WHEN existe, porém é usada para condições dentro da instrução CASE.

O operador utilizado é **>=**, então na verdade, o comando retorna o nome e e-mail dos procuradores com salário MAIORES OU IGUAIS a 30.000,00 e não apenas acima desse valor. Por isso, a questão está errada.

Errado.

048. (INÉDITA/2025) Julgue o item a seguir como CERTO ou ERRADO:

Em uma consulta SQL, a cláusula FROM corresponde à seleção do predicado que envolve atributos da relação determinada pela cláusula SELECT.



A cláusula FROM indica a(s) tabela(s) de onde os dados serão consultados. Os atributos são definidos na cláusula SELECT. O predicado (condição) é definido na cláusula WHERE. A questão confunde as funções das diferentes cláusulas do comando SELECT.

Errado.

049. (CESPE/CEBRASPE/CTI/TECNOLOGISTA JÚNIOR/2024) Considerando as linguagens e os fundamentos de bancos de dados relacionais, julgue o item subsequente.

No DML (data manipulation language), a instrução TRUNCATE elimina todas as linhas de uma tabela simultaneamente, enquanto a instrução DELETE oferece a possibilidade de excluir dados específicos ou todos os dados.



As descrições dos comandos estão corretas, porém TRUNCATE é um comando da DDL e não da DML.

Errado.

050. (VUNESP/TJM-SP/ANALISTA DE BANCO DE DADOS JUDICIÁRIO/2025) Considere a linguagem SQL no que se refere a comandos para manipulação de dados (DML). Suponha uma tabela de um banco de dados relacional, denominada Tab1, que contenha os campos Cod e Nome. Assinale a alternativa que indica o comando para atualizar o campo Cod para o valor 25, apenas para valores do campo Nome diferentes da string Teste:

- a) UPDATE Tab1 Cod = 25 HAVING Nome <> 'Teste'
- b) UPDATE Tab1 MAKE Cod = 25 FOR Nome <> 'Teste'
- c) UPDATE Tab1 SET Cod = 25 WHERE Nome <> 'Teste'
- d) UPDATE Tab1 SET Cod = 25 IN CASE Nome <> 'Teste'
- e) UPDATE Tab1 USE Cod = 25 AT Nome <> 'Teste'



a) Errada. HAVING é usado para filtrar grupos (GROUP BY), e não registros individuais em comandos DML. A sintaxe de atribuição também está incorreta.

b) Errada. As palavras-chave MAKE e FOR não fazem parte da sintaxe padrão do UPDATE.

c) Certa. Esta é a sintaxe padrão do SQL: UPDATE (tabela) SET (coluna = novo valor) WHERE (condição de filtro). O operador <> (ou!=) significa "diferente de".

- d) Errada. A sintaxe IN CASE não é utilizada para essa finalidade no comando UPDATE.
- e) Errada. As palavras-chave USE e AT não fazem parte da sintaxe padrão do UPDATE.

Letra c.

REFERÊNCIAS

DEVMEDIA. SQL: EXISTS. Disponível em: <https://www.devmedia.com.br/sql-exists/41176>. Acesso em: 08 out. 2025.

DEVMEDIA. SQL: Utilizando o Operador UNION e UNION ALL. Disponível em: <https://www.devmedia.com.br/sql-utilizando-o-operador-union-e-union-all/37457>. Acesso em: 11 out. 2025.

LEARNSQL. SQL Basics Cheat Sheet. Disponível em: <https://learnsql.com/blog/sql-basics-cheat-sheet/>. Acesso em: 09 fev. 2025.

MICROSOFT. O que é o Power Query?. Microsoft Learn, 05 fev. 2025. Disponível em: <https://learn.microsoft.com/pt-br/power-query>. Acesso em: 11 out. 2025.

SQL JOINS VENN DIAGRAM: A Visual Guide. Red9, 16 maio 2025. Disponível em: <https://red9.com/blog/sql-joins-venn-diagram/>. Acesso em: 11 out. 2025.

SQL JOINS. w3resource, 2025. Disponível em: <https://www.w3resource.com/sql/joins>. Acesso em: 02 out. 2025.

W3SCHOOLS. SQL Tutorial. Disponível em: <https://www.w3schools.com/sql/>. Acesso em: 02 out. 2025.

Abra



caminhos



crie

futuros

gran.com.br

